150 questions

d'entretien technique

expliquées simplement, avec un schéma — du niveau Junior au niveau Senior

150 interview questions, explained simply · Junior to Senior · FR / EN

- Junior · 56
- Confirmé · 67
- Senior · 27

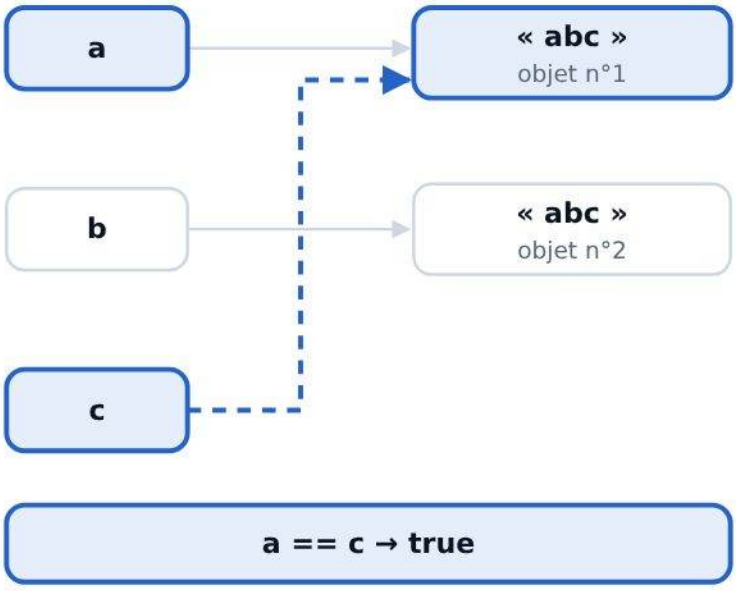
Java	29 questions
Spring Boot	28 questions
JPA / Hibernate	17 questions
SQL / PostgreSQL	19 questions
Angular	25 questions
RxJS	16 questions
REST, Git, Docker	16 questions

Rached Chakchouk

Ingénieur Full Stack Java / Angular · rached-chakchouk.vercel.app

== ou equals

== or equals

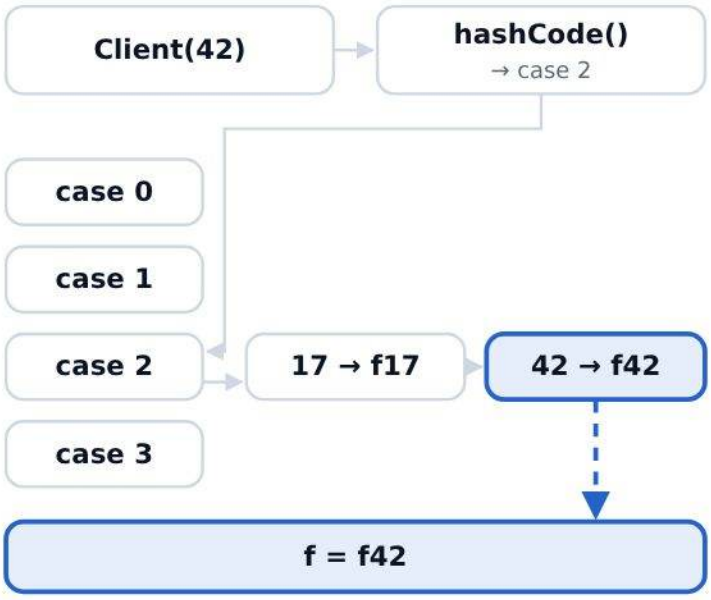


La réponse, étape par étape · *Step-by-step answer*

1. a pointe vers un premier objet « abc » en mémoire.
   a points to a first "abc" object in memory.
2. new crée un deuxième objet, même s'il contient le même texte.
   new creates a second object, even if it contains the same text.
3. == compare les adresses : ce sont deux objets différents, donc false.
   == compares addresses: these are two different objects, so false.
4. equals compare le contenu : « abc » = « abc », donc true.
   equals compares content: "abc" = "abc", so true.
5. c pointe vers le même objet que a : cette fois, == renvoie true.
   c points to the same object as a: this time, == returns true.

hashCode et equals

hashCode and equals

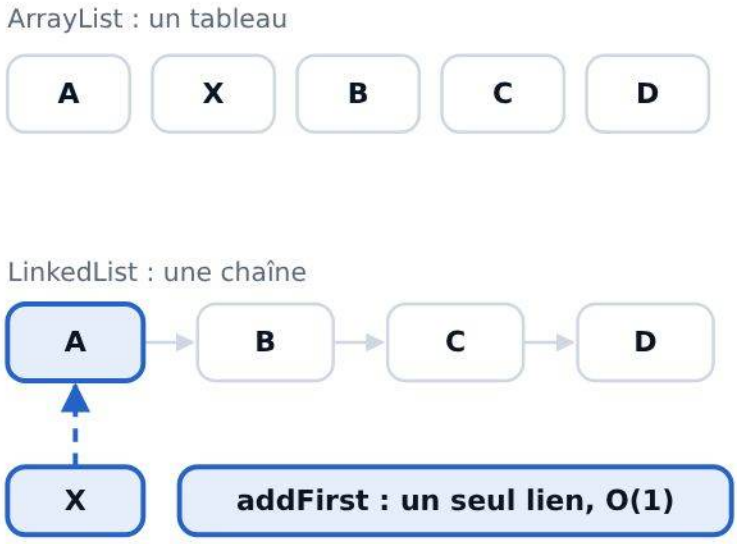


La réponse, étape par étape · *Step-by-step answer*

1. Une HashMap, c'est comme une armoire avec des tiroirs (les « cases »). Ici, 4 tiroirs vides.
   A HashMap is like a cabinet with drawers (the "buckets"). Here, 4 empty drawers.
2. On range le client 17. hashCode() donne le numéro du tiroir : le 2. On y met sa facture.
   We store customer 17. hashCode() gives the drawer number: 2. We put their invoice there.
3. On range le client 42. Lui aussi va dans le tiroir 2. Pas grave : un tiroir peut contenir plusieurs éléments.
   We store customer 42. It also goes into drawer 2. No problem: a drawer can hold several items.
4. On cherche le client 42. hashCode() dit « tiroir 2 ». Java ouvre directement ce tiroir, sans regarder les autres.
   We look for customer 42. hashCode() says "drawer 2". Java opens that drawer directly, without checking the others.
5. Dans le tiroir, equals() compare : est-ce le client 42 ? Le 17 ? Non.
   Inside the drawer, equals() compares: is it customer 42? The 17? No.
6. Le suivant, le 42 ? Oui, on l'a trouvé.
   The next one, the 42? Yes, we found it.
7. Résultat : f42. À retenir : deux objets égaux avec equals() doivent avoir le même hashCode(). Sinon, Java ouvre le mauvais tiroir.
   Result: f42. Remember: two objects that are equal with equals() must have the same hashCode(). Otherwise, Java opens the wrong drawer.

ArrayList ou LinkedList

ArrayList or LinkedList

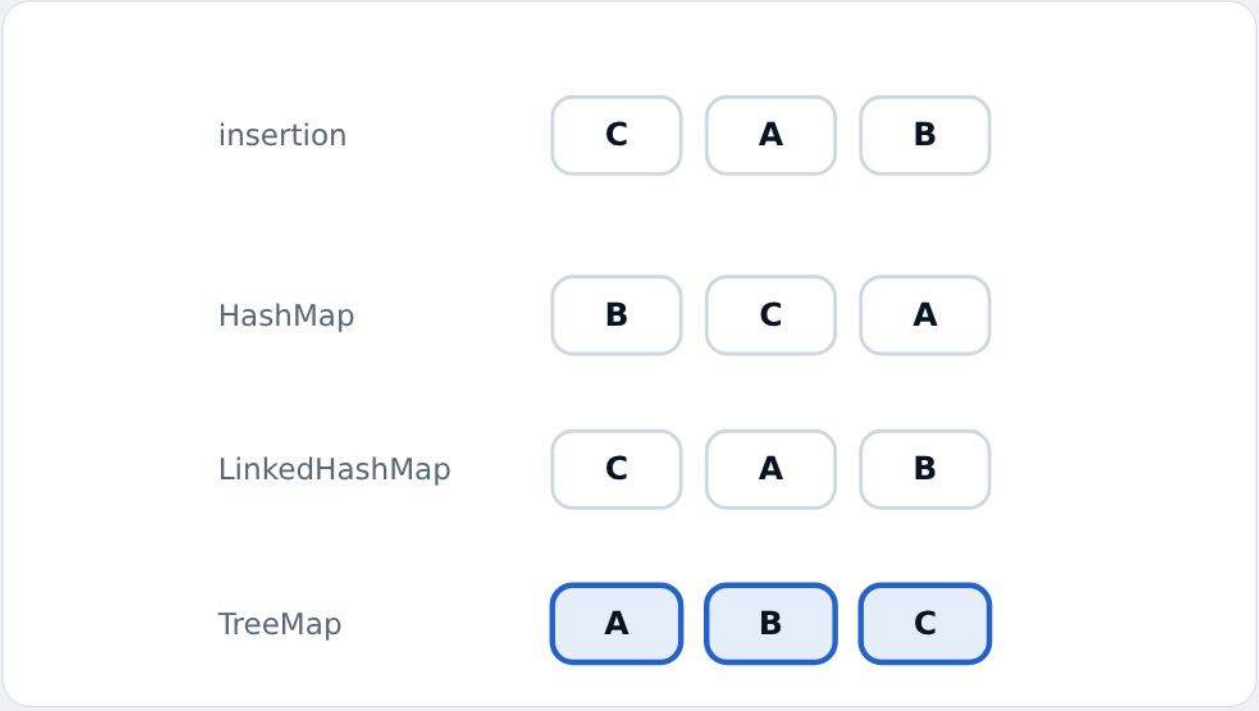


La réponse, étape par étape · *Step-by-step answer*

1. ArrayList range les éléments côte à côte, dans un tableau.
   ArrayList stores elements side by side, in an array.
2. Pour lire la case 2, Java calcule directement sa position : très rapide.
   To read slot 2, Java calculates its position directly: very fast.
3. Insérer au milieu oblige à décaler tous les éléments suivants : lent sur une grande liste.
   Inserting in the middle means shifting all the following elements: slow on a large list.
4. LinkedList : chaque élément connaît le suivant, comme les maillons d'une chaîne.
   LinkedList: each element knows the next one, like the links of a chain.
5. Pour lire l'élément 2, il faut suivre la chaîne depuis le début.
   To read element 2, you have to follow the chain from the start.
6. Ajouter en tête : on crée un maillon et on change un lien. Rien à décaler.
   Adding at the front: we create a link and change one pointer. Nothing to shift.

Les 3 Map

The 3 Maps

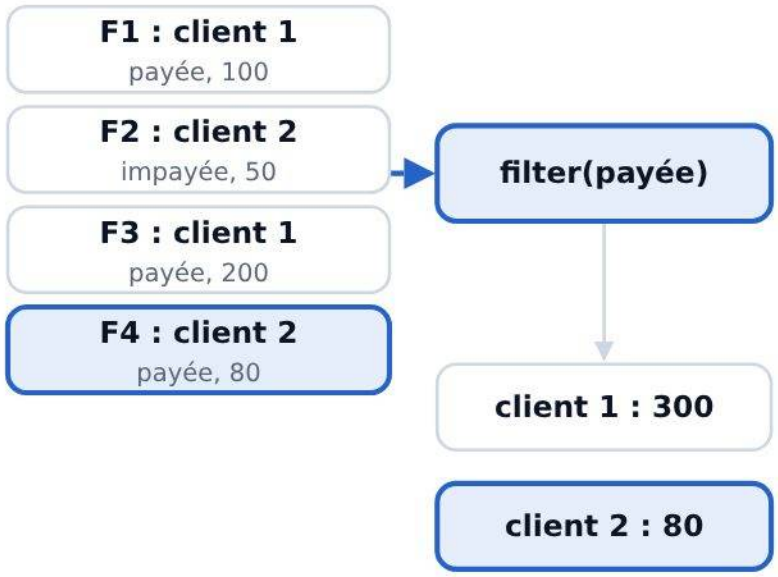


La réponse, étape par étape · *Step-by-step answer*

1. On ajoute C, puis A, puis B.
   We add C, then A, then B.
2. HashMap range selon le hashCode : l'ordre de parcours n'est pas garanti. C'est la plus rapide, O(1).
   HashMap stores by hashCode: iteration order is not guaranteed. It is the fastest, O(1).
3. LinkedHashMap garde l'ordre d'ajout : C, A, B.
   LinkedHashMap keeps insertion order: C, A, B.
4. TreeMap trie les clés : A, B, C. Un peu plus lente, O(log n).
   TreeMap sorts the keys: A, B, C. A bit slower, O(log n).

Streams

Streams

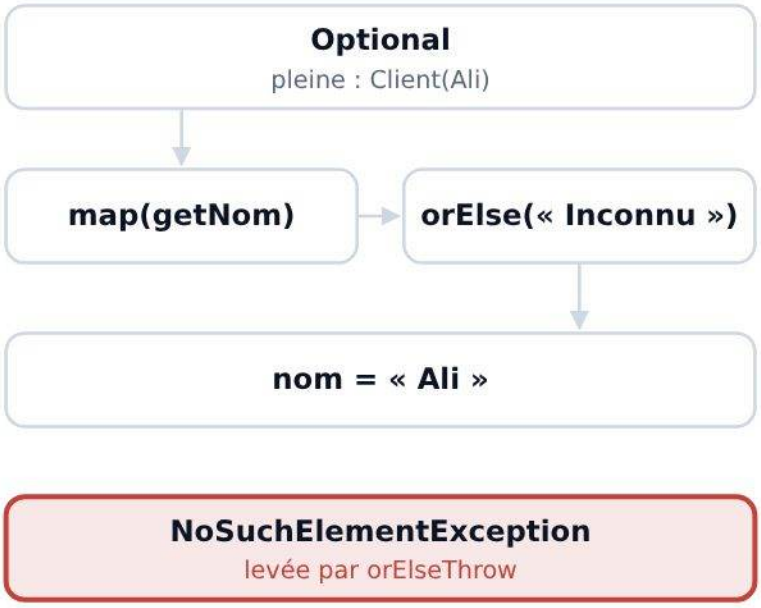


La réponse, étape par étape · *Step-by-step answer*

1. On prépare le flux des 4 factures. Rien ne s'exécute encore.
   We prepare the stream of 4 invoices. Nothing runs yet.
2. filter est déclaré, mais toujours rien : les opérations intermédiaires attendent.
   filter is declared, but still nothing: intermediate operations wait.
3. collect lance tout : F1 est payée, elle passe et s'ajoute au total du client 1.
   collect starts everything: F1 is paid, it passes and is added to customer 1's total.
4. F2 est impayée : le filtre la bloque.
   F2 is unpaid: the filter blocks it.
5. F3 est payée : le client 1 passe à 300.
   F3 is paid: customer 1 goes up to 300.
6. F4 est payée : client 2 = 80. Résultat final : une Map client → total.
   F4 is paid: customer 2 = 80. Final result: a Map customer → total.

Optional

Optional

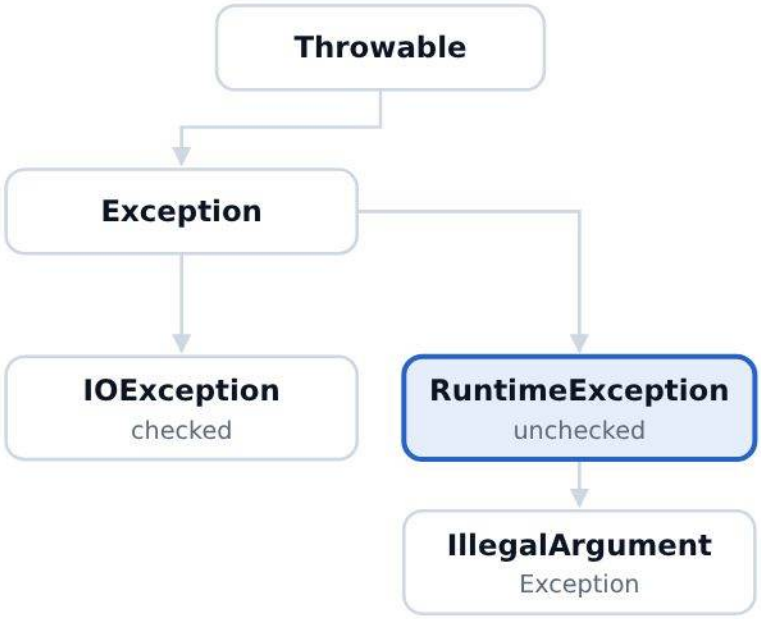


La réponse, étape par étape · *Step-by-step answer*

1. findById renvoie une boîte : pleine si le client existe, vide sinon. Jamais null.
   findById returns a box: full if the customer exists, empty otherwise. Never null.
2. La boîte est vide : map ne fait rien et transmet la boîte vide.
   The box is empty: map does nothing and passes the empty box along.
3. orElse donne une valeur par défaut quand la boîte est vide. Pas de NullPointerException.
   orElse gives a default value when the box is empty. No NullPointerException.
4. Si le client existait, map aurait sorti son nom : « Ali ».
   If the customer had existed, map would have taken out their name: "Ali".
5. orElseThrow lève une exception si la boîte est vide : utile quand l'absence est une vraie erreur.
   orElseThrow throws an exception if the box is empty: useful when absence is a real error.

Exceptions

Exceptions



La réponse, étape par étape · *Step-by-step answer*

1. IOException est checked : le compilateur t'oblige à la déclarer (throws) ou à la capturer (try/catch).
   IOException is checked: the compiler forces you to declare it (throws) or catch it (try/catch).
2. IllegalArgumentException hérite de RuntimeException : unchecked, aucune obligation. Elle signale une erreur de programmation ou de données.
   IllegalArgumentException extends RuntimeException: unchecked, no obligation. It signals a programming or data error.
3. À retenir pour Spring : @Transactional annule automatiquement (rollback) seulement sur les unchecked.
   Remember for Spring: @Transactional automatically rolls back only on unchecked exceptions.

BigDecimal

BigDecimal

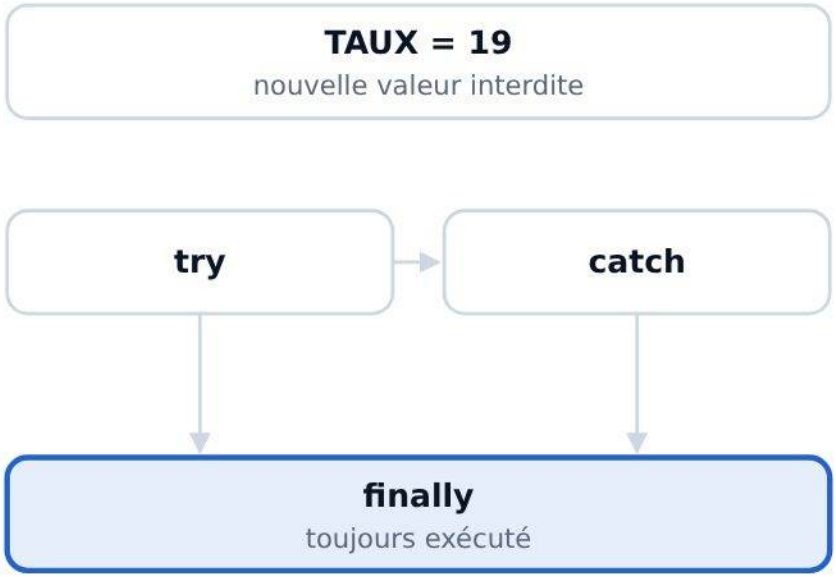


La réponse, étape par étape · *Step-by-step answer*

1. double stocke les nombres en binaire : 0,1 n’est pas exact. Sur des montants, ces petites erreurs s’accumulent.
   double stores numbers in binary: 0.1 is not exact. On amounts, these small errors add up.
2. BigDecimal calcule exactement. Astuce : le créer à partir d’un texte ("0.1"), pas d’un double.
   BigDecimal calculates exactly. Tip: create it from a string ("0.1"), not from a double.
3. On calcule la TVA à 19 %.
   We calculate VAT at 19%.
4. setScale(3, HALF_UP) arrondit au millime, comme le dinar tunisien.
   setScale(3, HALF_UP) rounds to the millime, like the Tunisian dinar.

final, finally, finalize

final, finally, finalize

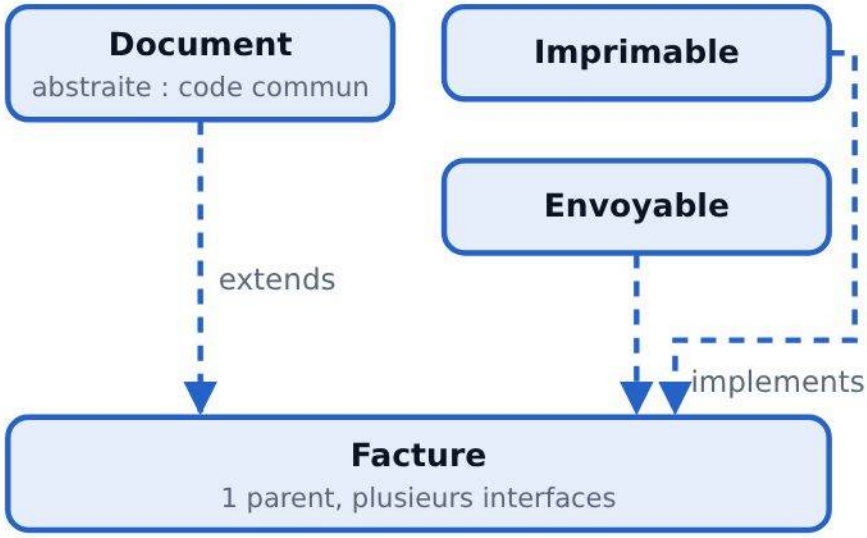


La réponse, étape par étape · *Step-by-step answer*

1. final : la variable ne peut recevoir une valeur qu'une seule fois.
   final: the variable can be assigned a value only once.
2. Essayer de la changer donne une erreur dès la compilation.
   Trying to change it gives an error at compile time.
3. Cas 1 : pas d'erreur. Le bloc finally s'exécute ensuite.
   Case 1: no error. The finally block runs afterwards.
4. Cas 2 : une exception. catch la traite, puis finally s'exécute quand même : la connexion est toujours fermée.
   Case 2: an exception. catch handles it, then finally runs anyway: the connection is always closed.
5. finalize() : vieille méthode appelée par le garbage collector, dépréciée. Aujourd'hui, on utilise try-with-resources.
   finalize(): an old method called by the garbage collector, deprecated. Today, we use try-with-resources.

Abstraite ou interface

Abstract class or interface

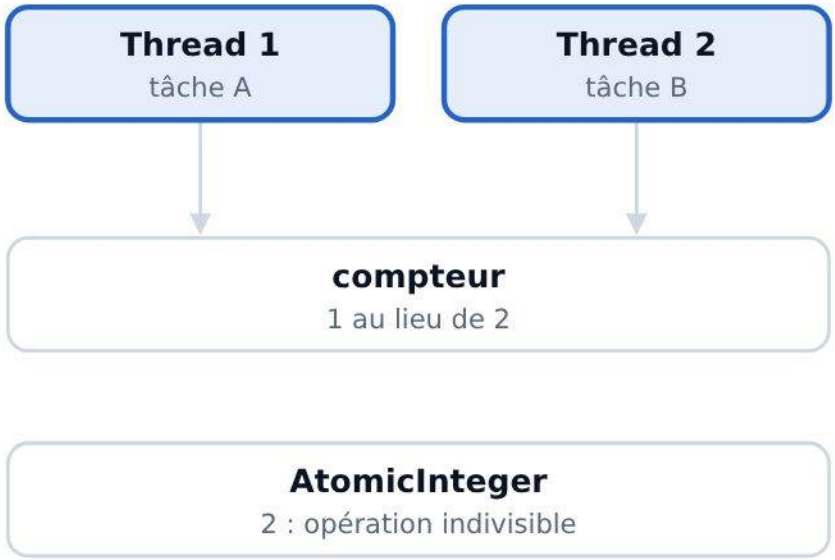


La réponse, étape par étape · *Step-by-step answer*

1. La classe abstraite partage des données (numero) et du code (totalTtc). On ne peut pas créer un Document directement.
   The abstract class shares data (numero) and code (totalTtc). You cannot create a Document directly.
2. Les interfaces décrivent des capacités : savoir s'imprimer, savoir s'envoyer.
   Interfaces describe abilities: being able to print, being able to send.
3. Facture hérite d'une seule classe, mais peut implémenter plusieurs interfaces.
   Facture extends only one class, but can implement several interfaces.

Concurrence

Concurrency

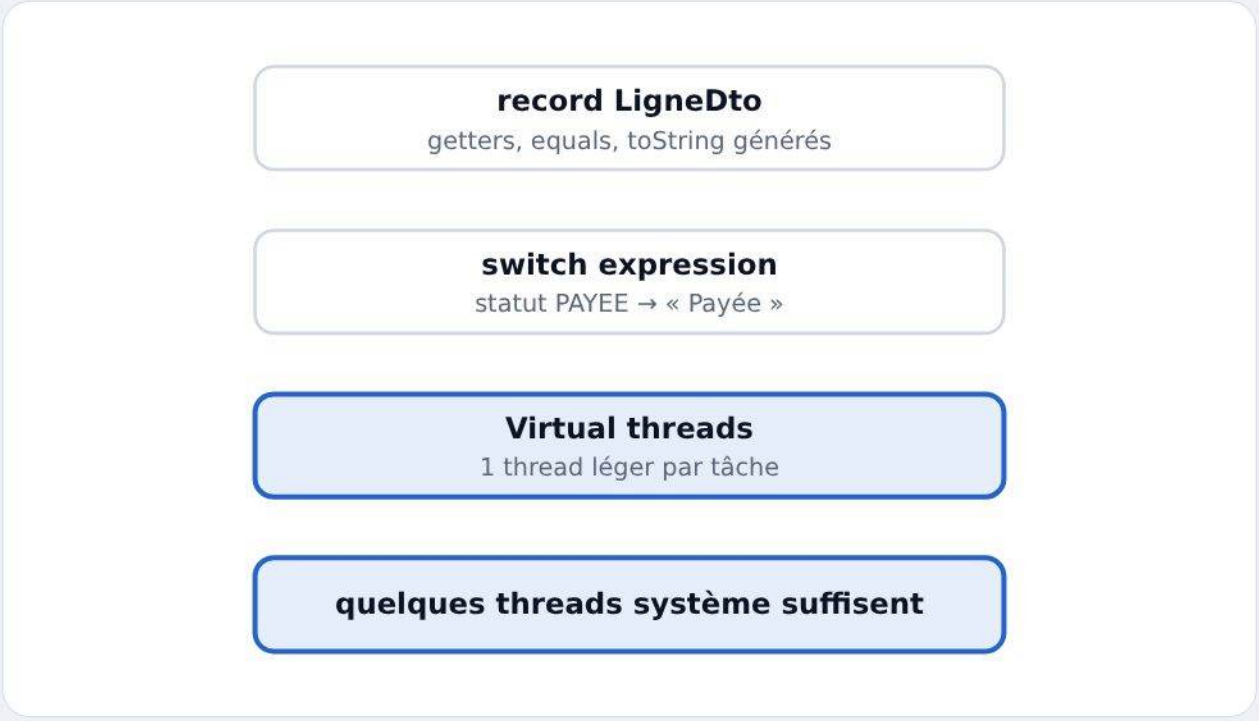


La réponse, étape par étape · *Step-by-step answer*

1. Thread 1 lit compteur : 0.
   Thread 1 reads compteur: 0.
2. Au même moment, Thread 2 lit aussi 0.
   At the same moment, Thread 2 also reads 0.
3. Les deux écrivent 1 : un ajout est perdu. C'est une « race condition ».
   Both write 1: one increment is lost. This is a "race condition".
4. AtomicInteger (ou synchronized) rend l'opération indivisible : le résultat est bien 2.
   AtomicInteger (or synchronized) makes the operation indivisible: the result is correctly 2.
5. ExecutorService réutilise un groupe de threads au lieu d'en créer un nouveau à chaque tâche.
   ExecutorService reuses a pool of threads instead of creating a new one for each task.

Java 17 et 21

Java 17 and 21

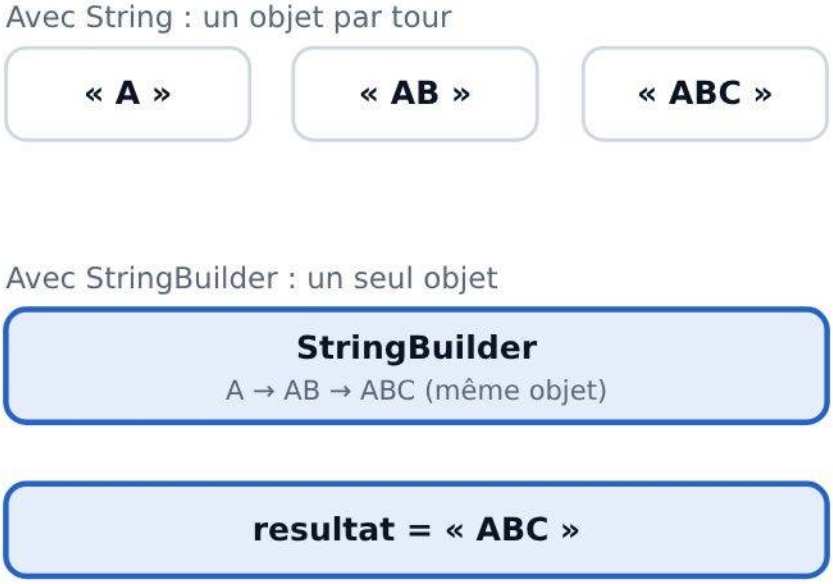


La réponse, étape par étape · *Step-by-step answer*

1. Java 17 : record écrit en une ligne une classe de données qui ne change pas.
   Java 17: record writes an immutable data class in one line.
2. switch devient une expression : il renvoie directement une valeur, sans break.
   switch becomes an expression: it returns a value directly, without break.
3. Java 21 : des milliers de threads virtuels très légers. Idéal pour beaucoup d'attentes (mails, appels HTTP).
   Java 21: thousands of very lightweight virtual threads. Ideal for lots of waiting (emails, HTTP calls).

StringBuilder

StringBuilder

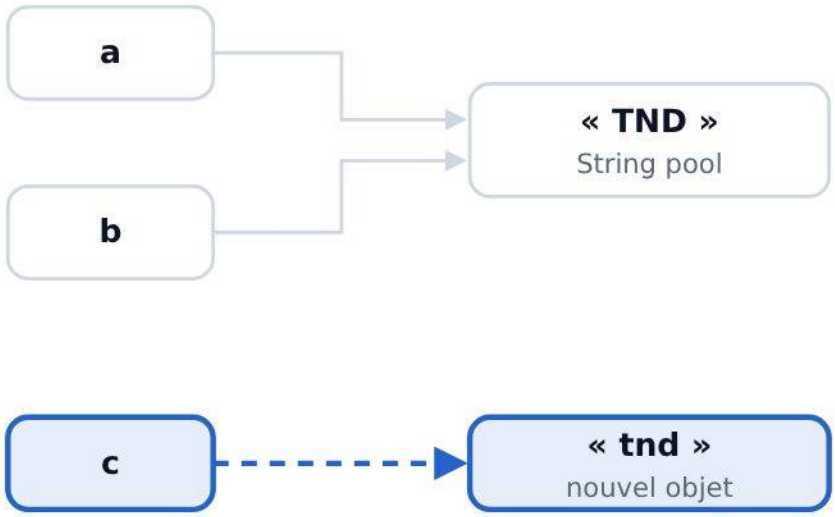


La réponse, étape par étape · *Step-by-step answer*

1. Tour 1 : la chaîne vaut « A ».
   Round 1: the string is "A".
2. Tour 2 : un nouvel objet « AB » est créé, l'ancien est jeté.
   Round 2: a new object "AB" is created, the old one is thrown away.
3. Tour 3 : encore un nouvel objet. Avec 10 000 lignes : 10 000 objets jetés.
   Round 3: yet another new object. With 10,000 lines: 10,000 objects thrown away.
4. StringBuilder modifie toujours le même objet.
   StringBuilder always modifies the same object.
5. toString() crée la chaîne finale une seule fois.
   toString() creates the final string only once.

String immuable

Immutable String

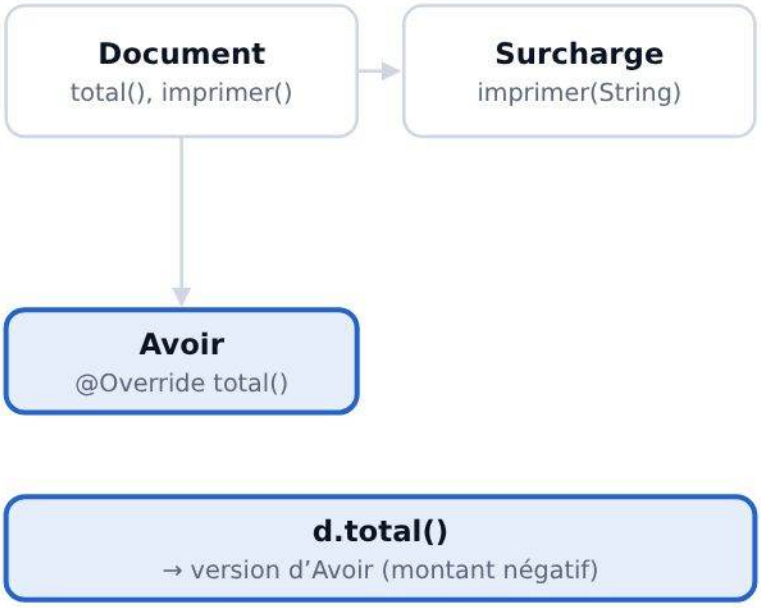


La réponse, étape par étape · *Step-by-step answer*

1. « TND » est rangé dans le String pool, une zone de chaînes partagées.
   "TND" is stored in the String pool, an area of shared strings.
2. b réutilise le même objet. C'est possible uniquement parce qu'une String ne peut pas changer.
   b reuses the same object. This is only possible because a String cannot change.
3. toLowerCase ne modifie pas « TND » : il crée un nouvel objet. a et b ne sont jamais touchés.
   toLowerCase does not modify "TND": it creates a new object. a and b are never touched.

Surcharge ou redéfinition

Overloading or overriding



La réponse, étape par étape · *Step-by-step answer*

1. Surcharge : deux méthodes imprimer avec des paramètres différents. Java choisit à la compilation selon les arguments.
   Overloading: two imprimer methods with different parameters. Java chooses at compile time based on the arguments.
2. Redéfinition : Avoir remplace total() de Document, avec la même signature.
   Overriding: Avoir replaces total() from Document, with the same signature.
3. Le choix se fait à l'exécution, selon l'objet réel : c'est le polymorphisme.
   The choice is made at runtime, based on the real object: this is polymorphism.

4 piliers POO

4 pillars of OOP

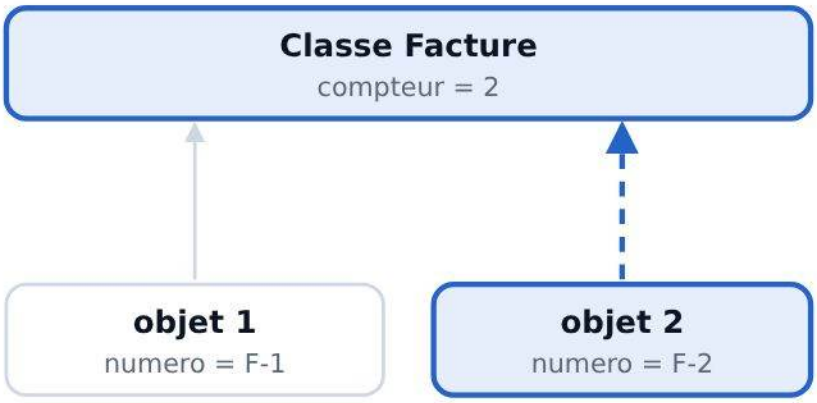


La réponse, étape par étape · *Step-by-step answer*

1. Abstraction : Document dit ce qu'il faut faire (total), sans dire comment.
   Abstraction: Document says what to do (total), without saying how.
2. Encapsulation : ht est caché (private). On y accède par getHt().
   Encapsulation: ht is hidden (private). We access it through getHt().
3. Héritage : Facture reprend tout ce que Document possède.
   Inheritance: Facture takes everything Document has.
4. Polymorphisme : d est déclaré Document, mais c'est la version de Facture qui s'exécute.
   Polymorphism: d is declared as Document, but it is Facture's version that runs.

static

static



La réponse, étape par étape · *Step-by-step answer*

1. compteur est static : il existe une seule fois, dans la classe.
   compteur is static: it exists only once, in the class.
2. Premier objet : le compteur passe à 1, l'objet reçoit F-1.
   First object: the counter goes to 1, the object gets F-1.
3. Chaque objet a son propre numero, mais tous partagent le même compteur.
   Each object has its own numero, but they all share the same counter.

Comparable / Comparator

Comparable / Comparator



La réponse, étape par étape · *Step-by-step answer*

1. Au départ, la liste n’est pas triée.
   At the start, the list is not sorted.
2. Comparable : l’ordre « naturel » est écrit dans la classe. Ici, par numéro.
   Comparable: the "natural" order is written in the class. Here, by number.
3. Comparator : un autre ordre, défini à l’extérieur. Ici, du total le plus grand au plus petit.
   Comparator: another order, defined outside. Here, from the largest total to the smallest.

Pile et tas

Stack and heap



La réponse, étape par étape · *Step-by-step answer*

1. Appel de creer() : un cadre est ajouté sur la pile.
   Call to creer(): a frame is added to the stack.
2. quantite est une variable simple : elle est rangée directement dans la pile.
   quantite is a simple variable: it is stored directly on the stack.
3. new crée l'objet dans le tas. La pile garde seulement une référence vers lui.
   new creates the object on the heap. The stack only keeps a reference to it.
4. Fin de la méthode : le cadre disparaît de la pile.
   End of the method: the frame disappears from the stack.
5. Plus personne ne pointe vers la Facture : le garbage collector libère la mémoire.
   Nothing points to the Facture anymore: the garbage collector frees the memory.

SOLID

SOLID

S : une seule responsabilité
FactureService valide, PdfService génère

O : ouvert à l'extension
nouvelle taxe = nouvelle classe

L : remplaçable par l'enfant
un Avoir peut remplacer un Document

I : petites interfaces
chacune son rôle

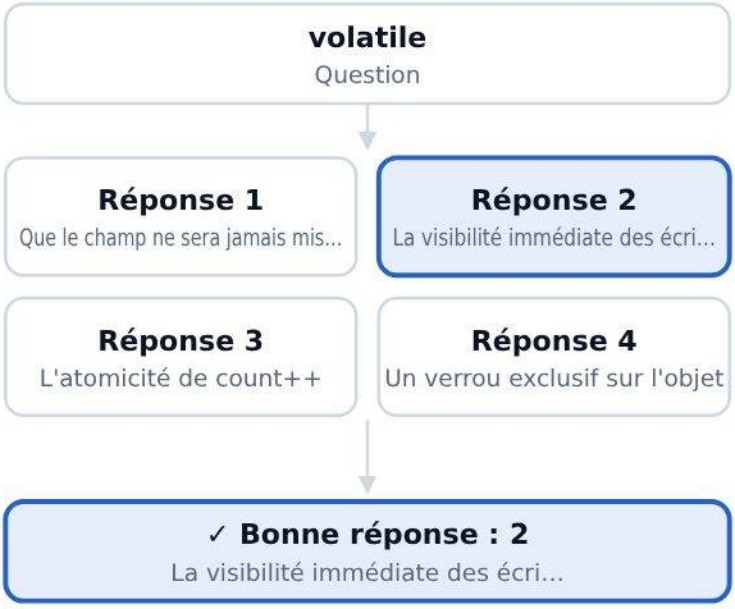
D : dépendre d'interfaces
CalculTaxe, pas TaxeTva

La réponse, étape par étape · *Step-by-step answer*

1. S : chaque classe fait une seule chose. Valider et générer un PDF sont séparés.
   S: each class does only one thing. Validating and generating a PDF are separate.
2. O : pour une nouvelle taxe, on ajoute une classe. On ne modifie pas le code qui marche.
   O: for a new tax, we add a class. We do not modify code that works.
3. L : partout où on attend un Document, un Avoir doit fonctionner aussi.
   L: wherever a Document is expected, an Avoir must work too.
4. I : plusieurs petites interfaces plutôt qu'une grosse qui oblige à tout implémenter.
   I: several small interfaces rather than one big one that forces you to implement everything.
5. D : le service reçoit une interface. Spring injecte l'implémentation : facile à changer et à tester.
   D: the service receives an interface. Spring injects the implementation: easy to change and to test.

volatile

volatile

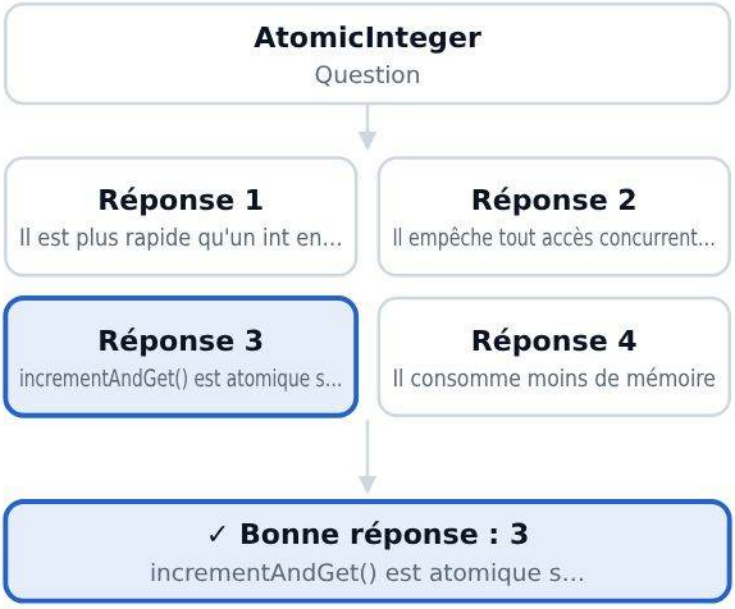


La réponse, étape par étape · *Step-by-step answer*

1. La question : Que garantit le mot-clé volatile sur un champ partagé entre threads ?
   The question: What does the volatile keyword guarantee on a field shared between threads?
2. Les réponses possibles : 1. Que le champ ne sera jamais mis en cache par la JVM · 2. La visibilité immédiate des écritures pour les autres threads · 3. L'atomicité de count++ · 4. Un verrou exclusif sur l'objet
   Possible answers: 1. The JVM never caches the field · 2. Writes are immediately visible to other threads · 3. count++ becomes atomic · 4. An exclusive lock on the object
3. La bonne réponse est la 2 : La visibilité immédiate des écritures pour les autres threads.
   The correct answer is 2: Writes are immediately visible to other threads.
4. volatile garantit la visibilité (et l'ordre) des lectures/écritures, pas l'atomicité : count++ reste une lecture + écriture non atomique.
   volatile guarantees visibility (and ordering) of reads/writes, not atomicity: count++ is still a non-atomic read + write.

AtomicInteger

AtomicInteger

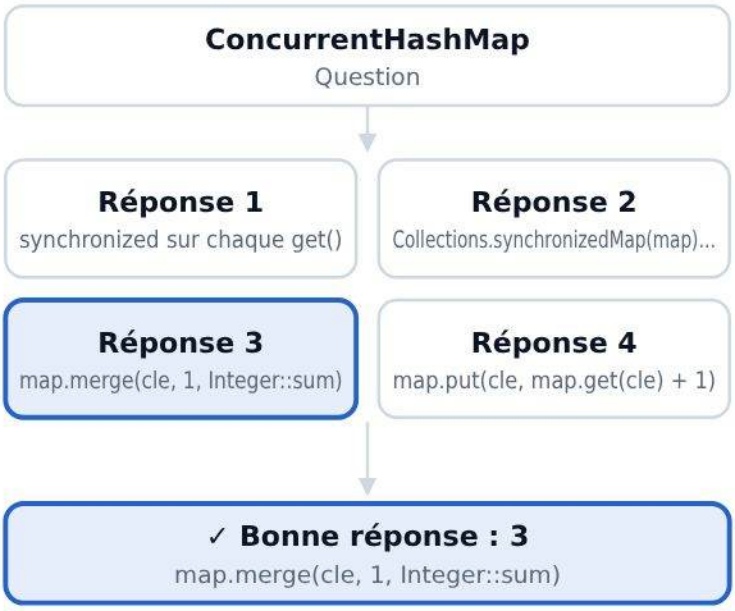


La réponse, étape par étape · *Step-by-step answer*

1. La question : Pourquoi utiliser AtomicInteger plutôt qu'un int dans un compteur partagé ?
   The question: Why use AtomicInteger rather than an int for a shared counter?
2. Les réponses possibles : 1. Il est plus rapide qu'un int en monothread · 2. Il empêche tout accès concurrent à l'objet · 3. incrementAndGet() est atomique sans verrou (CAS) · 4. Il consomme moins de mémoire
   Possible answers: 1. It is faster than an int in single-threaded code · 2. It blocks any concurrent access to the object · 3. incrementAndGet() is atomic without locking (CAS) · 4. It uses less memory
3. La bonne réponse est la 3 : incrementAndGet() est atomique sans verrou (CAS).
   The correct answer is 3: incrementAndGet() is atomic without locking (CAS).
4. AtomicInteger s'appuie sur le compare-and-swap du processeur : l'incrément est atomique sans synchronized.
   AtomicInteger relies on the CPU's compare-and-swap: the increment is atomic without synchronized.

ConcurrentHashMap

ConcurrentHashMap

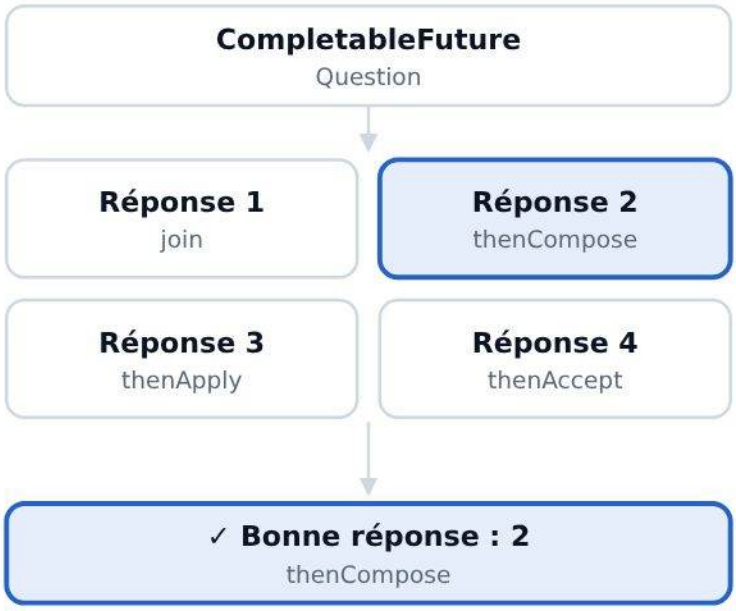


La réponse, étape par étape · *Step-by-step answer*

1. La question : Quelle est la bonne façon d'incrémenter un compteur dans une ConcurrentHashMap partagée ?
   The question: What is the right way to increment a counter in a shared ConcurrentHashMap?
2. Les réponses possibles : 1. synchronized sur chaque get() · 2. Collections.synchronizedMap(map).get(cle) · 3. map.merge(cle, 1, Integer::sum) · 4. map.put(cle, map.get(cle) + 1)
   Possible answers: 1. synchronized around each get() · 2. Collections.synchronizedMap(map).get(key) · 3. map.merge(key, 1, Integer::sum) · 4. map.put(key, map.get(key) + 1)
3. La bonne réponse est la 3 : map.merge(cle, 1, Integer::sum).
   The correct answer is 3: map.merge(key, 1, Integer::sum).
4. merge (comme compute) est atomique pour une clé ; get puis put sont deux opérations séparées qui peuvent perdre des mises à jour.
   merge (like compute) is atomic for a key; get then put are two separate operations that can lose updates.

CompletableFuture

CompletableFuture



La réponse, étape par étape · *Step-by-step answer*

1. La question : Avec CompletableFuture, quelle méthode enchaîne un appel qui renvoie lui-même un CompletableFuture, sans l'imbriquer ?
   The question: With CompletableFuture, which method chains a call that itself returns a CompletableFuture, without nesting it?
2. Les réponses possibles : 1. join · 2. thenCompose · 3. thenApply · 4. thenAccept
   Possible answers: 1. join · 2. thenCompose · 3. thenApply · 4. thenAccept
3. La bonne réponse est la 2 : thenCompose.
   The correct answer is 2: thenCompose.
4. thenCompose « aplatit » le résultat (comme flatMap) ; thenApply produirait un CompletableFuture<CompletableFuture<T>>.
   thenCompose flattens the result (like flatMap); thenApply would produce a CompletableFuture<CompletableFuture<T>>.

Garbage collector

Garbage collector

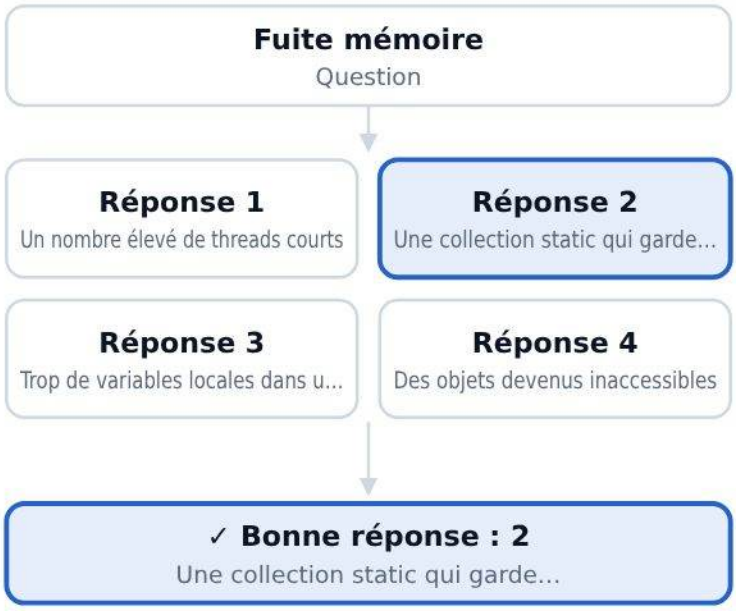


La réponse, étape par étape · *Step-by-step answer*

1. La question : Quel est le garbage collector par défaut de la JVM depuis Java 9 sur une machine serveur ?
   The question: Which garbage collector is the JVM default since Java 9 on a server-class machine?
2. Les réponses possibles : 1. G1 · 2. Serial · 3. CMS · 4. Parallel Old
   Possible answers: 1. G1 · 2. Serial · 3. CMS · 4. Parallel Old
3. La bonne réponse est la 1 : G1.
   The correct answer is 1: G1.
4. G1 est le GC par défaut depuis Java 9 : il découpe le tas en régions et vise des pauses courtes et prévisibles. CMS a été supprimé en Java 14.
   G1 is the default since Java 9: it splits the heap into regions and targets short, predictable pauses. CMS was removed in Java 14.

Fuite mémoire

Memory leak

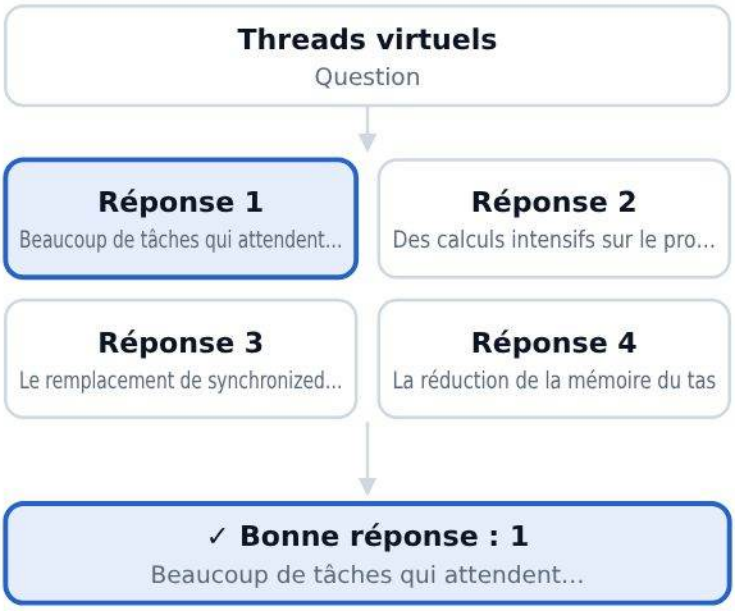


La réponse, étape par étape · *Step-by-step answer*

1. La question : Une application Java voit sa mémoire grimper jusqu'au OutOfMemoryError. Cause la plus probable ?
   The question: A Java application's memory grows until OutOfMemoryError. Most likely cause?
2. Les réponses possibles : 1. Un nombre élevé de threads courts · 2. Une collection static qui garde des références et grossit sans fin · 3. Trop de variables locales dans une méthode · 4. Des objets devenus inaccessibles
   Possible answers: 1. Many short-lived threads · 2. A static collection that keeps references and grows forever · 3. Too many local variables in a method · 4. Objects that became unreachable
3. La bonne réponse est la 2 : Une collection static qui garde des références et grossit sans fin.
   The correct answer is 2: A static collection that keeps references and grows forever.
4. Le GC ne libère que les objets inaccessibles : une Map static (cache sans limite, listeners jamais retirés) garde tout en vie.
   The GC only frees unreachable objects: a static Map (unbounded cache, listeners never removed) keeps everything alive.

Threads virtuels

Virtual threads

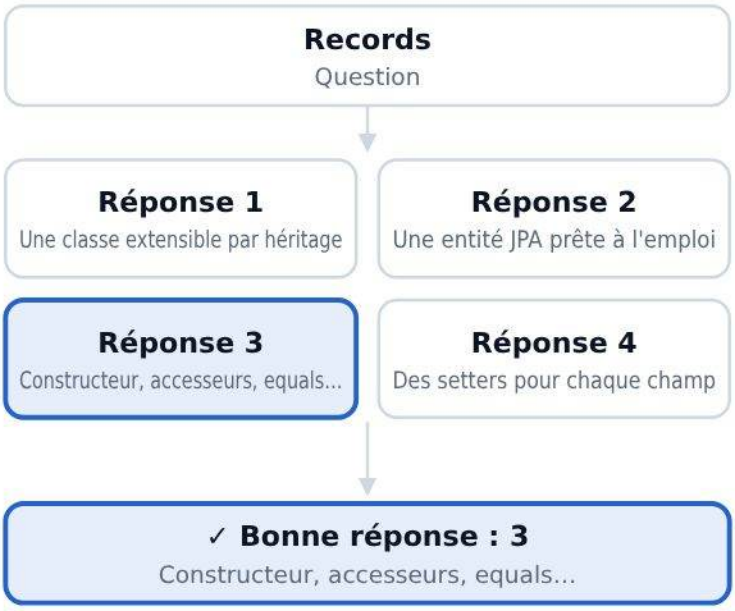


La réponse, étape par étape · *Step-by-step answer*

1. La question : Dans quel cas les threads virtuels (Java 21) apportent-ils le plus ?
   The question: When do virtual threads (Java 21) help the most?
2. Les réponses possibles : 1. Beaucoup de tâches qui attendent des entrées-sorties (HTTP, base de données) · 2. Des calculs intensifs sur le processeur · 3. Le remplacement de synchronized partout · 4. La réduction de la mémoire du tas
   Possible answers: 1. Many tasks waiting on I/O (HTTP, database) · 2. CPU-intensive computations · 3. Replacing synchronized everywhere · 4. Reducing heap memory
3. La bonne réponse est la 1 : Beaucoup de tâches qui attendent des entrées-sorties (HTTP, base de données).
   The correct answer is 1: Many tasks waiting on I/O (HTTP, database).
4. Un thread virtuel libère son thread porteur pendant l'attente d'E/S : on peut en lancer des millions. Pour du calcul pur, le nombre de cœurs reste la limite.
   A virtual thread releases its carrier thread while waiting on I/O: you can run millions. For pure computation, the number of cores is still the limit.

Records

Records

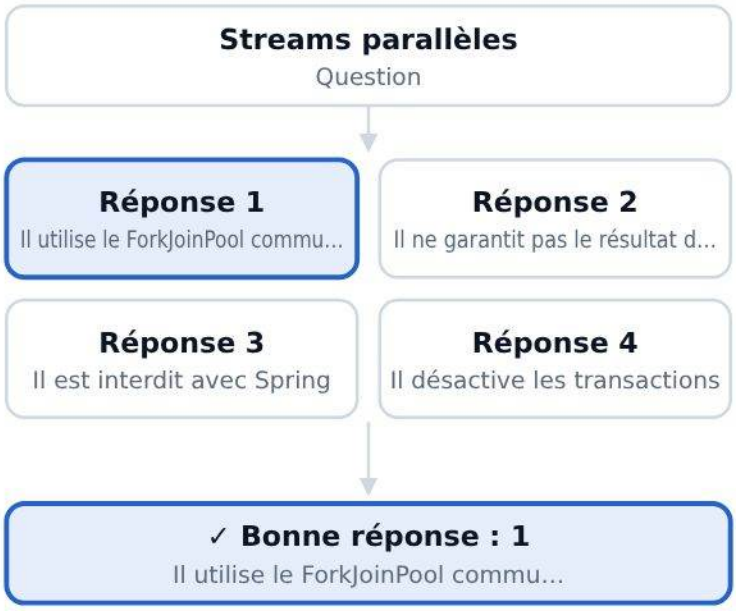


La réponse, étape par étape · *Step-by-step answer*

1. La question : Que génère automatiquement un record Java ?
   The question: What does a Java record generate automatically?
2. Les réponses possibles : 1. Une classe extensible par héritage · 2. Une entité JPA prête à l'emploi · 3. Constructeur, accesseurs, equals, hashCode et toString · 4. Des setters pour chaque champ
   Possible answers: 1. A class open to inheritance · 2. A ready-to-use JPA entity · 3. Constructor, accessors, equals, hashCode and toString · 4. Setters for every field
3. La bonne réponse est la 3 : Constructeur, accesseurs, equals, hashCode et toString.
   The correct answer is 3: Constructor, accessors, equals, hashCode and toString.
4. Un record est une classe finale immuable : champs final, constructeur canonique, accesseurs, equals/hashCode/toString. Pas de setters.
   A record is an immutable final class: final fields, canonical constructor, accessors, equals/hashCode/toString. No setters.

Streams parallèles

Parallel streams

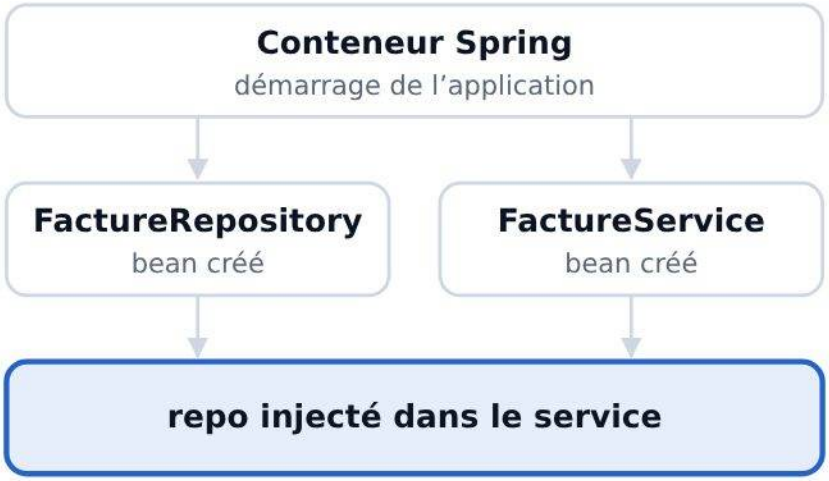


La réponse, étape par étape · *Step-by-step answer*

1. La question : Pourquoi éviter parallelStream() dans un service web qui appelle une base de données ?
   The question: Why avoid parallelStream() in a web service that calls a database?
2. Les réponses possibles : 1. Il utilise le ForkJoinPool commun partagé par toute l'application · 2. Il ne garantit pas le résultat d'un sum() · 3. Il est interdit avec Spring · 4. Il désactive les transactions
   Possible answers: 1. It uses the common ForkJoinPool shared by the whole application · 2. It does not guarantee the result of sum() · 3. It is forbidden with Spring · 4. It disables transactions
3. La bonne réponse est la 1 : Il utilise le ForkJoinPool commun partagé par toute l'application.
   The correct answer is 1: It uses the common ForkJoinPool shared by the whole application.
4. Tous les parallelStream partagent le ForkJoinPool commun (cœurs - 1 threads) : des appels bloquants le saturent pour toute l'application.
   All parallel streams share the common ForkJoinPool (cores - 1 threads): blocking calls starve it for the entire application.

Injection

Injection

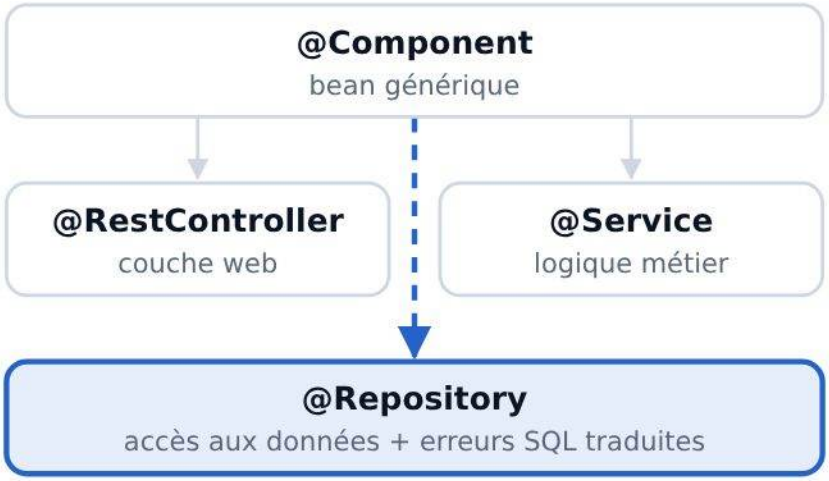


La réponse, étape par étape · *Step-by-step answer*

1. Au démarrage, Spring cherche les classes annotées (@Service, @Repository…).
   At startup, Spring looks for annotated classes (@Service, @Repository…).
2. Il crée d'abord le repository, car le service en a besoin.
   It creates the repository first, because the service needs it.
3. Puis il crée le service.
   Then it creates the service.
4. Il passe le repository au constructeur du service : c'est l'injection. La classe ne fait jamais new elle-même.
   It passes the repository to the service's constructor: this is injection. The class never calls new itself.
5. Avantage : dans un test, on passe un faux repository au constructeur, sans base de données.
   Advantage: in a test, we pass a fake repository to the constructor, with no database.

Les annotations

Annotations



La réponse, étape par étape · *Step-by-step answer*

1. @Component dit à Spring : « crée et gère cet objet ». Les autres annotations en sont des variantes.
   @Component tells Spring: "create and manage this object". The other annotations are variants of it.
2. @RestController : reçoit les requêtes HTTP et renvoie du JSON.
   @RestController: receives HTTP requests and returns JSON.
3. @Service : la logique métier (calculer, valider). Même effet que @Component, mais le rôle est clair.
   @Service: business logic (calculating, validating). Same effect as @Component, but the role is clear.
4. @Repository : l'accès à la base. Bonus : les erreurs SQL deviennent des exceptions Spring.
   @Repository: database access. Bonus: SQL errors become Spring exceptions.

Scopes

Scopes

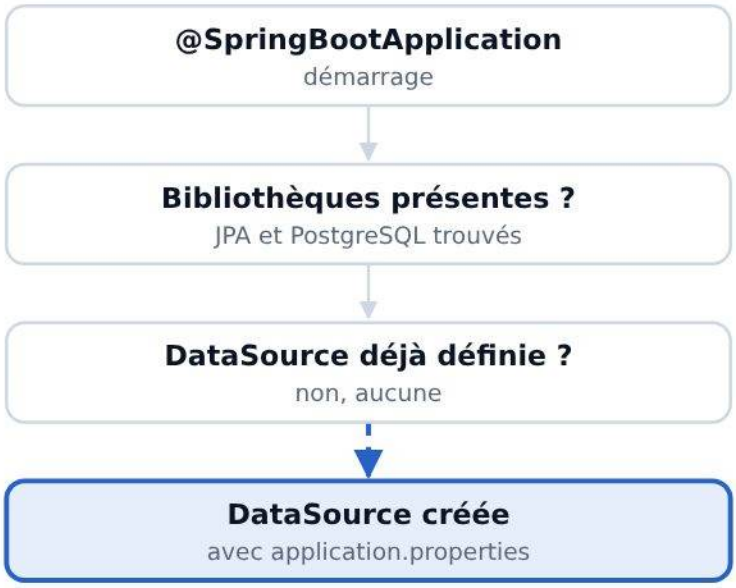


La réponse, étape par étape · *Step-by-step answer*

1. Singleton : Spring crée un seul TauxService et le donne à tout le monde.
   Singleton: Spring creates a single TauxService and gives it to everyone.
2. Prototype : chaque demande reçoit un objet tout neuf.
   Prototype: each request gets a brand-new object.
3. Piège : un singleton est partagé. Ne jamais y garder une donnée propre à une requête ou à un utilisateur.
   Trap: a singleton is shared. Never store data specific to a request or a user in it.

Auto-configuration

Auto-configuration

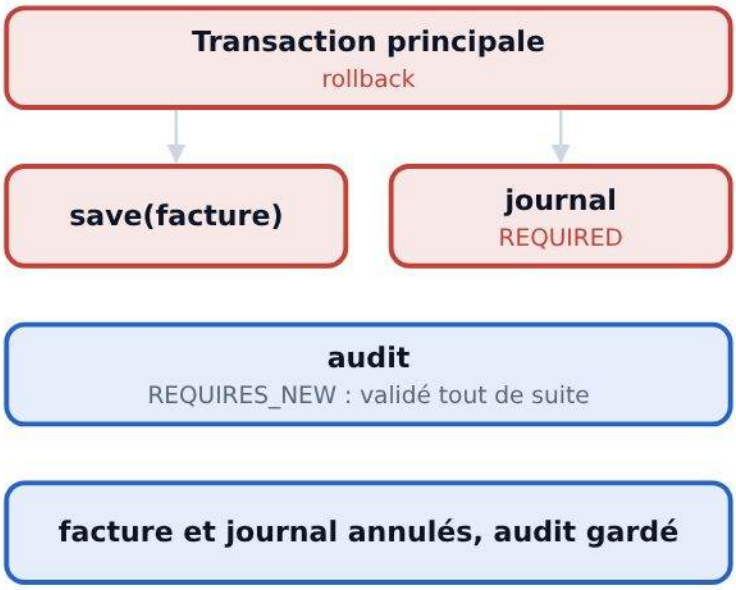


La réponse, étape par étape · *Step-by-step answer*

1. @SpringBootApplication active l'auto-configuration.
   @SpringBootApplication enables auto-configuration.
2. Spring Boot regarde les bibliothèques ajoutées : JPA et le driver PostgreSQL sont là.
   Spring Boot looks at the libraries you added: JPA and the PostgreSQL driver are there.
3. Il vérifie que tu n'as pas déjà configuré ta propre connexion.
   It checks that you have not already configured your own connection.
4. Alors il crée la connexion avec l'URL du fichier de config. Zéro code à écrire.
   So it creates the connection with the URL from the config file. Zero code to write.

Propagation

Propagation

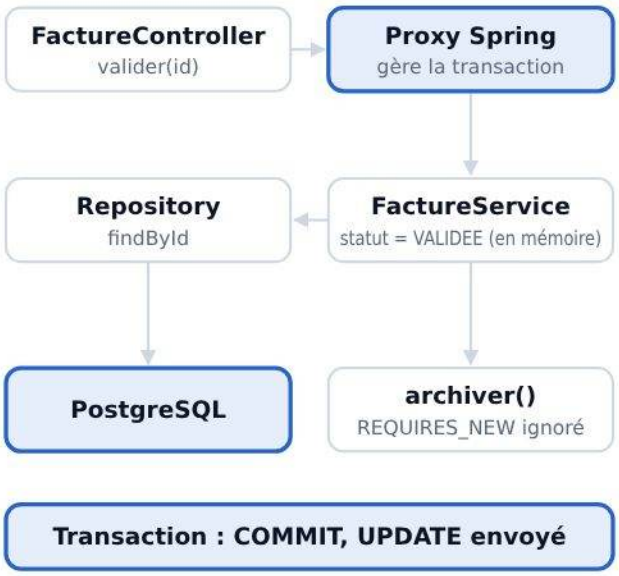


La réponse, étape par étape · *Step-by-step answer*

1. Le proxy ouvre une transaction avant la méthode.
   The proxy opens a transaction before the method.
2. La facture est enregistrée dans cette transaction.
   The invoice is saved in this transaction.
3. REQUIRED (le choix par défaut) : le journal rejoint la même transaction.
   REQUIRED (the default): the log joins the same transaction.
4. REQUIRES_NEW : l'audit ouvre sa propre transaction, validée à part.
   REQUIRES_NEW: the audit opens its own transaction, committed separately.
5. Une erreur arrive : tout ce qui est dans la transaction principale est annulé. L'audit, séparé, reste.
   An error occurs: everything in the main transaction is cancelled. The audit, being separate, stays.

@Transactional

@Transactional

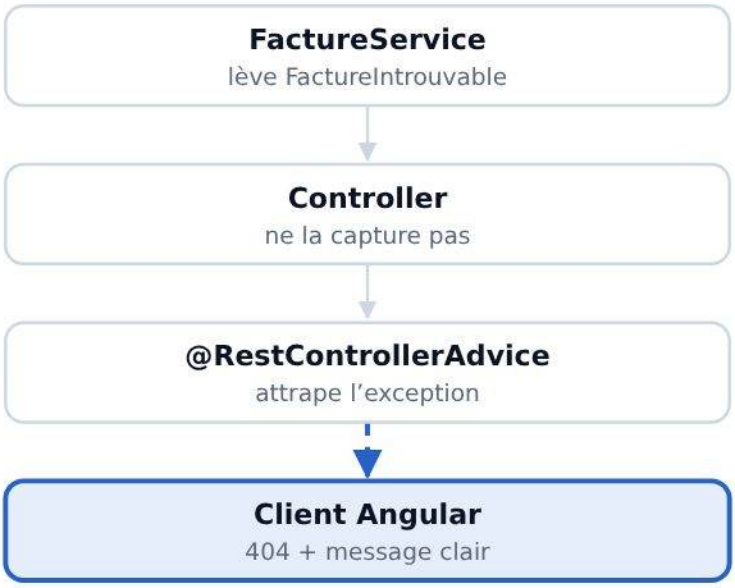


La réponse, étape par étape · *Step-by-step answer*

1. Le contrôleur pense appeler le service. En vrai, il appelle un intermédiaire créé par Spring : le proxy.
   The controller thinks it is calling the service. In reality, it calls a middleman created by Spring: the proxy.
2. Le proxy voit @Transactional. Il ouvre une transaction : tout ce qui suit sera validé ou annulé en un seul bloc.
   The proxy sees @Transactional. It opens a transaction: everything that follows will be committed or cancelled as one block.
3. Le proxy passe ensuite la main au vrai service.
   The proxy then hands over to the real service.
4. Le service lit la facture dans la base de données.
   The service reads the invoice from the database.
5. On change le statut. Rien n'est encore écrit en base : Hibernate note juste le changement.
   We change the status. Nothing is written to the database yet: Hibernate just notes the change.
6. Piège : this.archiver() appelle la méthode directement, sans passer par le proxy. Son @Transactional est donc ignoré.
   Trap: this.archiver() calls the method directly, without going through the proxy. So its @Transactional is ignored.
7. Tout s'est bien passé : le proxy valide (commit) et la modification est écrite en base. En cas d'erreur, tout aurait été annulé (rollback).
   Everything went well: the proxy commits and the change is written to the database. If there had been an error, everything would have been cancelled (rollback).

Erreurs globales

Global error handling

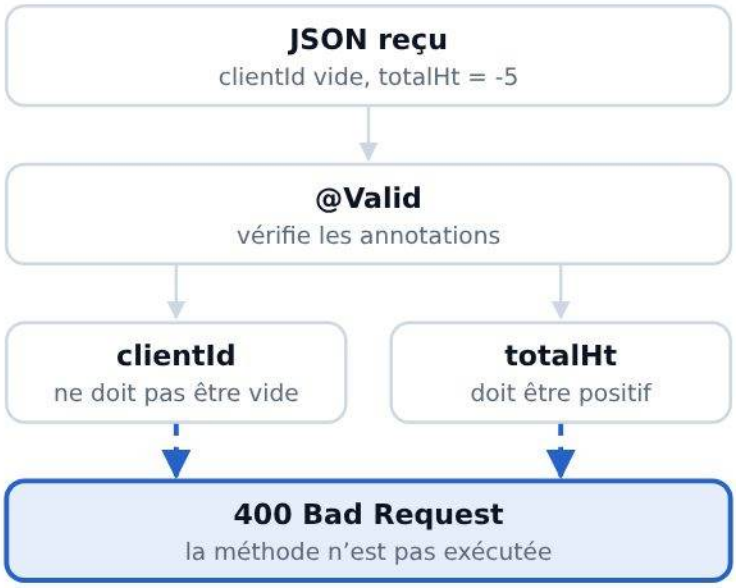


La réponse, étape par étape · *Step-by-step answer*

1. Le service ne trouve pas la facture 42 : il lève une exception.
   The service cannot find invoice 42: it throws an exception.
2. L'exception remonte à travers le contrôleur. Pas besoin de try/catch partout.
   The exception travels up through the controller. No need for try/catch everywhere.
3. Le @RestControllerAdvice l'attrape, comme un filet de sécurité commun à toute l'API.
   The @RestControllerAdvice catches it, like a safety net shared by the whole API.
4. Il renvoie un code 404 et un message JSON propre, toujours au même format.
   It returns a 404 code and a clean JSON message, always in the same format.

Validation

Validation

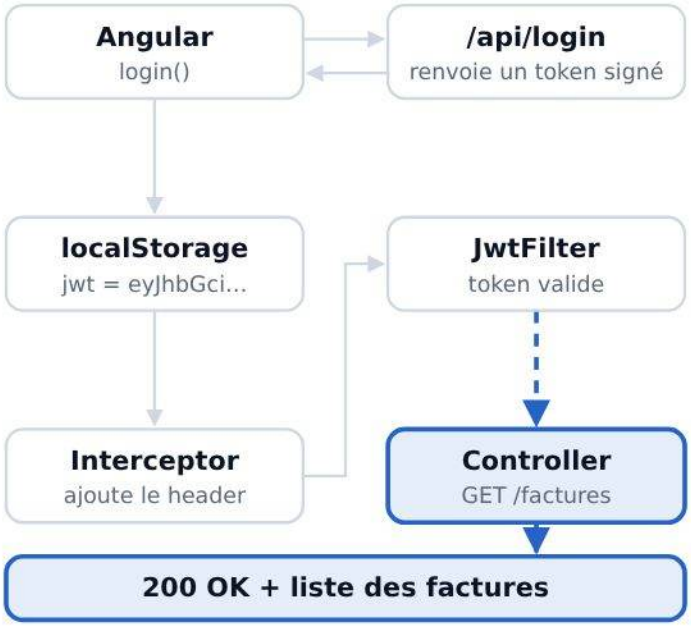


La réponse, étape par étape · *Step-by-step answer*

1. Le front envoie un JSON incorrect.
   The front end sends invalid JSON.
2. @Valid demande à Spring de vérifier les règles du DTO avant d'entrer dans la méthode.
   @Valid asks Spring to check the DTO's rules before entering the method.
3. Deux règles ne sont pas respectées.
   Two rules are not respected.
4. Réponse 400 avec la liste des erreurs. Le code de la méthode n'a jamais été exécuté.
   Response 400 with the list of errors. The method's code was never executed.

JWT

JWT

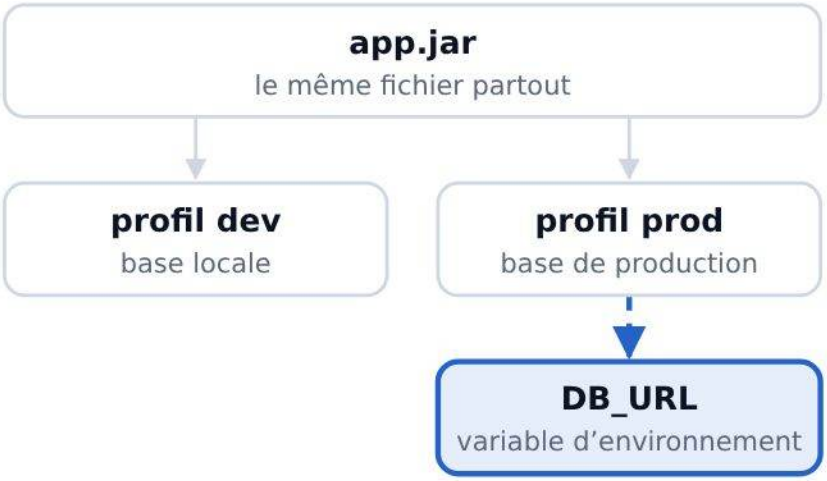


La réponse, étape par étape · *Step-by-step answer*

1. L'utilisateur se connecte : Angular envoie son identifiant et son mot de passe au serveur.
   The user logs in: Angular sends their username and password to the server.
2. Le serveur vérifie. Si c'est bon, il renvoie un token : une sorte de badge d'accès signé.
   The server checks. If it is correct, it sends back a token: a kind of signed access badge.
3. Angular garde ce badge de côté, dans localStorage.
   Angular keeps this badge aside, in localStorage.
4. À chaque nouvelle demande, l'interceptor colle automatiquement le badge sur la requête.
   With every new request, the interceptor automatically sticks the badge onto the request.
5. La requête arrive au serveur. Le JwtFilter, comme un vigile, l'arrête à l'entrée.
   The request reaches the server. The JwtFilter, like a security guard, stops it at the door.
6. Le vigile vérifie le badge : est-il vrai ? Est-il encore valable ? Si oui, il laisse entrer.
   The guard checks the badge: is it real? Is it still valid? If so, it lets the request in.
7. Le contrôleur répond avec les factures. Badge faux ou expiré : erreur 401, accès refusé.
   The controller responds with the invoices. Fake or expired badge: error 401, access denied.

Profils

Profiles

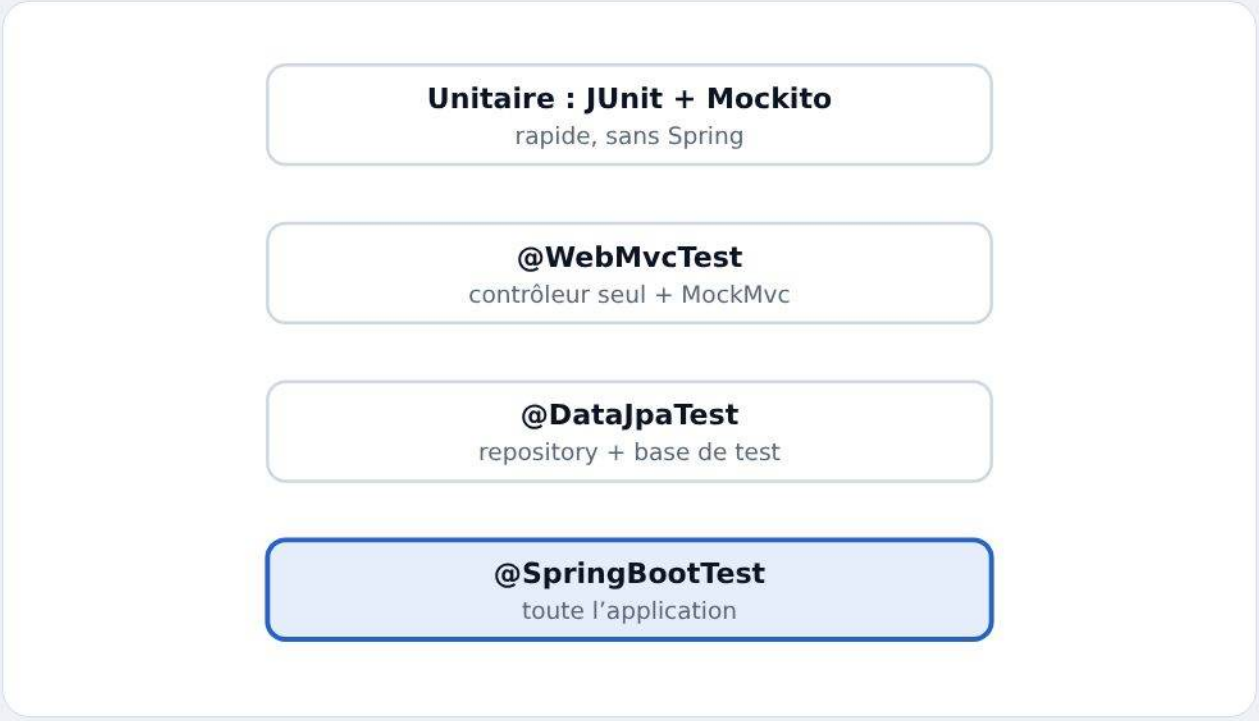


La réponse, étape par étape · *Step-by-step answer*

1. On construit une seule application.
   We build a single application.
2. En développement, le profil dev pointe vers la base locale.
   In development, the dev profile points to the local database.
3. En production, on active le profil prod au lancement.
   In production, we activate the prod profile at launch.
4. L'URL et le mot de passe viennent de variables d'environnement, jamais du code.
   The URL and password come from environment variables, never from the code.

Tests

Tests

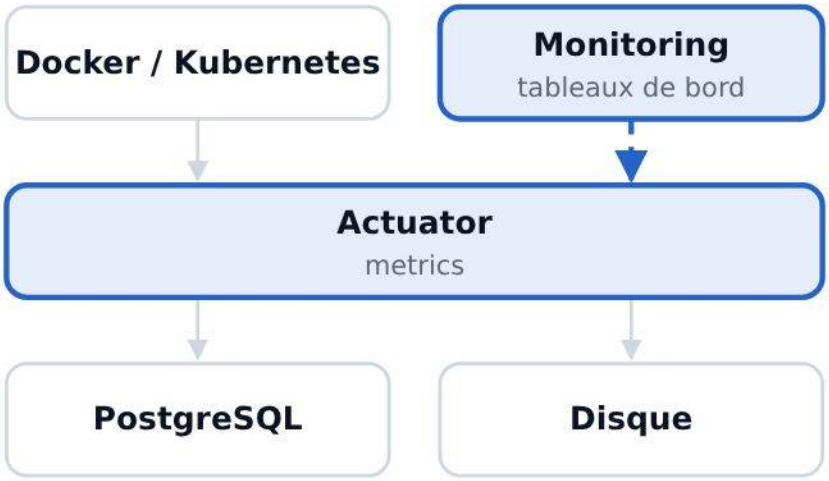


La réponse, étape par étape · *Step-by-step answer*

1. Test unitaire : on teste le service seul, le repository est remplacé par un faux (mock). Très rapide, on en écrit beaucoup.
   Unit test: we test the service alone, the repository is replaced by a fake (mock). Very fast, we write many of them.
2. On teste le contrôleur seul : URL, JSON, codes HTTP. Le service est remplacé par un faux.
   We test the controller alone: URL, JSON, HTTP codes. The service is replaced by a fake.
3. On teste les requêtes du repository sur une vraie base de test.
   We test the repository's queries on a real test database.
4. Test d'intégration : toute l'application démarre, avec un vrai PostgreSQL dans Docker. Plus lent, on en écrit moins.
   Integration test: the whole application starts, with a real PostgreSQL in Docker. Slower, we write fewer of them.

Actuator

Actuator



La réponse, étape par étape · *Step-by-step answer*

1. On ajoute Actuator : l'application expose des adresses de surveillance.
   We add Actuator: the application exposes monitoring endpoints.
2. /health vérifie la base, le disque… et répond UP ou DOWN. Si DOWN, l'app peut être redémarrée automatiquement.
   /health checks the database, the disk… and answers UP or DOWN. If DOWN, the app can be restarted automatically.
3. /metrics donne des chiffres : nombre de requêtes, temps de réponse, mémoire.
   /metrics gives numbers: number of requests, response time, memory.

Champ ou constructeur

Field or constructor

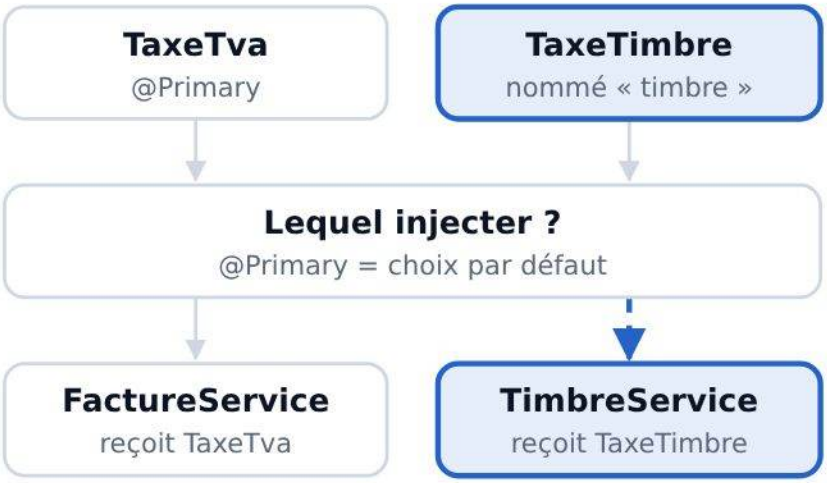


La réponse, étape par étape · *Step-by-step answer*

1. Par champ : Spring remplit le champ en cachette. Impossible de le rendre final.
   By field: Spring fills the field behind the scenes. Impossible to make it final.
2. Par constructeur : la dépendance est obligatoire, final, et visible dans la signature.
   By constructor: the dependency is mandatory, final, and visible in the signature.
3. Pour tester, on passe un faux repository au constructeur. Avec un champ, il faudrait démarrer Spring.
   To test, we pass a fake repository to the constructor. With a field, we would have to start Spring.

@Primary / @Qualifier

@Primary / @Qualifier

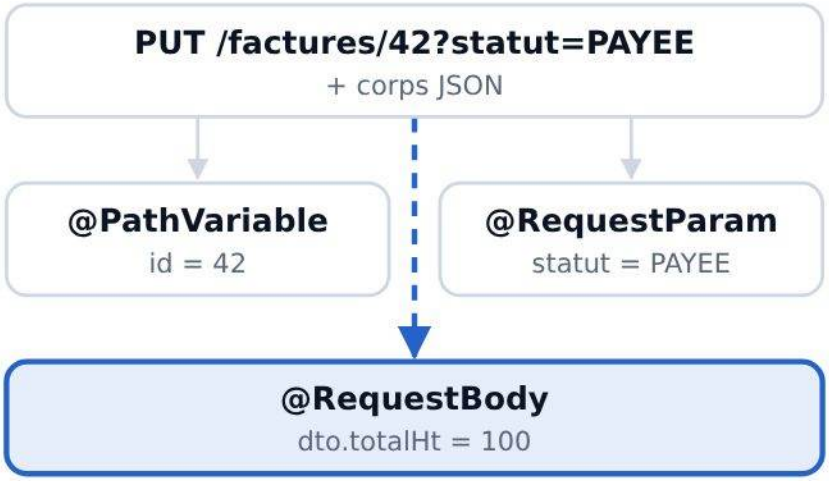


La réponse, étape par étape · *Step-by-step answer*

1. Deux beans implémentent CalculTaxe : sans indication, Spring ne sait pas lequel choisir.
   Two beans implement CalculTaxe: without a hint, Spring does not know which one to choose.
2. @Primary désigne le choix par défaut.
   @Primary marks the default choice.
3. @Qualifier demande un bean précis, par son nom.
   @Qualifier asks for a specific bean, by its name.

Lire la requête

Reading the request

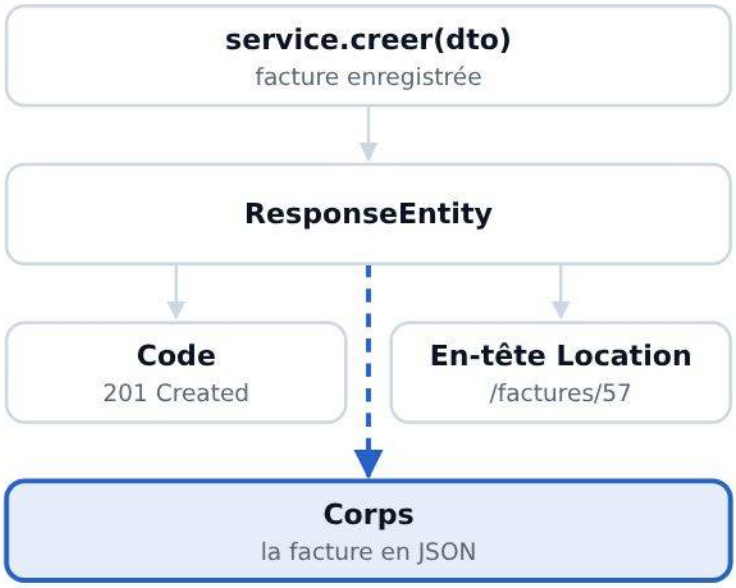


La réponse, étape par étape · *Step-by-step answer*

1. Une requête arrive avec trois informations à trois endroits différents.
   A request arrives with three pieces of information in three different places.
2. @PathVariable lit un morceau de l'adresse : l'identifiant de la ressource.
   @PathVariable reads a piece of the URL: the resource's identifier.
3. @RequestParam lit ce qui suit le « ? » : un filtre ou une option.
   @RequestParam reads what comes after the "?": a filter or an option.
4. @RequestBody transforme le JSON du corps en objet Java.
   @RequestBody turns the JSON body into a Java object.

ResponseEntity

ResponseEntity

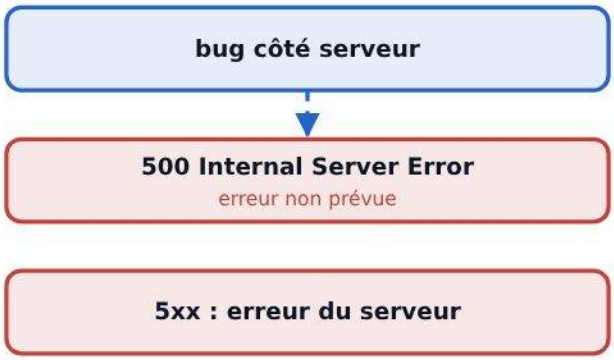


La réponse, étape par étape · *Step-by-step answer*

1. Le service crée la facture.
   The service creates the invoice.
2. ResponseEntity permet de choisir le code : 201 = ressource créée.
   ResponseEntity lets you choose the code: 201 = resource created.
3. On ajoute l'adresse de la nouvelle facture dans l'en-tête Location.
   We add the new invoice's address in the Location header.
4. Et on renvoie la facture créée dans le corps.
   And we return the created invoice in the body.

Codes HTTP

HTTP codes

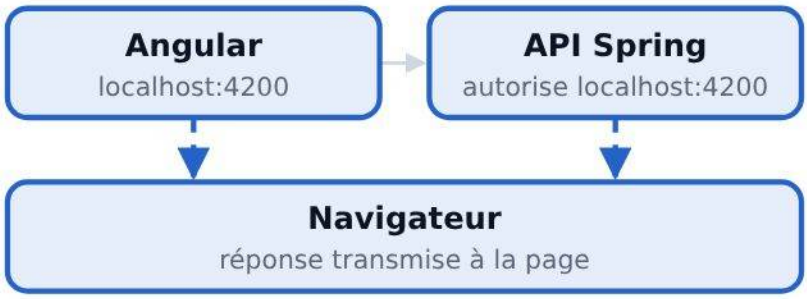


La réponse, étape par étape · *Step-by-step answer*

1. 200 : tout s'est bien passé.
   200: everything went well.
2. 201 : une ressource a été créée.
   201: a resource was created.
3. 204 : action réussie, pas de contenu à renvoyer.
   204: action succeeded, no content to return.
4. 400 : la requête est mal formée ou invalide.
   400: the request is malformed or invalid.
5. 401 : l'utilisateur n'est pas identifié.
   401: the user is not identified.
6. 403 : identifié, mais pas autorisé. Ne pas confondre avec 401.
   403: identified, but not allowed. Do not confuse with 401.
7. 404 : la ressource n'existe pas.
   404: the resource does not exist.
8. 500 : le problème vient du serveur, pas du client.
   500: the problem comes from the server, not the client.

CORS

CORS

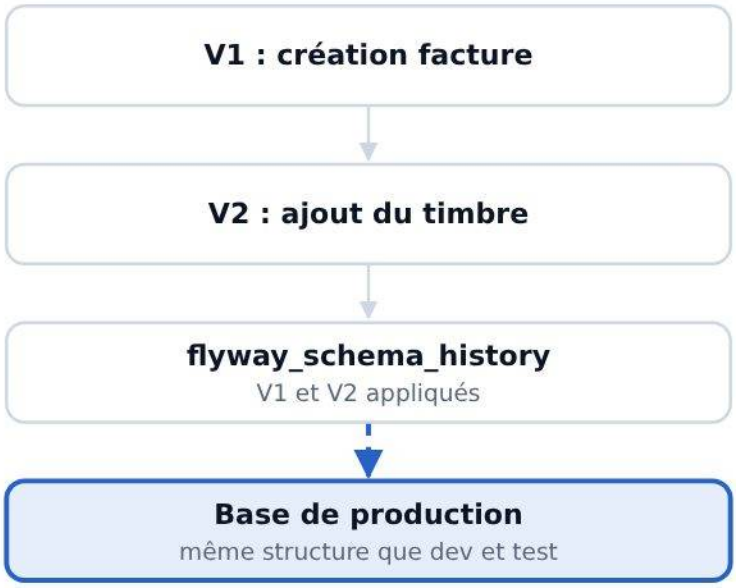


La réponse, étape par étape · *Step-by-step answer*

1. La page Angular appelle une API sur un autre port : c’est une autre « origine ».
   The Angular page calls an API on another port: it is another "origin".
2. Sans autorisation de l’API, le navigateur bloque la réponse. C’est l’erreur CORS dans la console.
   Without permission from the API, the browser blocks the response. That is the CORS error in the console.
3. L’API déclare l’origine autorisée : le navigateur laisse passer la réponse.
   The API declares the allowed origin: the browser lets the response through.

Flyway

Flyway



La réponse, étape par étape · *Step-by-step answer*

1. Chaque changement de base est un script numéroté, rangé dans Git.
   Each database change is a numbered script, stored in Git.
2. Pour ajouter le timbre fiscal, on écrit un nouveau script V2. On ne modifie jamais V1.
   To add the timbre fiscal, we write a new V2 script. We never modify V1.
3. Au démarrage, Flyway regarde l'historique et n'applique que les scripts manquants.
   At startup, Flyway looks at the history and applies only the missing scripts.
4. Résultat : tous les environnements ont exactement la même structure.
   Result: all environments have exactly the same structure.

Logs

Logs

Jamais de mot de passe ni de token

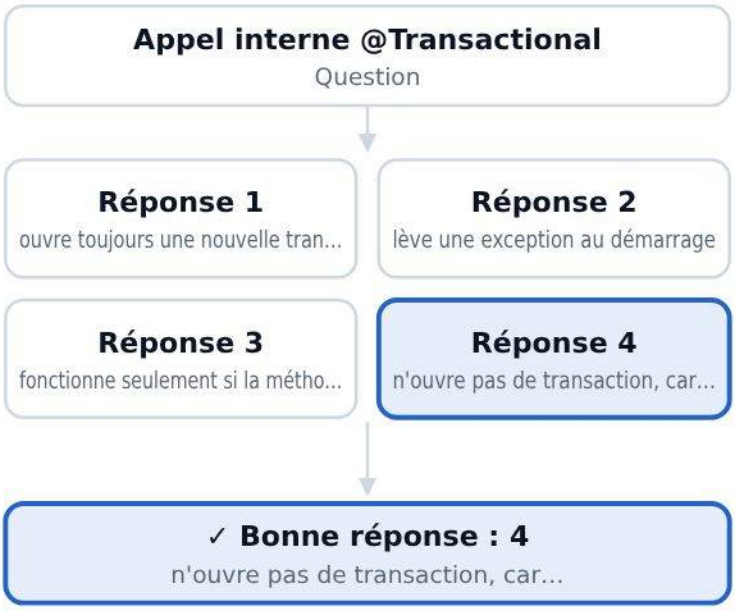
DEBUG pas affiché en prod	**INFO** affiché
WARN affiché	**ERROR** affiché + erreur

La réponse, étape par étape · *Step-by-step answer*

1. DEBUG : détails pour le développement. En production (niveau INFO), ils sont masqués.
   DEBUG: details for development. In production (INFO level), they are hidden.
2. INFO : les événements normaux importants.
   INFO: important normal events.
3. WARN : quelque chose d'anormal, mais l'application continue.
   WARN: something abnormal, but the application keeps running.
4. ERROR : un vrai problème. On ajoute l'exception pour avoir la cause.
   ERROR: a real problem. We add the exception to get the cause.
5. Les logs sont lus par beaucoup de monde : on n'y écrit jamais de données secrètes.
   Logs are read by many people: we never write secret data in them.

Appel interne @Transactional

@Transactional self-call

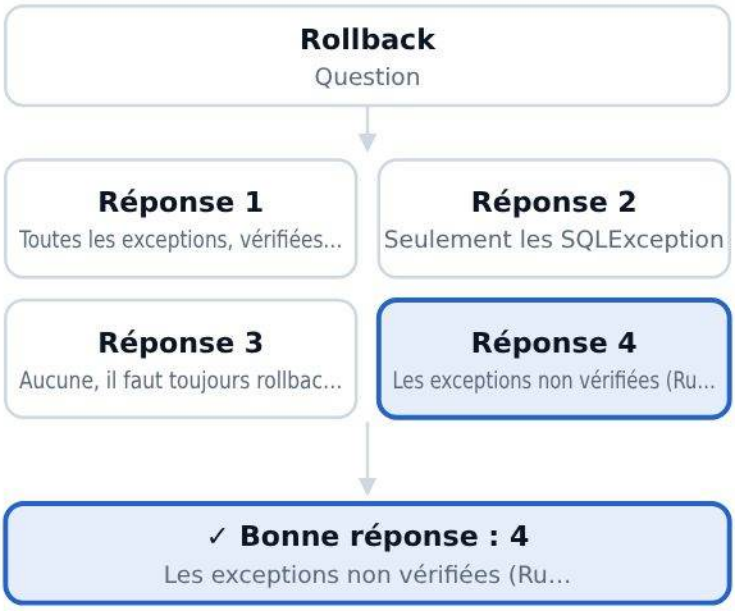


La réponse, étape par étape · *Step-by-step answer*

1. La question : Une méthode @Transactional appelée depuis une autre méthode de la même classe :
   The question: A @Transactional method called from another method of the same class:
2. Les réponses possibles : 1. ouvre toujours une nouvelle transaction · 2. lève une exception au démarrage · 3. fonctionne seulement si la méthode est private · 4. n'ouvre pas de transaction, car l'appel ne passe pas par le proxy
   Possible answers: 1. always starts a new transaction · 2. throws an exception at startup · 3. only works if the method is private · 4. does not start a transaction, because the call bypasses the proxy
3. La bonne réponse est la 4 : n'ouvre pas de transaction, car l'appel ne passe pas par le proxy.
   The correct answer is 4: does not start a transaction, because the call bypasses the proxy.
4. Spring applique @Transactional via un proxy : un appel this.methode() contourne le proxy. Solution : déplacer la méthode dans un autre bean.
   Spring applies @Transactional through a proxy: a this.method() call bypasses it. Fix: move the method to another bean.

Rollback

Rollback

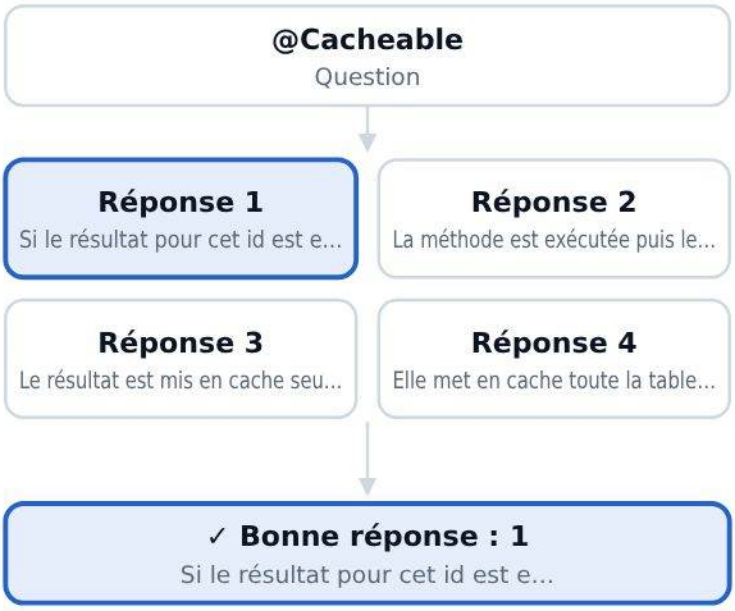


La réponse, étape par étape · *Step-by-step answer*

1. La question : Par défaut, quelles exceptions provoquent le rollback d'une méthode @Transactional ?
   The question: By default, which exceptions trigger a rollback of a @Transactional method?
2. Les réponses possibles : 1. Toutes les exceptions, vérifiées comprises · 2. Seulement les SQLException · 3. Aucune, il faut toujours rollbackFor · 4. Les exceptions non vérifiées (RuntimeException) et les Error
   Possible answers: 1. All exceptions, including checked ones · 2. Only SQLException · 3. None, rollbackFor is always required · 4. Unchecked exceptions (RuntimeException) and Errors
3. La bonne réponse est la 4 : Les exceptions non vérifiées (RuntimeException) et les Error.
   The correct answer is 4: Unchecked exceptions (RuntimeException) and Errors.
4. Par défaut, rollback sur RuntimeException et Error ; une exception vérifiée valide la transaction sauf si rollbackFor est précisé.
   By default, rollback on RuntimeException and Error; a checked exception commits unless rollbackFor is set.

@Cacheable

@Cacheable

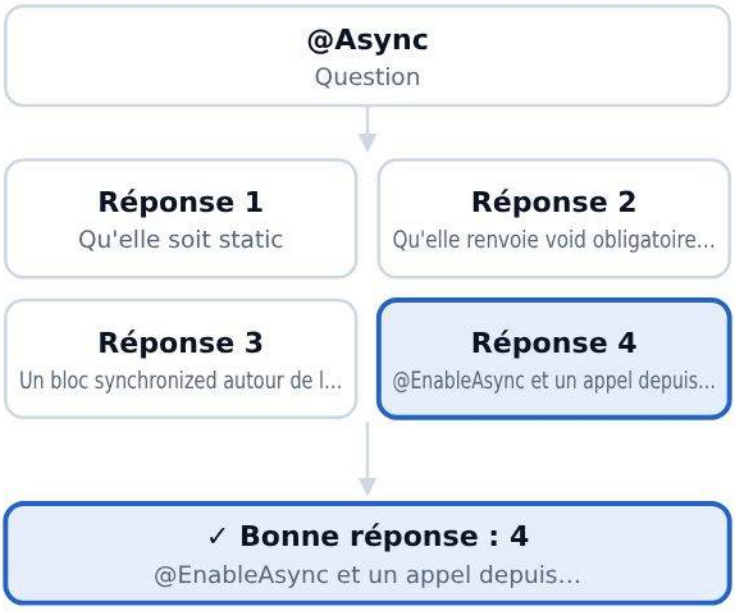


La réponse, étape par étape · *Step-by-step answer*

1. La question : Que fait @Cacheable("clients") sur une méthode findById(id) ?
   The question: What does @Cacheable("clients") do on a findById(id) method?
2. Les réponses possibles : 1. Si le résultat pour cet id est en cache, la méthode n'est pas exécutée · 2. La méthode est exécutée puis le cache est vidé · 3. Le résultat est mis en cache seulement en cas d'erreur · 4. Elle met en cache toute la table clients au démarrage
   Possible answers: 1. If the result for that id is cached, the method is not executed · 2. The method runs then the cache is cleared · 3. The result is cached only on error · 4. It caches the whole clients table at startup
3. La bonne réponse est la 1 : Si le résultat pour cet id est en cache, la méthode n'est pas exécutée.
   The correct answer is 1: If the result for that id is cached, the method is not executed.
4. Au premier appel le résultat est stocké sous la clé id ; les appels suivants le renvoient sans exécuter la méthode. @CacheEvict invalide.
   On the first call the result is stored under the id key; later calls return it without running the method. @CacheEvict invalidates it.

@Async

@Async



La réponse, étape par étape · *Step-by-step answer*

1. La question : Que faut-il pour qu'une méthode @Async s'exécute vraiment dans un autre thread ?
   The question: What does an @Async method need to really run in another thread?
2. Les réponses possibles : 1. Qu'elle soit static · 2. Qu'elle renvoie void obligatoirement · 3. Un bloc synchronized autour de l'appel · 4. @EnableAsync et un appel depuis un autre bean (via le proxy)
   Possible answers: 1. It must be static · 2. It must return void · 3. A synchronized block around the call · 4. @EnableAsync and a call from another bean (through the proxy)
3. La bonne réponse est la 4 : @EnableAsync et un appel depuis un autre bean (via le proxy).
   The correct answer is 4: @EnableAsync and a call from another bean (through the proxy).
4. Comme @Transactional, @Async passe par un proxy : il faut @EnableAsync et un appel externe. Elle peut renvoyer void ou CompletableFuture.
   Like @Transactional, @Async works through a proxy: you need @EnableAsync and an external call. It may return void or CompletableFuture.

Refresh token

Refresh token

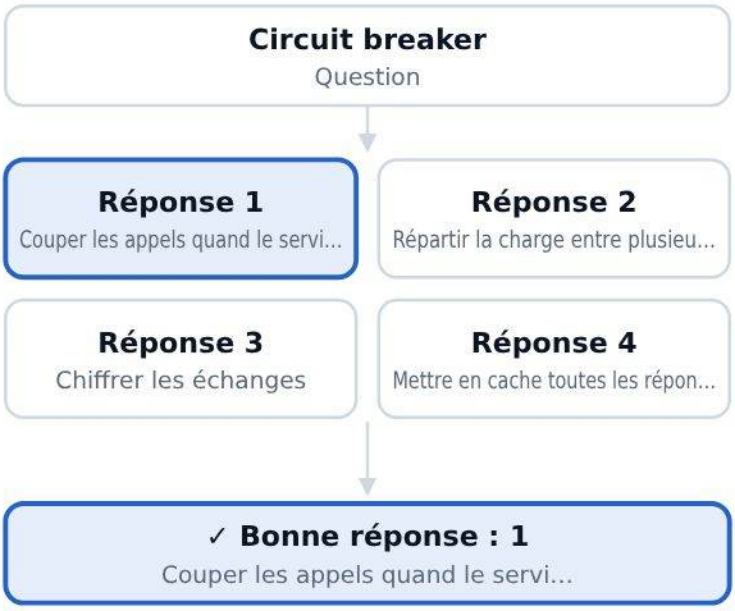


La réponse, étape par étape · *Step-by-step answer*

1. La question : Pourquoi associer un access token JWT court (15 min) à un refresh token ?
   The question: Why pair a short JWT access token (15 min) with a refresh token?
2. Les réponses possibles : 1. Chiffrer le contenu du JWT · 2. Éviter de signer les tokens · 3. Permettre au serveur de stocker les sessions en mémoire · 4. Limiter les dégâts d'un token volé sans redemander le mot de passe
   Possible answers: 1. Encrypt the JWT content · 2. Avoid signing tokens · 3. Let the server keep sessions in memory · 4. Limit the damage of a stolen token without asking for the password again
3. La bonne réponse est la 4 : Limiter les dégâts d'un token volé sans redemander le mot de passe.
   The correct answer is 4: Limit the damage of a stolen token without asking for the password again.
4. Un JWT ne se révoque pas facilement : on le garde court. Le refresh token, stocké et révocable côté serveur, en obtient un nouveau.
   A JWT is hard to revoke, so keep it short. The refresh token, stored and revocable server-side, obtains a new one.

Circuit breaker

Circuit breaker

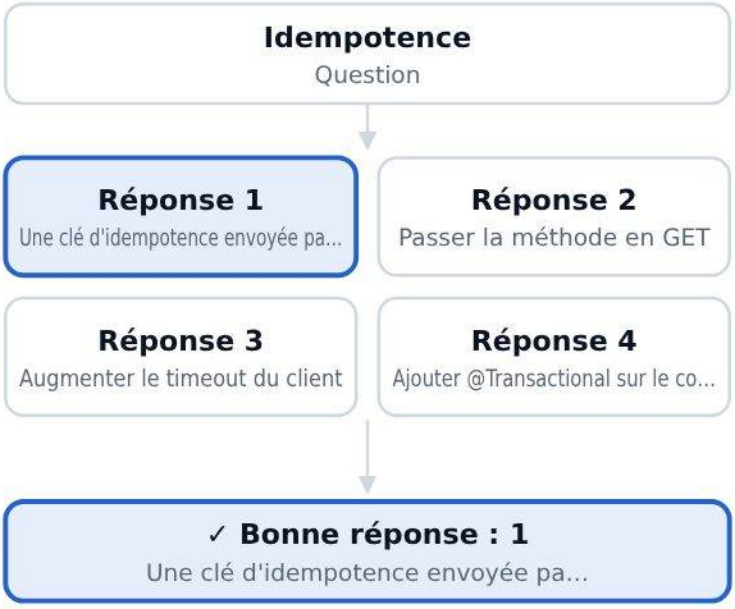


La réponse, étape par étape · *Step-by-step answer*

1. La question : Quel est le rôle d'un circuit breaker (Resilience4j) devant un service distant ?
   The question: What is the role of a circuit breaker (Resilience4j) in front of a remote service?
2. Les réponses possibles : 1. Couper les appels quand le service échoue trop souvent et renvoyer un repli · 2. Répartir la charge entre plusieurs instances · 3. Chiffrer les échanges · 4. Mettre en cache toutes les réponses
   Possible answers: 1. Stop calls when the service fails too often and return a fallback · 2. Balance load across instances · 3. Encrypt the traffic · 4. Cache every response
3. La bonne réponse est la 1 : Couper les appels quand le service échoue trop souvent et renvoyer un repli.
   The correct answer is 1: Stop calls when the service fails too often and return a fallback.
4. Après trop d'échecs, le circuit s'ouvre : les appels échouent tout de suite (fallback) au lieu d'attendre, puis il teste à nouveau (semi-ouvert).
   After too many failures the circuit opens: calls fail fast (fallback) instead of waiting, then it tests again (half-open).

Idempotence

Idempotency

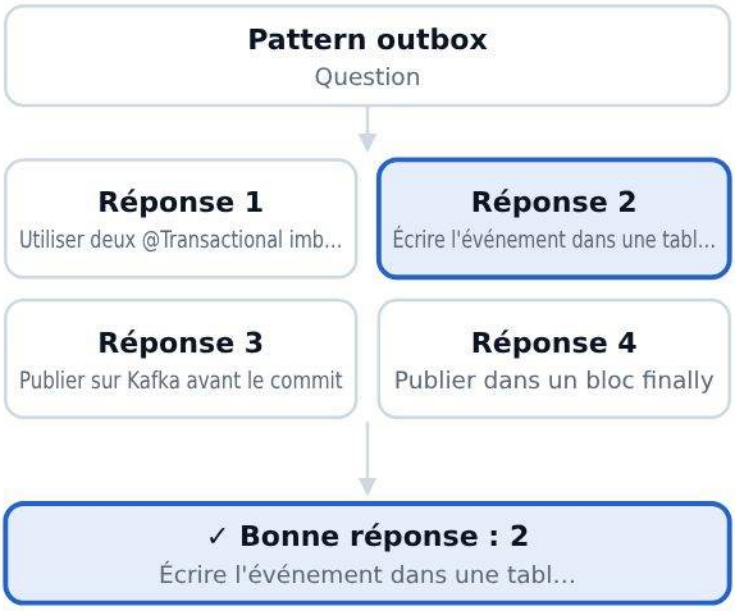


La réponse, étape par étape · *Step-by-step answer*

1. La question : Un client renvoie un POST /paiements après un timeout. Comment éviter un double paiement ?
   The question: A client resends POST /payments after a timeout. How do you avoid a double payment?
2. Les réponses possibles : 1. Une clé d'idempotence envoyée par le client et vérifiée côté serveur · 2. Passer la méthode en GET · 3. Augmenter le timeout du client · 4. Ajouter @Transactional sur le contrôleur
   Possible answers: 1. An idempotency key sent by the client and checked server-side · 2. Switch the method to GET · 3. Increase the client timeout · 4. Add @Transactional on the controller
3. La bonne réponse est la 1 : Une clé d'idempotence envoyée par le client et vérifiée côté serveur.
   The correct answer is 1: An idempotency key sent by the client and checked server-side.
4. Le serveur mémorise la clé (Idempotency-Key) et renvoie le résultat déjà produit au lieu de traiter deux fois la même demande.
   The server stores the key (Idempotency-Key) and returns the result already produced instead of processing the same request twice.

Pattern outbox

Outbox pattern

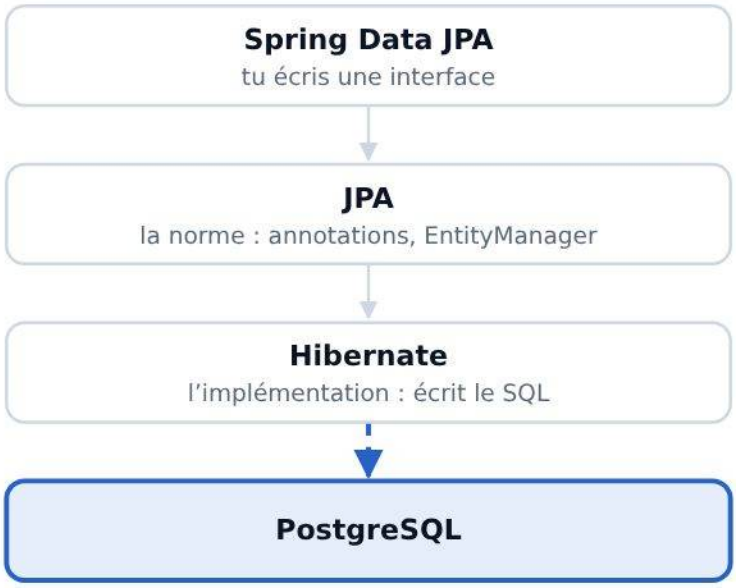


La réponse, étape par étape · *Step-by-step answer*

1. La question : Comment enregistrer une commande ET publier un événement Kafka sans risquer d'en perdre un ?
   The question: How do you save an order AND publish a Kafka event without risking losing one?
2. Les réponses possibles : 1. Utiliser deux @Transactional imbriquées · 2. Écrire l'événement dans une table outbox dans la même transaction, puis le publier à part · 3. Publier sur Kafka avant le commit · 4. Publier dans un bloc finally
   Possible answers: 1. Use two nested @Transactional · 2. Write the event to an outbox table in the same transaction, then publish it separately · 3. Publish to Kafka before the commit · 4. Publish in a finally block
3. La bonne réponse est la 2 : Écrire l'événement dans une table outbox dans la même transaction, puis le publier à part.
   The correct answer is 2: Write the event to an outbox table in the same transaction, then publish it separately.
4. La base et Kafka ne partagent pas de transaction. L'outbox garantit que l'événement est écrit avec la commande ; un relais le publie ensuite.
   The database and Kafka share no transaction. The outbox guarantees the event is written with the order; a relay publishes it afterwards.

JPA, Hibernate, Spring Data

JPA, Hibernate, Spring Data

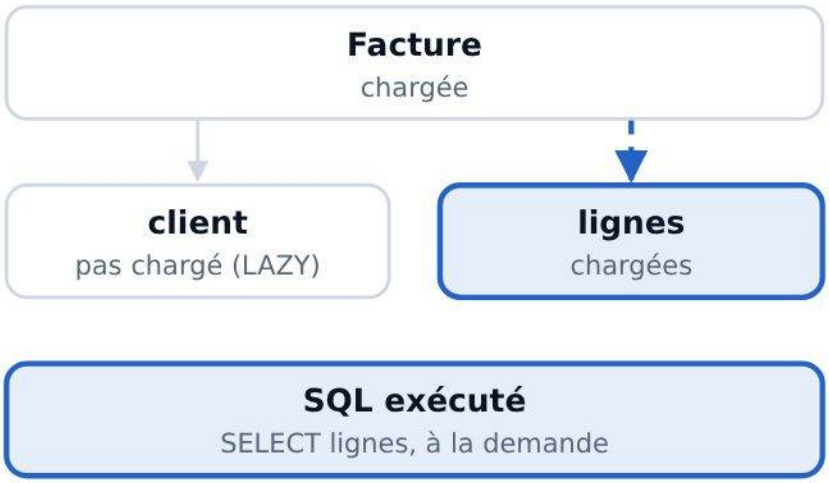


La réponse, étape par étape · *Step-by-step answer*

1. Tu déclares seulement une méthode dans une interface. Spring Data comprend son nom.
   You only declare a method in an interface. Spring Data understands its name.
2. Spring Data passe par JPA, la norme Java.
   Spring Data goes through JPA, the Java standard.
3. Hibernate, qui applique cette norme, écrit la requête SQL.
   Hibernate, which implements this standard, writes the SQL query.
4. Le SQL est exécuté sur PostgreSQL. Tu n'as écrit aucune requête.
   The SQL runs on PostgreSQL. You did not write a single query.

LAZY ou EAGER

LAZY or EAGER

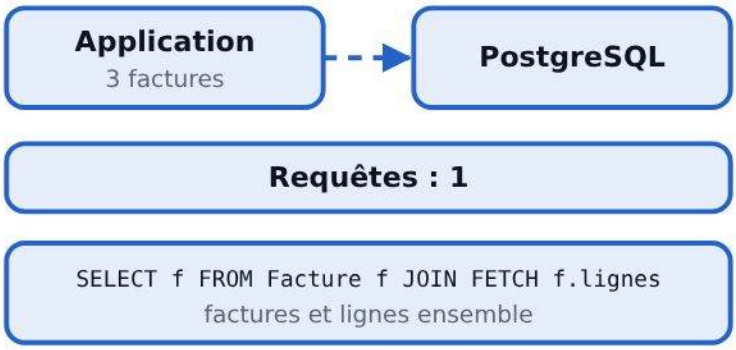


La réponse, étape par étape · *Step-by-step answer*

1. findById charge seulement la facture.
   findById loads only the invoice.
2. Le client et les lignes sont LAZY : Hibernate met juste une « boîte vide » à leur place.
   The customer and the lines are LAZY: Hibernate just puts an "empty box" in their place.
3. Au premier accès, Hibernate va chercher les lignes. En EAGER, elles auraient été chargées dès le début, même inutiles.
   On first access, Hibernate fetches the lines. With EAGER, they would have been loaded from the start, even if useless.

N+1

N+1



La réponse, étape par étape · *Step-by-step answer*

1. On demande toutes les factures : 1 requête. Mais leurs lignes ne sont pas encore chargées.
   We ask for all invoices: 1 query. But their lines are not loaded yet.
2. Dans la boucle, on demande les lignes de la facture 1 : Hibernate fait une nouvelle requête.
   In the loop, we ask for the lines of invoice 1: Hibernate runs a new query.
3. Facture 2 : encore une requête.
   Invoice 2: another query.
4. Facture 3 : encore une. 3 factures = 4 requêtes. 1 000 factures = 1 001 requêtes.
   Invoice 3: another one. 3 invoices = 4 queries. 1,000 invoices = 1,001 queries.
5. C'est le problème N+1 : 1 requête au départ, puis 1 de plus pour chaque facture. C'est très lent.
   This is the N+1 problem: 1 query at the start, then 1 more for each invoice. It is very slow.
6. La solution : JOIN FETCH. On charge factures et lignes en même temps, en 1 seule requête, quel que soit leur nombre.
   The solution: JOIN FETCH. We load invoices and lines together, in just 1 query, however many there are.

LazyInitializationException

LazyInitializationException

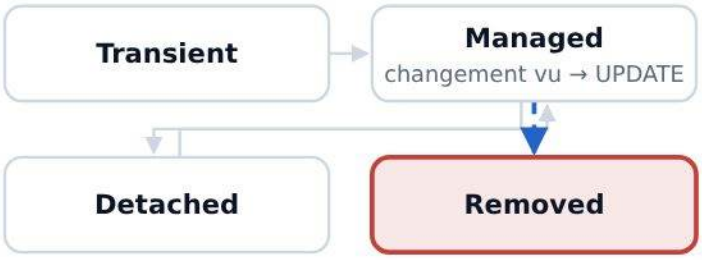


La réponse, étape par étape · *Step-by-step answer*

1. Dans le service, la session Hibernate est ouverte : la facture est chargée, sans ses lignes.
   In the service, the Hibernate session is open: the invoice is loaded, without its lines.
2. La méthode se termine : la session se ferme. La facture arrive dans le contrôleur.
   The method ends: the session closes. The invoice reaches the controller.
3. Le contrôleur demande les lignes, mais il n'y a plus de session pour les charger : erreur.
   The controller asks for the lines, but there is no session left to load them: error.
4. Solution : tout charger pendant que la session est encore ouverte.
   Solution: load everything while the session is still open.

États d’une entité

Entity states



La réponse, étape par étape · *Step-by-step answer*

1. new crée un objet Java normal : Hibernate ne le connaît pas.
   new creates a normal Java object: Hibernate does not know about it.
2. persist : l’entité devient « managed », suivie par Hibernate.
   persist: the entity becomes "managed", tracked by Hibernate.
3. Chaque changement est remarqué et enregistré au commit, sans appeler save().
   Every change is noticed and saved at commit, without calling save().
4. detach (ou fin de session) : l’entité n’est plus suivie. Ses changements sont ignorés.
   detach (or end of session): the entity is no longer tracked. Its changes are ignored.
5. merge : on la rattache, elle est de nouveau suivie.
   merge: we reattach it, it is tracked again.
6. remove : elle sera supprimée au commit.
   remove: it will be deleted at commit.

@Version

@Version

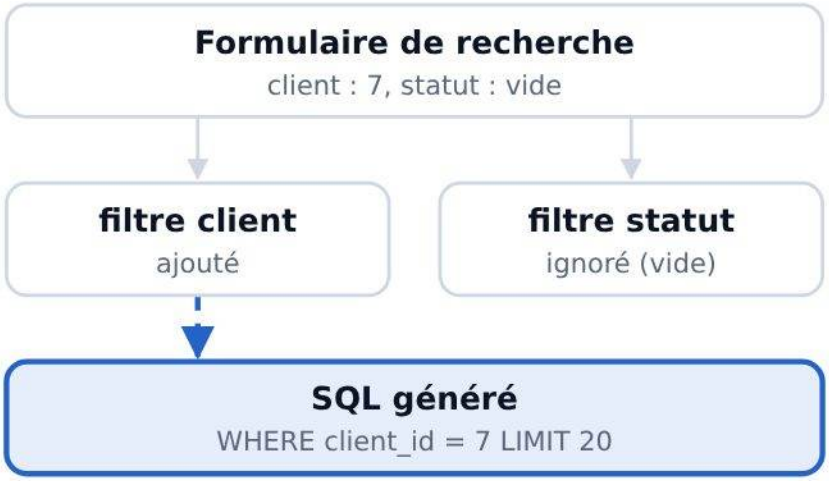


La réponse, étape par étape · *Step-by-step answer*

1. Ali et Sami ouvrent la même facture, en version 1.
   Ali and Sami open the same invoice, at version 1.
2. Ali enregistre en premier : la version passe à 2.
   Ali saves first: the version goes to 2.
3. Sami enregistre avec la version 1, qui n'existe plus : refusé. Il n'écrase pas le travail d'Ali.
   Sami saves with version 1, which no longer exists: rejected. He does not overwrite Ali's work.
4. On lui affiche « modifiée par quelqu'un d'autre » et il recharge la facture.
   He is shown "modified by someone else" and he reloads the invoice.

Filtres dynamiques

Dynamic filters



La réponse, étape par étape · *Step-by-step answer*

1. L'utilisateur remplit seulement le client.
   The user fills in only the customer.
2. Le client est renseigné : on ajoute la condition.
   The customer is filled in: we add the condition.
3. Le statut est vide : on n'ajoute rien.
   The status is empty: we add nothing.
4. La requête ne contient que les filtres utiles, avec la pagination.
   The query contains only the useful filters, with pagination.

save ou saveAndFlush

save or saveAndFlush



La réponse, étape par étape · *Step-by-step answer*

1. save garde f1 en mémoire. Le SQL peut attendre.
   save keeps f1 in memory. The SQL can wait.
2. saveAndFlush envoie tout de suite tout ce qui attend. Une erreur (doublon…) apparaît ici.
   saveAndFlush immediately sends everything that is waiting. An error (duplicate…) shows up here.
3. f3 attend à son tour.
   f3 waits in turn.
4. À la fin de la transaction, le reste est envoyé puis validé.
   At the end of the transaction, the rest is sent and then committed.

Relation bidirectionnelle

Bidirectional relationship

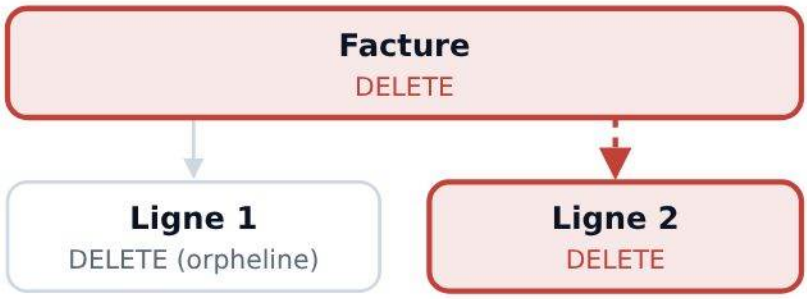


La réponse, étape par étape · *Step-by-step answer*

1. Côté Facture : la liste des lignes. mappedBy veut dire « ce n'est pas moi qui gère la colonne ».
   On the Facture side: the list of lines. mappedBy means "I am not the one managing the column".
2. Côté Ligne : @ManyToOne porte la vraie colonne facture_id. C'est le côté « propriétaire ».
   On the Ligne side: @ManyToOne holds the real facture_id column. It is the "owning" side.
3. ajouterLigne remplit les deux côtés en même temps, pour que Java et la base restent d'accord.
   ajouterLigne fills both sides at the same time, so that Java and the database stay in agreement.

cascade, orphanRemoval

cascade, orphanRemoval



La réponse, étape par étape · *Step-by-step answer*

1. cascade ALL : en enregistrant la facture, ses lignes sont enregistrées aussi.
   cascade ALL: when saving the invoice, its lines are saved too.
2. orphanRemoval : une ligne retirée de la liste est supprimée en base.
   orphanRemoval: a line removed from the list is deleted in the database.
3. Supprimer la facture supprime aussi ses lignes. À réserver aux enfants qui n'existent pas sans leur parent.
   Deleting the invoice also deletes its lines. Reserve this for children that cannot exist without their parent.

Pagination

Pagination



La réponse, étape par étape · *Step-by-step answer*

1. On demande la première page de 20 factures, les plus récentes d'abord.
   We ask for the first page of 20 invoices, most recent first.
2. Hibernate charge seulement 20 lignes, pas les 1 250.
   Hibernate loads only 20 rows, not all 1,250.
3. Une deuxième requête compte le total, pour afficher « page 1 sur 63 ».
   A second query counts the total, to display "page 1 of 63".
4. L'objet Page contient tout ce qu'il faut pour la pagination côté Angular.
   The Page object contains everything needed for pagination on the Angular side.

Projection

Projection

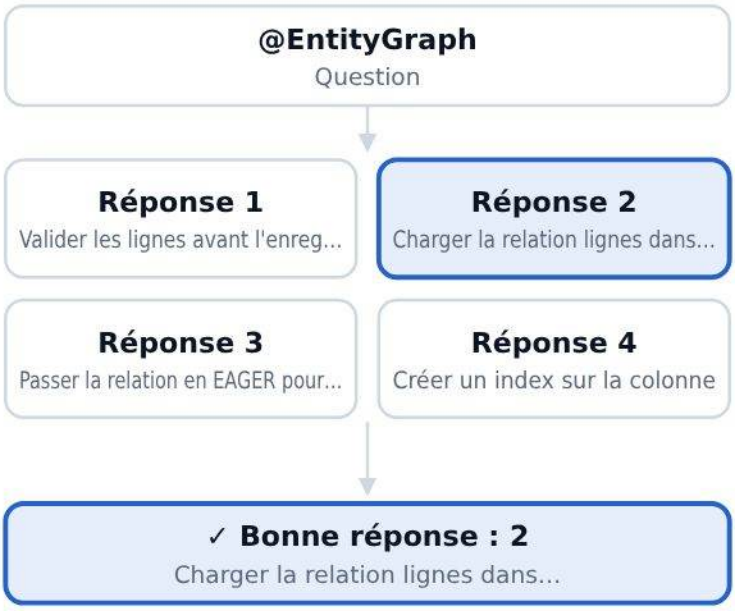


La réponse, étape par étape · *Step-by-step answer*

1. Pour une simple liste, charger l'entité complète ramène beaucoup de colonnes inutiles.
   For a simple list, loading the full entity brings back many useless columns.
2. Avec une projection, on ne lit que les 2 colonnes affichées : plus rapide.
   With a projection, we read only the 2 displayed columns: faster.
3. Un DTO n'a pas de relations LAZY : pas de LazyInitializationException possible.
   A DTO has no LAZY relationships: no LazyInitializationException possible.

@EntityGraph

@EntityGraph



La réponse, étape par étape · *Step-by-step answer*

1. La question : À quoi sert @EntityGraph(attributePaths = "lignes") sur une méthode de repository ?
   The question: What does @EntityGraph(attributePaths = "lines") on a repository method do?
2. Les réponses possibles : 1. Valider les lignes avant l'enregistrement · 2. Charger la relation lignes dans la même requête pour éviter le N+1 · 3. Passer la relation en EAGER pour toute l'application · 4. Créer un index sur la colonne
   Possible answers: 1. Validate lines before saving · 2. Load the lines relation in the same query to avoid N+1 · 3. Make the relation EAGER for the whole application · 4. Create an index on the column
3. La bonne réponse est la 2 : Charger la relation lignes dans la même requête pour éviter le N+1.
   The correct answer is 2: Load the lines relation in the same query to avoid N+1.
4. L'entity graph ajoute un fetch join pour cette requête seulement : la relation reste LAZY ailleurs.
   The entity graph adds a fetch join for that query only: the relation stays LAZY elsewhere.

Verrou pessimiste

Pessimistic lock

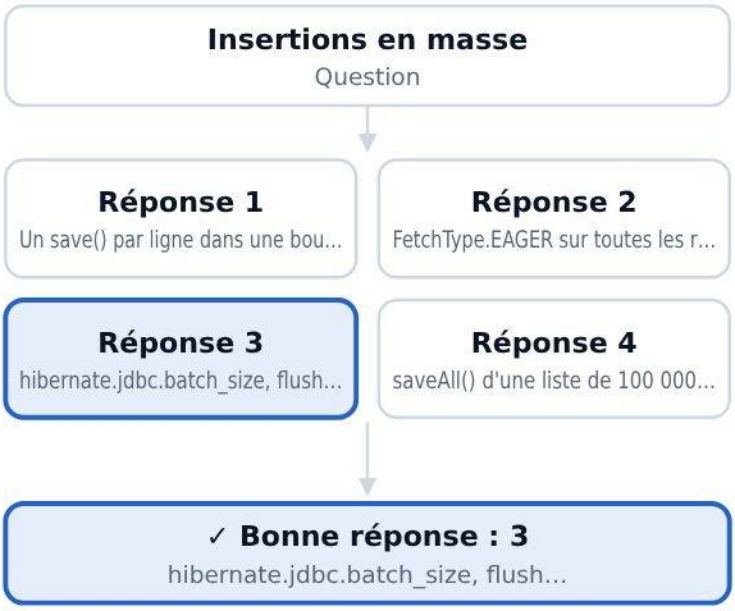


La réponse, étape par étape · *Step-by-step answer*

1. La question : Deux utilisateurs réservent le dernier article en même temps. Que fait @Lock(PESSIMISTIC_WRITE) ?
   The question: Two users book the last item at the same time. What does @Lock(PESSIMISTIC_WRITE) do?
2. Les réponses possibles : 1. Une requête en lecture seule · 2. Un SELECT ... FOR UPDATE qui bloque la ligne jusqu'à la fin de la transaction · 3. Une vérification de version au moment du commit · 4. Un cache de second niveau
   Possible answers: 1. A read-only query · 2. A SELECT ... FOR UPDATE that locks the row until the transaction ends · 3. A version check at commit time · 4. A second-level cache
3. La bonne réponse est la 2 : Un SELECT ... FOR UPDATE qui bloque la ligne jusqu'à la fin de la transaction.
   The correct answer is 2: A SELECT ... FOR UPDATE that locks the row until the transaction ends.
4. Le verrou pessimiste pose un verrou en base ; l'optimiste (@Version) laisse passer et détecte le conflit au commit.
   The pessimistic lock takes a database lock; the optimistic one (@Version) lets it through and detects the conflict at commit.

Insertions en masse

Bulk inserts



La réponse, étape par étape · *Step-by-step answer*

1. La question : Pour insérer 100 000 lignes avec Hibernate, quelle combinaison est la plus efficace ?
   The question: To insert 100,000 rows with Hibernate, which combination is most efficient?
2. Les réponses possibles : 1. Un save() par ligne dans une boucle sans transaction · 2. FetchType.EAGER sur toutes les relations · 3. hibernate.jdbc.batch_size, flush() et clear() réguliers, ids non IDENTITY · 4. saveAll() d'une liste de 100 000 entités d'un coup
   Possible answers: 1. One save() per row in a loop without a transaction · 2. FetchType.EAGER on every relation · 3. hibernate.jdbc.batch_size, regular flush() and clear(), non-IDENTITY ids · 4. saveAll() on a list of 100,000 entities at once
3. La bonne réponse est la 3 : hibernate.jdbc.batch_size, flush() et clear() réguliers, ids non IDENTITY.
   The correct answer is 3: hibernate.jdbc.batch_size, regular flush() and clear(), non-IDENTITY ids.
4. Le batch JDBC regroupe les INSERT ; flush/clear vident le contexte de persistance. Avec IDENTITY, Hibernate désactive le batch d'insertion.
   JDBC batching groups INSERTs; flush/clear empty the persistence context. With IDENTITY, Hibernate disables insert batching.

Cache de second niveau

Second-level cache

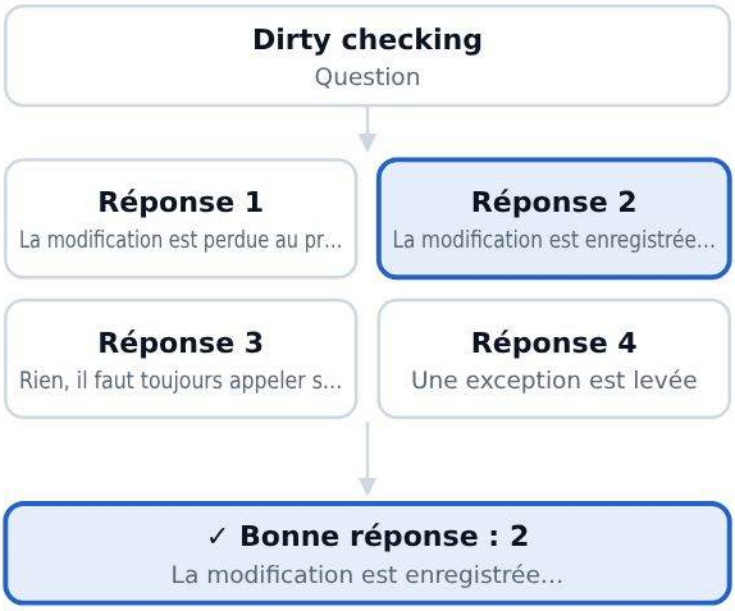


La réponse, étape par étape · *Step-by-step answer*

1. La question : Quelle différence entre le cache de 1er et de 2e niveau d'Hibernate ?
   The question: What is the difference between Hibernate's first- and second-level caches?
2. Les réponses possibles : 1. Le 1er vit dans la session (transaction), le 2e est partagé entre sessions · 2. Le 1er est sur disque, le 2e en mémoire · 3. Le 2e est activé par défaut · 4. Le 1er ne concerne que les requêtes JPQL
   Possible answers: 1. The first lives in the session (transaction), the second is shared across sessions · 2. The first is on disk, the second in memory · 3. The second is on by default · 4. The first only applies to JPQL queries
3. La bonne réponse est la 1 : Le 1er vit dans la session (transaction), le 2e est partagé entre sessions.
   The correct answer is 1: The first lives in the session (transaction), the second is shared across sessions.
4. Le cache de 1er niveau est le contexte de persistance d'une session ; le 2e niveau (Ehcache, Caffeine...) est optionnel et partagé.
   The first-level cache is a session's persistence context; the second level (Ehcache, Caffeine...) is optional and shared.

Dirty checking

Dirty checking

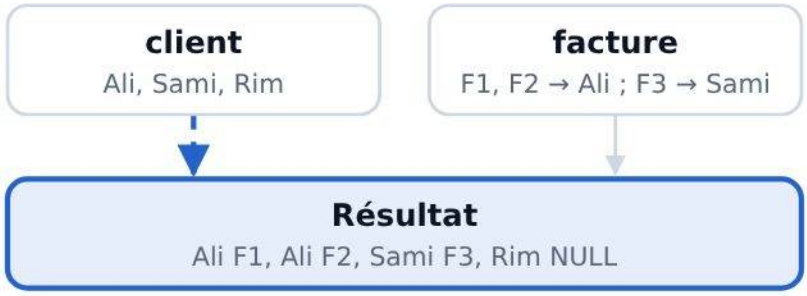


La réponse, étape par étape · *Step-by-step answer*

1. La question : Dans une méthode @Transactional, on modifie une entité chargée sans appeler save(). Que se passe-t-il ?
   The question: In a @Transactional method, you modify a loaded entity without calling save(). What happens?
2. Les réponses possibles : 1. La modification est perdue au prochain find() · 2. La modification est enregistrée au commit grâce au dirty checking · 3. Rien, il faut toujours appeler save() · 4. Une exception est levée
   Possible answers: 1. The change is lost at the next find() · 2. The change is saved at commit thanks to dirty checking · 3. Nothing, save() is always required · 4. An exception is thrown
3. La bonne réponse est la 2 : La modification est enregistrée au commit grâce au dirty checking.
   The correct answer is 2: The change is saved at commit thanks to dirty checking.
4. Une entité gérée est surveillée : au flush, Hibernate compare son état et génère l'UPDATE automatiquement.
   A managed entity is tracked: at flush, Hibernate compares its state and generates the UPDATE automatically.

Jointures

Joins



La réponse, étape par étape · *Step-by-step answer*

1. Deux tables : 3 clients, 3 factures. Rim n'a aucune facture.
   Two tables: 3 customers, 3 invoices. Rim has no invoice.
2. INNER JOIN : seulement les clients qui ont une facture. Rim disparaît.
   INNER JOIN: only customers who have an invoice. Rim disappears.
3. LEFT JOIN : tous les clients. Rim apparaît, avec NULL à la place du numéro.
   LEFT JOIN: all customers. Rim appears, with NULL instead of the number.

WHERE ou HAVING

WHERE or HAVING



La réponse, étape par étape · *Step-by-step answer*

1. On part de toutes les factures.
   We start from all invoices.
2. WHERE filtre les lignes une par une, avant tout regroupement.
   WHERE filters rows one by one, before any grouping.
3. GROUP BY regroupe par client et calcule la somme.
   GROUP BY groups by customer and calculates the sum.
4. HAVING filtre les groupes, avec la somme. WHERE ne peut pas le faire.
   HAVING filters groups, using the sum. WHERE cannot do that.

Index

Indexes

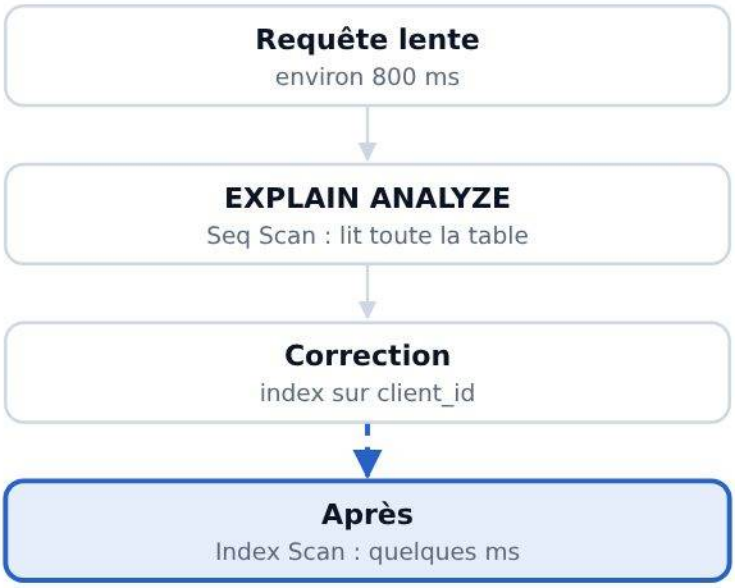


La réponse, étape par étape · *Step-by-step answer*

1. Sans index, PostgreSQL lit toute la table, ligne par ligne.
   Without an index, PostgreSQL reads the whole table, row by row.
2. On crée un index : une sorte de sommaire trié de la colonne.
   We create an index: a kind of sorted table of contents for the column.
3. La recherche va directement à la bonne ligne, comme l'index d'un livre. En échange, les écritures sont un peu plus lentes.
   The search goes straight to the right row, like the index of a book. In exchange, writes are a bit slower.

Requête lente

Slow query



La réponse, étape par étape · *Step-by-step answer*

1. Une requête est lente.
   A query is slow.
2. EXPLAIN ANALYZE montre comment PostgreSQL l'exécute vraiment.
   EXPLAIN ANALYZE shows how PostgreSQL really runs it.
3. On ajoute l'index qui manque.
   We add the missing index.
4. On vérifie à nouveau : la requête est devenue très rapide.
   We check again: the query has become very fast.

ACID

ACID



La réponse, étape par étape · *Step-by-step answer*

1. Atomicité : la facture payée et le paiement sont enregistrés ensemble, ou pas du tout.
   Atomicity: the paid invoice and the payment are saved together, or not at all.
2. Cohérence : les règles (clé étrangère, montant positif…) sont toujours respectées.
   Consistency: the rules (foreign key, positive amount…) are always respected.
3. Isolation : pendant ce temps, les autres utilisateurs ne voient pas le travail à moitié fait.
   Isolation: meanwhile, other users do not see the half-done work.
4. Durabilité : après COMMIT, c’est enregistré, même si le serveur tombe juste après.
   Durability: after COMMIT, it is saved, even if the server crashes right after.

Fonctions de fenêtrage

Window functions



La réponse, étape par étape · *Step-by-step answer*

1. Toutes les factures. Chaque ligne est gardée, sans regroupement.
   All invoices. Each row is kept, with no grouping.
2. PARTITION BY fait un groupe par client. Pour Ali, les factures sont classées de la plus récente à la plus ancienne.
   PARTITION BY makes one group per customer. For Ali, the invoices are ranked from most recent to oldest.
3. Même chose pour Sami. Les lignes restent séparées, contrairement à GROUP BY.
   Same for Sami. Rows stay separate, unlike GROUP BY.
4. On garde rang = 1 : la dernière facture de chaque client.
   We keep rang = 1: each customer's latest invoice.

DELETE, TRUNCATE, DROP

DELETE, TRUNCATE, DROP



La réponse, étape par étape · *Step-by-step answer*

1. DELETE retire les lignes choisies.
   DELETE removes the selected rows.
2. TRUNCATE vide toute la table d’un coup.
   TRUNCATE empties the whole table at once.
3. DROP supprime la table elle-même.
   DROP deletes the table itself.
4. En facturation, on garde l’historique : on change le statut au lieu de supprimer.
   In invoicing, we keep the history: we change the status instead of deleting.

Clés

Keys



La réponse, étape par étape · *Step-by-step answer*

1. La clé primaire identifie chaque facture de façon unique.
   The primary key uniquely identifies each invoice.
2. La clé étrangère relie chaque ligne à une facture qui existe.
   The foreign key links each line to an invoice that exists.
3. PostgreSQL refuse une ligne qui pointe vers une facture inexistante.
   PostgreSQL rejects a line that points to a non-existent invoice.

Normalisation

Normalization

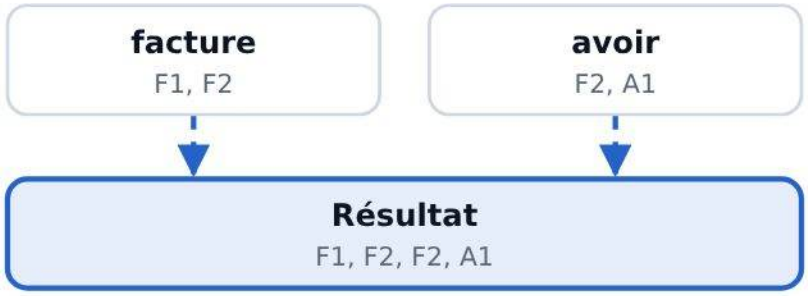


La réponse, étape par étape · *Step-by-step answer*

1. Avant : l'adresse d'Ali est recopiée dans chaque facture.
   Before: Ali's address is copied into every invoice.
2. Ali déménage : il faut modifier toutes les factures. Si on en oublie une, les données se contredisent.
   Ali moves: every invoice must be changed. If we forget one, the data contradicts itself.
3. Après : l'adresse est stockée une seule fois. Une seule modification suffit.
   After: the address is stored only once. A single change is enough.

UNION

UNION



La réponse, étape par étape · *Step-by-step answer*

1. Deux requêtes, avec une valeur en commun : F2.
   Two queries, with one value in common: F2.
2. UNION empile les résultats et retire les doublons : il doit comparer, donc c'est plus lent.
   UNION stacks the results and removes duplicates: it has to compare, so it is slower.
3. UNION ALL empile tout, sans vérifier : plus rapide.
   UNION ALL stacks everything, without checking: faster.

EXISTS

EXISTS



La réponse, étape par étape · *Step-by-step answer*

1. Pour chaque client, une question : a-t-il au moins une facture impayée ?
   For each customer, one question: do they have at least one unpaid invoice?
2. EXISTS parcourt ses factures et s'arrête dès la première impayée.
   EXISTS goes through their invoices and stops at the first unpaid one.
3. Ali est gardé. Pas besoin de compter toutes ses factures : c'est souvent rapide.
   Ali is kept. No need to count all his invoices: it is often fast.

Vues

Views

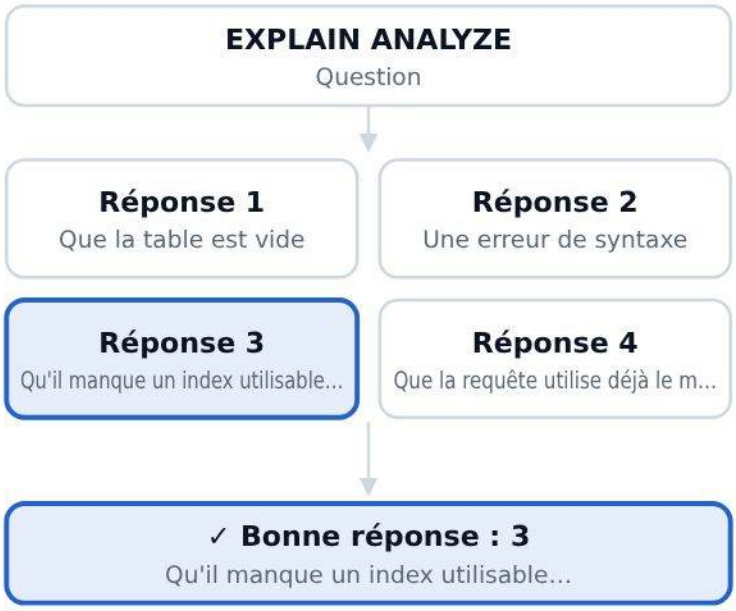


La réponse, étape par étape · *Step-by-step answer*

1. Une vue est une requête enregistrée sous un nom.
   A view is a query saved under a name.
2. Chaque fois qu'on l'interroge, PostgreSQL relance la requête sur la table.
   Each time we query it, PostgreSQL reruns the query on the table.
3. Une vue matérialisée garde le résultat déjà calculé : lecture très rapide.
   A materialized view keeps the already calculated result: very fast reads.
4. Mais il faut la rafraîchir pour voir les nouvelles factures.
   But it must be refreshed to see new invoices.

EXPLAIN ANALYZE

EXPLAIN ANALYZE

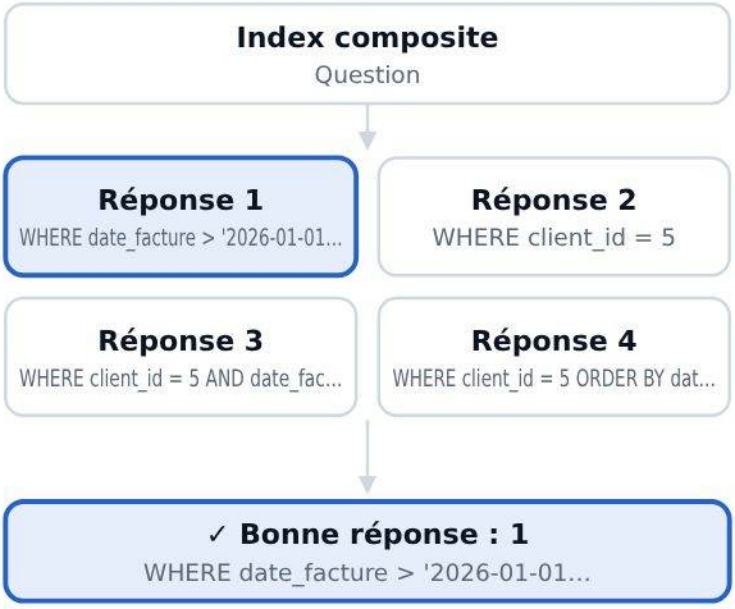


La réponse, étape par étape · *Step-by-step answer*

1. La question : Dans un EXPLAIN ANALYZE, un « Seq Scan » sur une grosse table filtrée par client_id indique souvent :
   The question: In an EXPLAIN ANALYZE, a "Seq Scan" on a large table filtered by client_id often means:
2. Les réponses possibles : 1. Que la table est vide · 2. Une erreur de syntaxe · 3. Qu'il manque un index utilisable sur client_id · 4. Que la requête utilise déjà le meilleur index
   Possible answers: 1. The table is empty · 2. A syntax error · 3. A usable index on client_id is missing · 4. The query already uses the best index
3. La bonne réponse est la 3 : Qu'il manque un index utilisable sur client_id.
   The correct answer is 3: A usable index on client_id is missing.
4. Seq Scan = lecture de toute la table. Sur un filtre sélectif, un index (Index Scan) est en général bien plus rapide.
   Seq Scan = reading the whole table. On a selective filter, an index (Index Scan) is usually much faster.

Index composite

Composite index



La réponse, étape par étape · *Step-by-step answer*

1. La question : Avec un index sur (client_id, date_facture), quelle requête ne peut PAS l'utiliser efficacement ?
   The question: With an index on (client_id, invoice_date), which query can NOT use it efficiently?
2. Les réponses possibles : 1. WHERE date_facture > '2026-01-01' seule · 2. WHERE client_id = 5 · 3. WHERE client_id = 5 AND date_facture > '2026-01-01' · 4. WHERE client_id = 5 ORDER BY date_facture
   Possible answers: 1. WHERE invoice_date > '2026-01-01' alone · 2. WHERE client_id = 5 · 3. WHERE client_id = 5 AND invoice_date > '2026-01-01' · 4. WHERE client_id = 5 ORDER BY invoice_date
3. La bonne réponse est la 1 : WHERE date_facture > '2026-01-01' seule.
   The correct answer is 1: WHERE invoice_date > '2026-01-01' alone.
4. Un index composite se lit de gauche à droite : sans la première colonne (client_id), il n'est pas exploitable comme index de recherche.
   A composite index is read left to right: without its first column (client_id), it cannot be used as a search index.

Niveaux d'isolation

Isolation levels



La réponse, étape par étape · *Step-by-step answer*

1. La question : Quel est le niveau d'isolation par défaut de PostgreSQL ?
   The question: What is PostgreSQL's default isolation level?
2. Les réponses possibles : 1. SERIALIZABLE · 2. REPEATABLE READ · 3. READ UNCOMMITTED · 4. READ COMMITTED
   Possible answers: 1. SERIALIZABLE · 2. REPEATABLE READ · 3. READ UNCOMMITTED · 4. READ COMMITTED
3. La bonne réponse est la 4 : READ COMMITTED.
   The correct answer is 4: READ COMMITTED.
4. PostgreSQL est en READ COMMITTED par défaut : chaque requête voit les données validées au moment où elle démarre.
   PostgreSQL defaults to READ COMMITTED: each statement sees data committed when it starts.

Deadlock

Deadlock

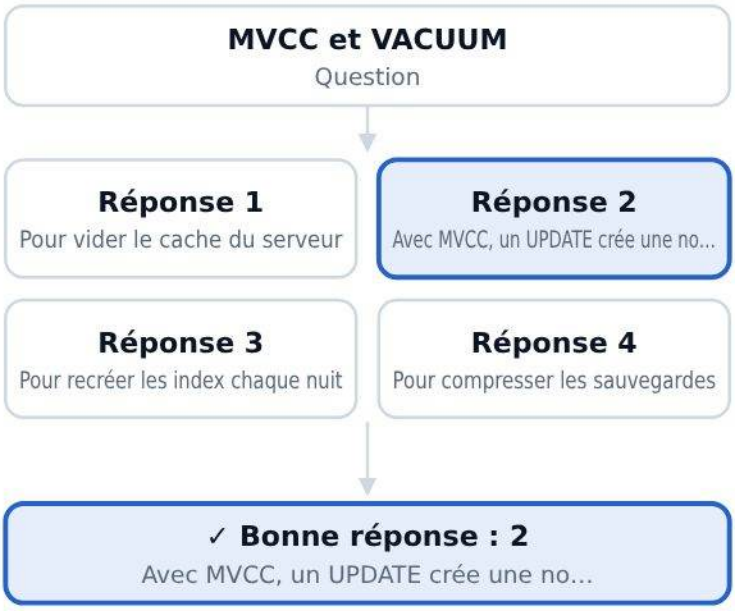


La réponse, étape par étape · *Step-by-step answer*

1. La question : Deux transactions mettent à jour les lignes A puis B, et B puis A. Comment éviter le deadlock ?
   The question: Two transactions update rows A then B, and B then A. How do you avoid the deadlock?
2. Les réponses possibles : 1. Toujours verrouiller les lignes dans le même ordre · 2. Augmenter le nombre de connexions · 3. Supprimer les index · 4. Passer en READ UNCOMMITTED
   Possible answers: 1. Always lock rows in the same order · 2. Increase the number of connections · 3. Drop the indexes · 4. Switch to READ UNCOMMITTED
3. La bonne réponse est la 1 : Toujours verrouiller les lignes dans le même ordre.
   The correct answer is 1: Always lock rows in the same order.
4. Un deadlock vient d'un ordre d'acquisition inversé. Un ordre constant (par id) supprime le cycle ; PostgreSQL annule sinon l'une des deux.
   A deadlock comes from reversed lock order. A consistent order (by id) removes the cycle; otherwise PostgreSQL aborts one of them.

MVCC et VACUUM

MVCC and VACUUM

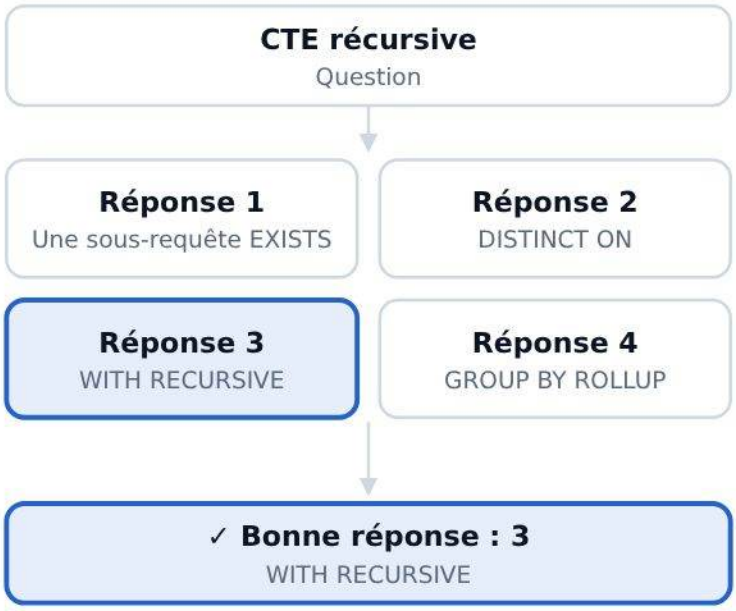


La réponse, étape par étape · *Step-by-step answer*

1. La question : Pourquoi PostgreSQL a-t-il besoin de VACUUM ?
   The question: Why does PostgreSQL need VACUUM?
2. Les réponses possibles : 1. Pour vider le cache du serveur · 2. Avec MVCC, un UPDATE crée une nouvelle version de ligne : les anciennes doivent être nettoyées · 3. Pour recréer les index chaque nuit · 4. Pour compresser les sauvegardes
   Possible answers: 1. To clear the server cache · 2. With MVCC, an UPDATE creates a new row version: old ones must be cleaned up · 3. To rebuild indexes every night · 4. To compress backups
3. La bonne réponse est la 2 : Avec MVCC, un UPDATE crée une nouvelle version de ligne : les anciennes doivent être nettoyées.
   The correct answer is 2: With MVCC, an UPDATE creates a new row version: old ones must be cleaned up.
4. MVCC garde les anciennes versions pour les transactions en cours ; (auto)VACUUM récupère l'espace des versions mortes.
   MVCC keeps old versions for running transactions; (auto)VACUUM reclaims the space of dead versions.

CTE récursive

Recursive CTE



La réponse, étape par étape · *Step-by-step answer*

1. La question : Quel outil SQL permet de parcourir une hiérarchie (catégories parentes / enfants) en une requête ?
   The question: Which SQL tool walks a hierarchy (parent / child categories) in one query?
2. Les réponses possibles : 1. Une sous-requête EXISTS · 2. DISTINCT ON · 3. WITH RECURSIVE · 4. GROUP BY ROLLUP
   Possible answers: 1. An EXISTS subquery · 2. DISTINCT ON · 3. WITH RECURSIVE · 4. GROUP BY ROLLUP
3. La bonne réponse est la 3 : WITH RECURSIVE.
   The correct answer is 3: WITH RECURSIVE.
4. Une CTE récursive part d'une ligne racine puis rejoint la table à elle-même jusqu'à épuiser les enfants.
   A recursive CTE starts from a root row then joins the table to itself until no children remain.

Pagination par clé

Keyset pagination

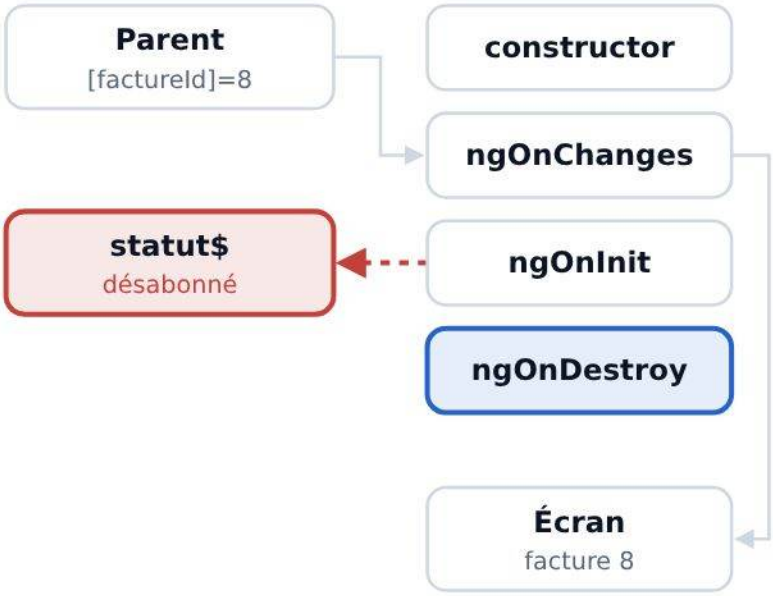


La réponse, étape par étape · *Step-by-step answer*

1. La question : Pourquoi OFFSET 100000 LIMIT 20 est-il lent, et quelle alternative ?
   The question: Why is OFFSET 100000 LIMIT 20 slow, and what is the alternative?
2. Les réponses possibles : 1. OFFSET n'utilise jamais d'index ; il faut DISTINCT · 2. LIMIT force un tri complet ; il faut GROUP BY · 3. C'est rapide, aucune alternative n'est utile · 4. La base lit et jette 100 000 lignes ; on pagine plutôt avec WHERE id > dernier_id
   Possible answers: 1. OFFSET never uses an index; use DISTINCT · 2. LIMIT forces a full sort; use GROUP BY · 3. It is fast, no alternative is needed · 4. The database reads and discards 100,000 rows; paginate with WHERE id > last_id instead
3. La bonne réponse est la 4 : La base lit et jette 100 000 lignes ; on pagine plutôt avec WHERE id > dernier_id.
   The correct answer is 4: The database reads and discards 100,000 rows; paginate with WHERE id > last_id instead.
4. OFFSET parcourt toutes les lignes sautées. La pagination par clé (keyset) repart du dernier id vu grâce à l'index.
   OFFSET walks through every skipped row. Keyset pagination resumes from the last seen id using the index.

Cycle de vie

Lifecycle

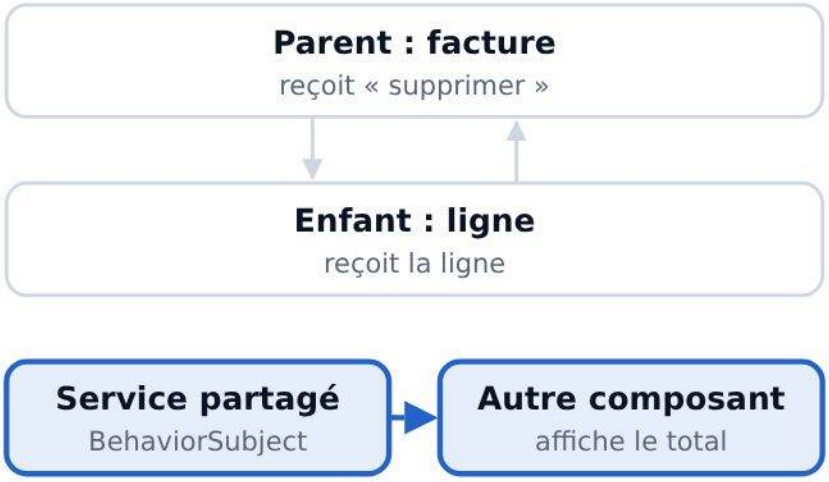


La réponse, étape par étape · *Step-by-step answer*

1. Angular crée le composant. Le constructeur sert seulement à recevoir les services, rien d'autre.
   Angular creates the component. The constructor is only used to receive services, nothing else.
2. Le parent donne factureId = 7. ngOnChanges se déclenche et charge la facture 7.
   The parent passes factureId = 7. ngOnChanges fires and loads invoice 7.
3. ngOnInit se lance une seule fois. On s'abonne aux mises à jour du statut.
   ngOnInit runs only once. We subscribe to status updates.
4. Le parent passe à la facture 8. ngOnChanges se relance et charge la 8. ngOnInit, lui, ne se relance pas.
   The parent switches to invoice 8. ngOnChanges fires again and loads invoice 8. ngOnInit does not run again.
5. On quitte l'écran : ngOnDestroy arrête l'abonnement. Sinon, il continuerait à tourner pour rien (fuite mémoire).
   We leave the screen: ngOnDestroy stops the subscription. Otherwise, it would keep running for nothing (memory leak).

Communication

Communication

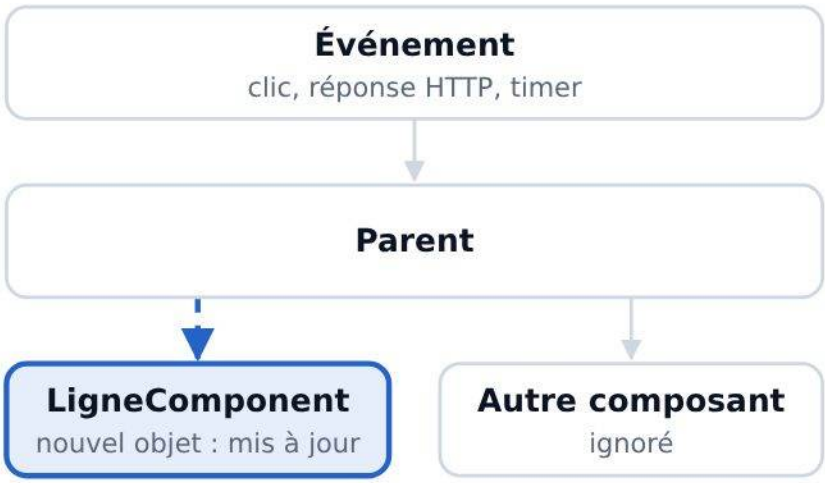


La réponse, étape par étape · *Step-by-step answer*

1. @Input : le parent donne une donnée à l'enfant.
   @Input: the parent gives data to the child.
2. @Output : l'enfant prévient le parent qu'il s'est passé quelque chose.
   @Output: the child tells the parent that something happened.
3. Pour des composants éloignés, on passe par un service partagé : l'un envoie, l'autre écoute.
   For distant components, we go through a shared service: one sends, the other listens.

Détection des changements

Change detection

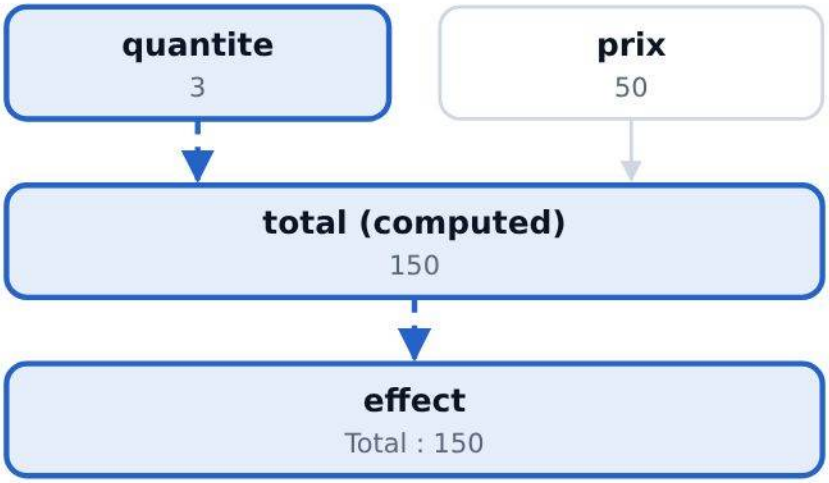


La réponse, étape par étape · *Step-by-step answer*

1. Après chaque événement, Angular vérifie les composants pour mettre l'écran à jour.
   After each event, Angular checks the components to update the screen.
2. Avec OnPush, un composant n'est vérifié que si son @Input change de référence (ou s'il a son propre événement).
   With OnPush, a component is checked only if its @Input changes reference (or if it has its own event).
3. Modifier l'objet existant ne change pas sa référence : l'écran n'est pas mis à jour.
   Modifying the existing object does not change its reference: the screen is not updated.
4. On crée un nouvel objet : Angular voit le changement. D'où l'habitude de ne pas modifier les objets.
   We create a new object: Angular sees the change. Hence the habit of not modifying objects.

Signals

Signals



La réponse, étape par étape · *Step-by-step answer*

1. Deux signals : des valeurs qu'Angular surveille.
   Two signals: values that Angular watches.
2. computed calcule le total à partir d'eux, et sait de qui il dépend.
   computed calculates the total from them, and knows what it depends on.
3. effect s'exécute à chaque changement du total.
   effect runs every time the total changes.
4. quantite passe à 3 : seul ce qui en dépend est recalculé, automatiquement.
   quantite goes to 3: only what depends on it is recalculated, automatically.

Standalone

Standalone

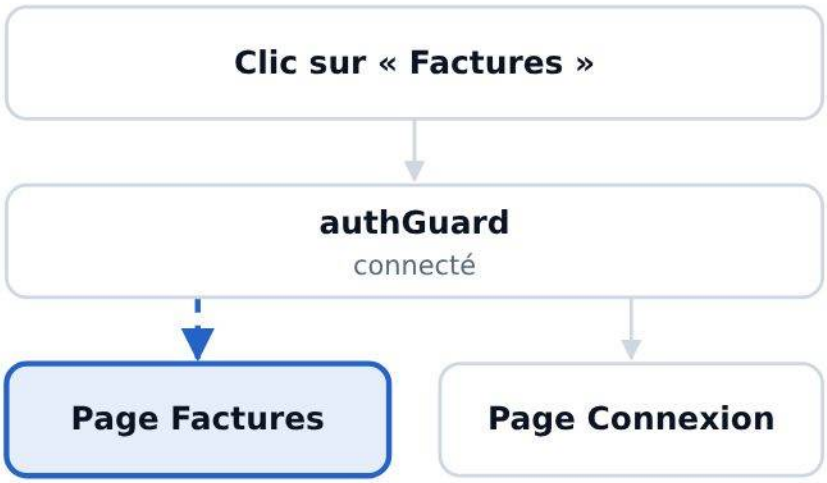


La réponse, étape par étape · *Step-by-step answer*

1. Avant, chaque composant devait être déclaré dans un NgModule.
   Before, each component had to be declared in an NgModule.
2. Un composant standalone importe directement ce dont il a besoin.
   A standalone component directly imports what it needs.
3. Il peut être chargé à la demande depuis une route, sans module intermédiaire.
   It can be loaded on demand from a route, without an intermediate module.

Guards

Guards

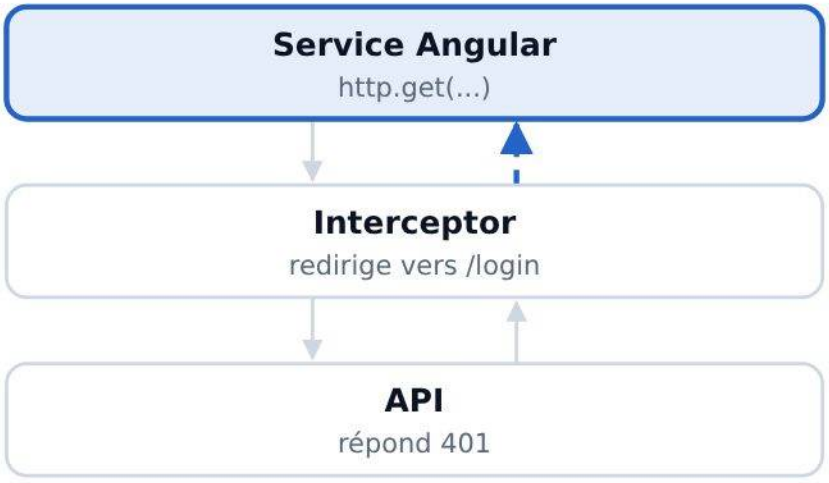


La réponse, étape par étape · *Step-by-step answer*

1. Avant d’ouvrir la page, Angular exécute le guard.
   Before opening the page, Angular runs the guard.
2. Utilisateur non connecté : le guard l’envoie vers la page de connexion.
   User not logged in: the guard sends them to the login page.
3. Utilisateur connecté : la page s’ouvre normalement.
   User logged in: the page opens normally.

Interceptor

Interceptor

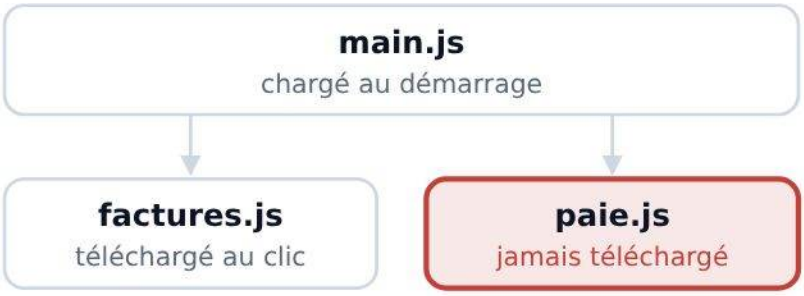


La réponse, étape par étape · *Step-by-step answer*

1. Un service lance une requête, sans s'occuper du token.
   A service sends a request, without worrying about the token.
2. L'interceptor ajoute le token à chaque requête, à un seul endroit.
   The interceptor adds the token to every request, in a single place.
3. La réponse revient en 401 : l'interceptor redirige vers la connexion, pour tous les écrans.
   The response comes back as 401: the interceptor redirects to login, for all screens.
4. L'erreur continue jusqu'au service, qui peut afficher un message.
   The error continues up to the service, which can display a message.

Lazy loading

Lazy loading

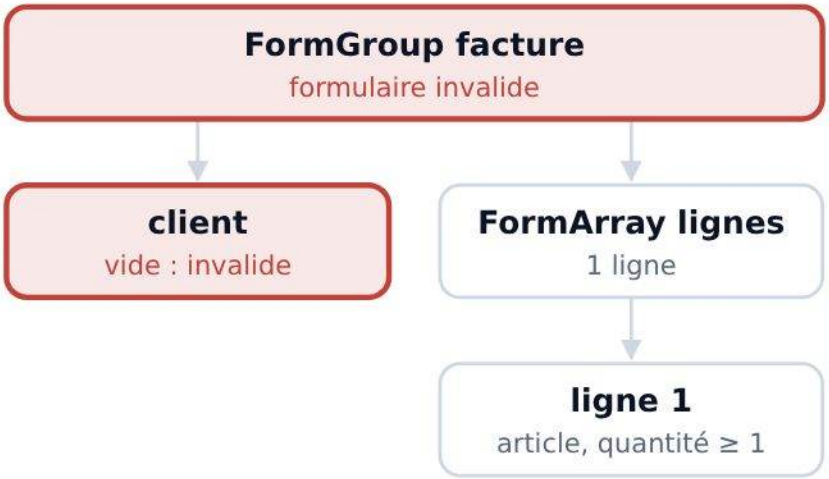


La réponse, étape par étape · *Step-by-step answer*

1. Au démarrage, seul le code de base est téléchargé : premier affichage rapide.
   At startup, only the base code is downloaded: fast first display.
2. L'utilisateur va sur /factures : ce code est téléchargé à ce moment-là.
   The user goes to /factures: that code is downloaded at that moment.
3. S'il ne va jamais sur la paie, ce code n'est jamais téléchargé.
   If they never go to payroll, that code is never downloaded.

Reactive forms

Reactive forms



La réponse, étape par étape · *Step-by-step answer*

1. Le formulaire est décrit en TypeScript : un client et une liste de lignes.
   The form is described in TypeScript: a customer and a list of lines.
2. Un clic sur « Ajouter » crée une ligne, avec ses propres règles.
   A click on "Add" creates a line, with its own rules.
3. Si le client est vide, tout le formulaire est invalide : on désactive le bouton Enregistrer.
   If the customer is empty, the whole form is invalid: we disable the Save button.

Longues listes

Long lists



La réponse, étape par étape · *Step-by-step answer*

1. track dit comment reconnaître chaque élément. Après un rechargement, Angular ne recrée que ce qui a changé.
   track says how to recognize each item. After a reload, Angular recreates only what changed.
2. Afficher 1 000 lignes d’un coup alourdit fortement la page.
   Displaying 1,000 rows at once makes the page much heavier.
3. Le virtual scroll n’affiche que les lignes visibles et les remplace pendant le défilement.
   Virtual scroll displays only the visible rows and swaps them while scrolling.

Pipes

Pipes

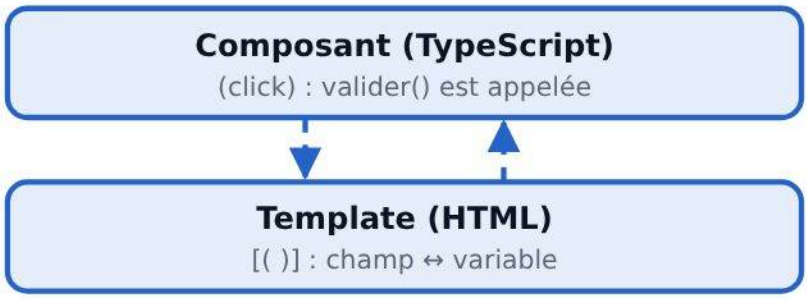


La réponse, étape par étape · *Step-by-step answer*

1. Premier affichage : les deux pipes calculent « 100.000 TND ».
   First display: both pipes calculate "100.000 TND".
2. Nouvelle vérification, total inchangé : le pipe pur ne refait rien, l'impur recalcule.
   New check, total unchanged: the pure pipe does nothing again, the impure one recalculates.
3. La valeur change : le pipe pur recalcule. Pur = calcul seulement quand c'est nécessaire.
   The value changes: the pure pipe recalculates. Pure = calculate only when needed.

Data binding

Data binding

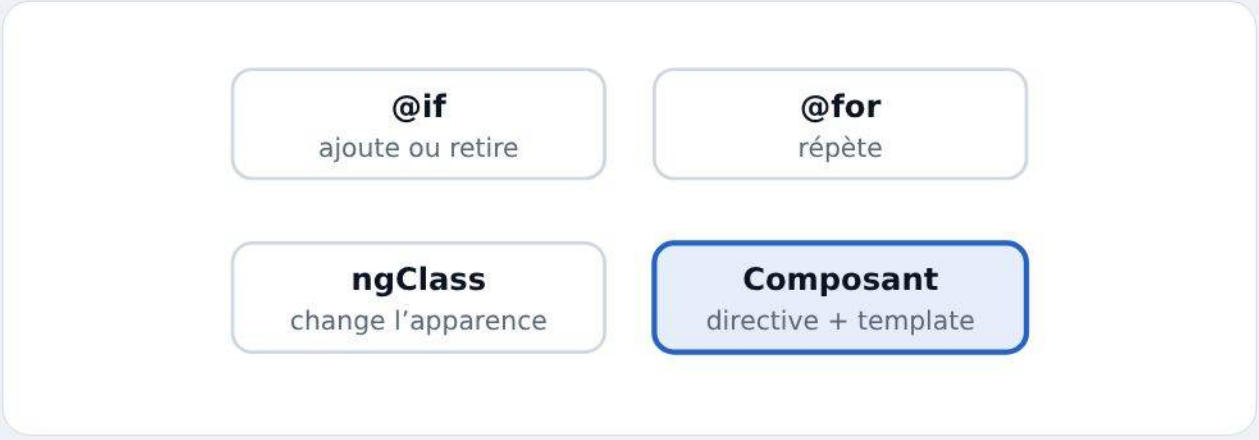


La réponse, étape par étape · *Step-by-step answer*

1. Interpolation : la valeur du code s'affiche dans la page.
   Interpolation: the value from the code is displayed on the page.
2. Property binding : le code contrôle une propriété HTML.
   Property binding: the code controls an HTML property.
3. Event binding : la page prévient le code qu'un événement a eu lieu.
   Event binding: the page tells the code that an event happened.
4. Two-way binding : le champ et la variable restent synchronisés dans les deux sens.
   Two-way binding: the field and the variable stay in sync in both directions.

Directives

Directives

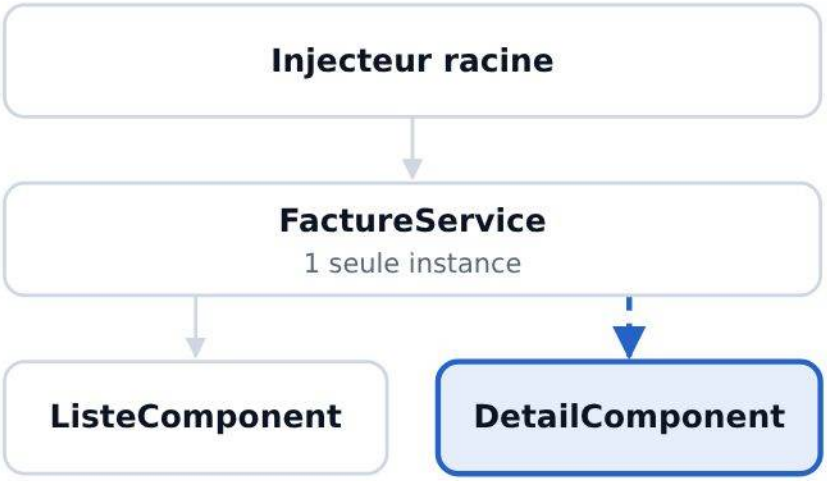


La réponse, étape par étape · *Step-by-step answer*

1. Directive structurelle : @if ajoute ou retire un bloc selon une condition.
   Structural directive: @if adds or removes a block based on a condition.
2. @for répète un bloc pour chaque élément de la liste.
   @for repeats a block for each item in the list.
3. Directive d'attribut : ngClass change le style, sans ajouter ni retirer d'élément.
   Attribute directive: ngClass changes the style, without adding or removing any element.
4. Un composant est une directive avec son propre template.
   A component is a directive with its own template.

Injection Angular

Angular injection

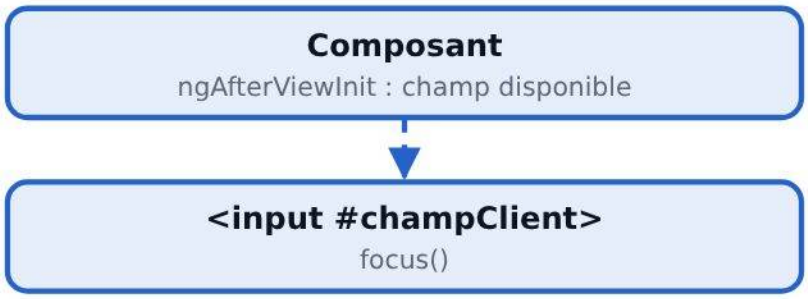


La réponse, étape par étape · *Step-by-step answer*

1. providedIn root : Angular crée un seul FactureService pour toute l'application.
   providedIn root: Angular creates a single FactureService for the whole application.
2. Un composant le demande avec inject()…
   One component asks for it with inject()…
3. …un autre par son constructeur. Les deux reçoivent la même instance, comme un singleton Spring.
   …another through its constructor. Both receive the same instance, like a Spring singleton.

@ViewChild

@ViewChild



La réponse, étape par étape · *Step-by-step answer*

1. #champClient pose une étiquette sur l'élément du template.
   #champClient puts a label on the template element.
2. Dans le constructeur, la vue n'existe pas encore : impossible d'y accéder.
   In the constructor, the view does not exist yet: impossible to access it.
3. Dans ngAfterViewInit, la vue est prête : on peut placer le curseur dans le champ.
   In ngAfterViewInit, the view is ready: we can put the cursor in the field.

Erreurs HTTP

HTTP errors

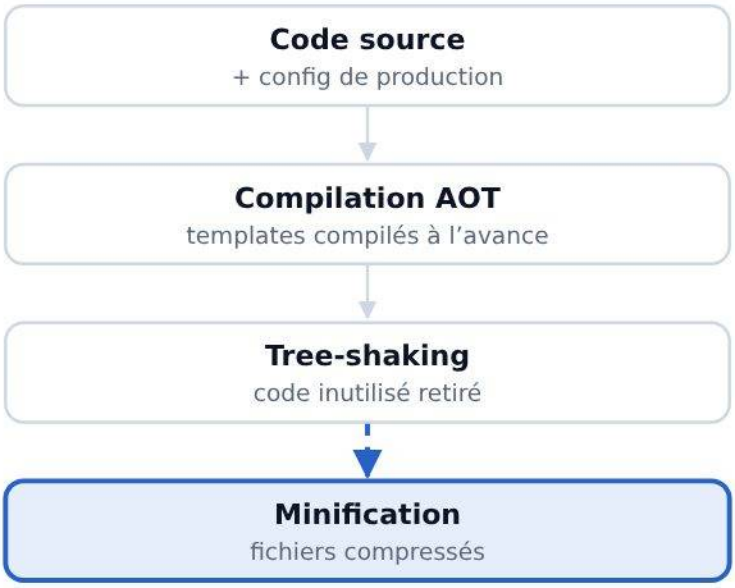


La réponse, étape par étape · *Step-by-step answer*

1. On demande la facture au serveur.
   We ask the server for the invoice.
2. Le serveur répond par une erreur.
   The server responds with an error.
3. catchError attrape l’erreur : on prépare un message et une valeur de repli.
   catchError catches the error: we prepare a message and a fallback value.
4. L’écran affiche un message clair au lieu de planter.
   The screen shows a clear message instead of crashing.

Build de production

Production build

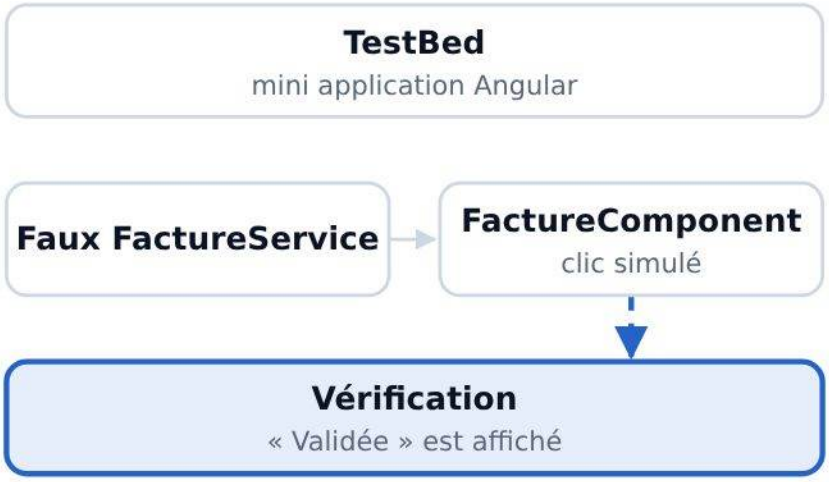


La réponse, étape par étape · *Step-by-step answer*

1. Le code de l'application, avec la config de production (adresse de l'API).
   The application code, with the production config (the API address).
2. Les templates sont compilés à l'avance : le navigateur n'a plus ce travail à faire.
   Templates are compiled ahead of time: the browser no longer has to do this work.
3. Le code jamais utilisé est supprimé.
   Code that is never used is removed.
4. Les noms sont raccourcis et les espaces retirés : des fichiers beaucoup plus légers.
   Names are shortened and spaces removed: much lighter files.

Tester un composant

Testing a component

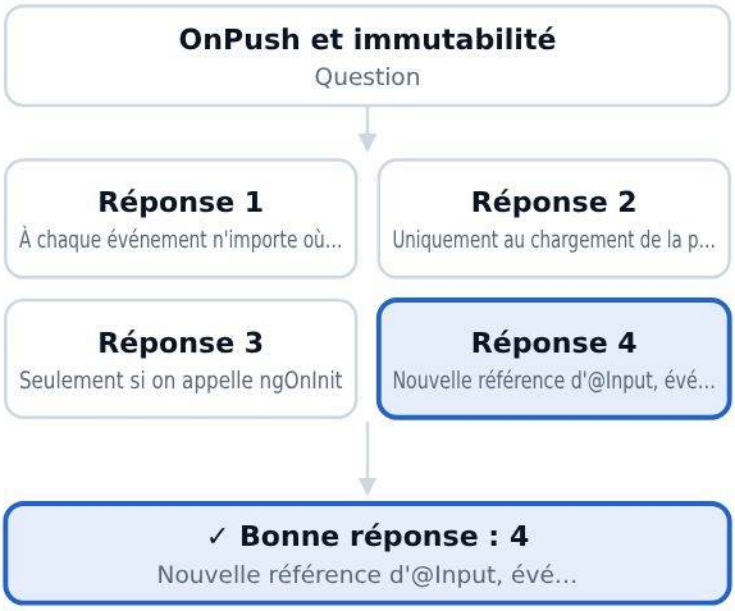


La réponse, étape par étape · *Step-by-step answer*

1. TestBed prépare un mini environnement Angular juste pour ce composant.
   TestBed sets up a mini Angular environment just for this component.
2. On remplace le vrai service par un faux : pas d'appel HTTP réel.
   We replace the real service with a fake one: no real HTTP call.
3. On crée le composant, on simule un clic et on met l'affichage à jour.
   We create the component, simulate a click and update the display.
4. On vérifie que le texte attendu est bien affiché.
   We check that the expected text is actually displayed.

OnPush et immutabilité

OnPush and immutability

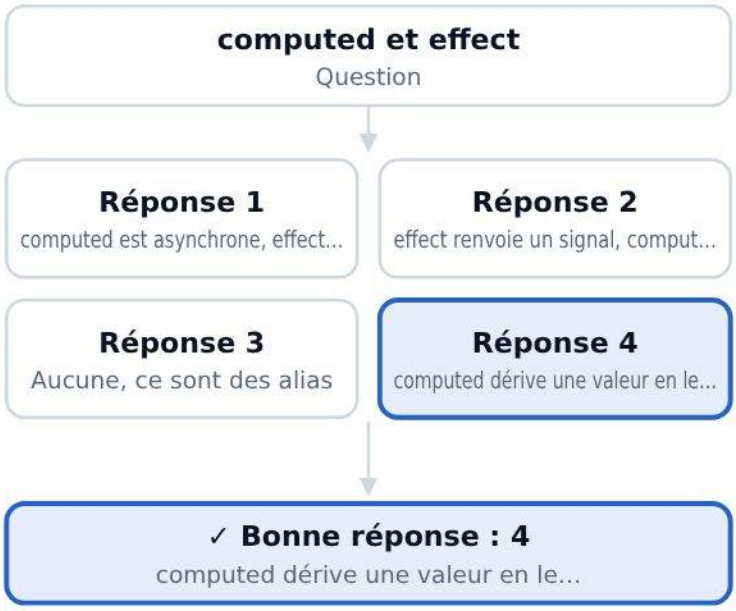


La réponse, étape par étape · *Step-by-step answer*

1. La question : Avec ChangeDetectionStrategy.OnPush, quand le composant est-il revérifié ?
   The question: With ChangeDetectionStrategy.OnPush, when is the component checked again?
2. Les réponses possibles : 1. À chaque événement n'importe où dans l'application · 2. Uniquement au chargement de la page · 3. Seulement si on appelle ngOnInit · 4. Nouvelle référence d'@Input, événement dans le composant, async pipe ou signal mis à jour
   Possible answers: 1. On any event anywhere in the app · 2. Only when the page loads · 3. Only when ngOnInit is called · 4. New @Input reference, event in the component, async pipe or updated signal
3. La bonne réponse est la 4 : Nouvelle référence d'@Input, événement dans le composant, async pipe ou signal mis à jour.
   The correct answer is 4: New @Input reference, event in the component, async pipe or updated signal.
4. OnPush ignore les mutations internes d'un objet : il faut une nouvelle référence, un événement local, l'async pipe ou un signal.
   OnPush ignores in-place mutations: it needs a new reference, a local event, the async pipe or a signal.

computed et effect

computed and effect

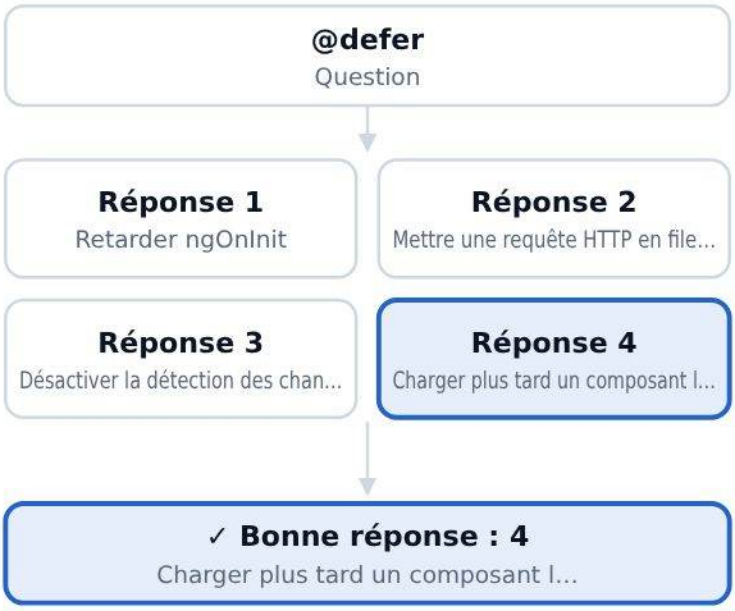


La réponse, étape par étape · *Step-by-step answer*

1. La question : Quelle différence entre computed() et effect() avec les signals ?
   The question: What is the difference between computed() and effect() with signals?
2. Les réponses possibles : 1. computed est asynchrone, effect synchrone · 2. effect renvoie un signal, computed non · 3. Aucune, ce sont des alias · 4. computed dérive une valeur en lecture seule ; effect exécute un effet de bord
   Possible answers: 1. computed is async, effect sync · 2. effect returns a signal, computed does not · 3. None, they are aliases · 4. computed derives a read-only value; effect runs a side effect
3. La bonne réponse est la 4 : computed dérive une valeur en lecture seule ; effect exécute un effet de bord.
   The correct answer is 4: computed derives a read-only value; effect runs a side effect.
4. computed calcule une valeur mise en cache à partir d'autres signals ; effect sert aux effets de bord (logs, localStorage), pas à dériver un état.
   computed caches a value derived from other signals; effect is for side effects (logs, localStorage), not for deriving state.

@defer

@defer

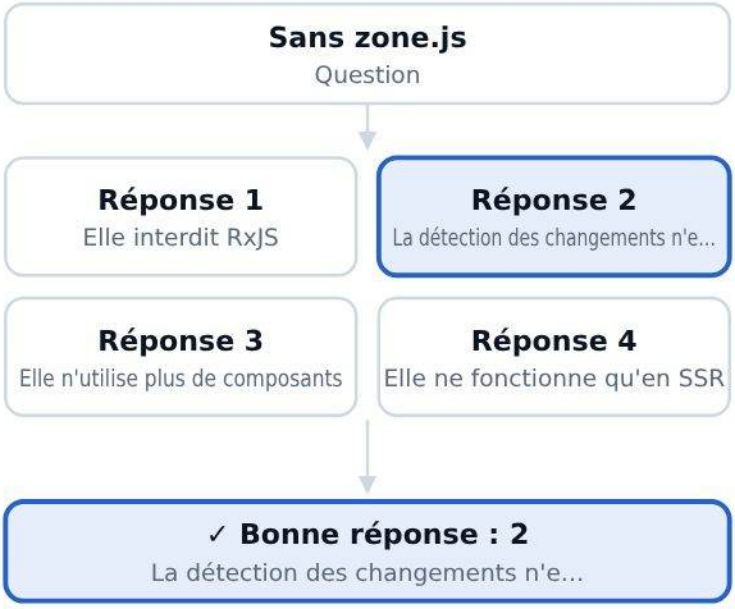


La réponse, étape par étape · *Step-by-step answer*

1. La question : À quoi sert le bloc @defer (Angular 17+) dans un template ?
   The question: What is the @defer block (Angular 17+) used for in a template?
2. Les réponses possibles : 1. Retarder ngOnInit · 2. Mettre une requête HTTP en file d'attente · 3. Désactiver la détection des changements · 4. Charger plus tard un composant lourd (à l'affichage, au clic...) dans un bundle séparé
   Possible answers: 1. Delay ngOnInit · 2. Queue an HTTP request · 3. Disable change detection · 4. Load a heavy component later (on view, on click...) in a separate bundle
3. La bonne réponse est la 4 : Charger plus tard un composant lourd (à l'affichage, au clic...) dans un bundle séparé.
   The correct answer is 4: Load a heavy component later (on view, on click...) in a separate bundle.
4. @defer crée un chunk à part et ne le charge qu'au déclencheur choisi (on viewport, on interaction, on idle), avec @placeholder et @loading.
   @defer creates a separate chunk and loads it on the chosen trigger (on viewport, on interaction, on idle), with @placeholder and @loading.

Sans zone.js

Zoneless

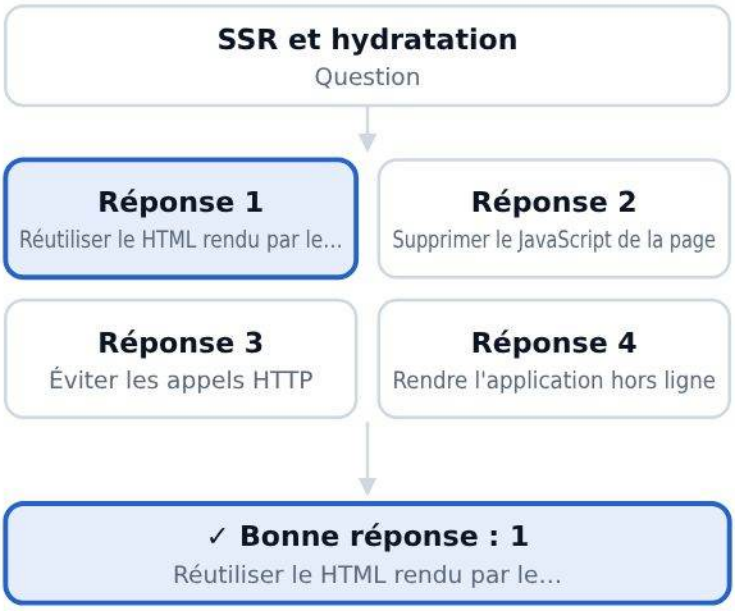


La réponse, étape par étape · *Step-by-step answer*

1. La question : Que change une application Angular « zoneless » ?
   The question: What changes in a zoneless Angular application?
2. Les réponses possibles : 1. Elle interdit RxJS · 2. La détection des changements n'est plus déclenchée par zone.js mais par les signals, l'async pipe et les événements du template · 3. Elle n'utilise plus de composants · 4. Elle ne fonctionne qu'en SSR
   Possible answers: 1. It forbids RxJS · 2. Change detection is no longer triggered by zone.js but by signals, the async pipe and template events · 3. It no longer uses components · 4. It only works with SSR
3. La bonne réponse est la 2 : La détection des changements n'est plus déclenchée par zone.js mais par les signals, l'async pipe et les événements du template.
   The correct answer is 2: Change detection is no longer triggered by zone.js but by signals, the async pipe and template events.
4. Sans zone.js, Angular ne patche plus setTimeout ni les promesses : il réagit aux signals et notifications explicites, avec moins de vérifications inutiles.
   Without zone.js, Angular stops patching setTimeout and promises: it reacts to signals and explicit notifications, with fewer useless checks.

SSR et hydratation

SSR and hydration



La réponse, étape par étape · *Step-by-step answer*

1. La question : Quel est l'intérêt de l'hydratation avec le rendu côté serveur (SSR) ?
   The question: What is the benefit of hydration with server-side rendering (SSR)?
2. Les réponses possibles : 1. Réutiliser le HTML rendu par le serveur au lieu de tout reconstruire côté client · 2. Supprimer le JavaScript de la page · 3. Éviter les appels HTTP · 4. Rendre l'application hors ligne
   Possible answers: 1. Reuse the server-rendered HTML instead of rebuilding it on the client · 2. Remove JavaScript from the page · 3. Avoid HTTP calls · 4. Make the app work offline
3. La bonne réponse est la 1 : Réutiliser le HTML rendu par le serveur au lieu de tout reconstruire côté client.
   The correct answer is 1: Reuse the server-rendered HTML instead of rebuilding it on the client.
4. Le serveur envoie un HTML complet (rapide, bon pour le SEO) ; l'hydratation y rattache les composants Angular sans le redessiner.
   The server sends full HTML (fast, good for SEO); hydration attaches Angular components to it without redrawing.

track dans @for

track in @for

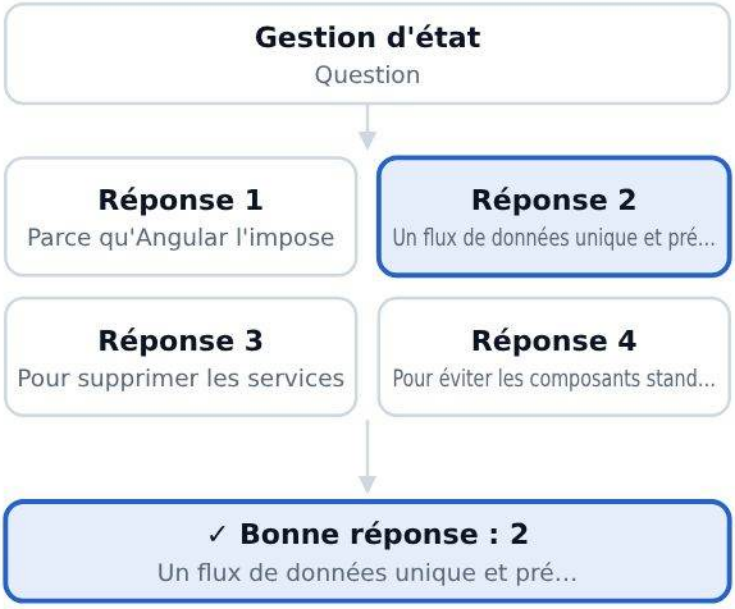


La réponse, étape par étape · *Step-by-step answer*

1. La question : Pourquoi l'expression track est-elle obligatoire dans @for ?
   The question: Why is the track expression required in @for?
2. Les réponses possibles : 1. Pour trier la liste · 2. Pour filtrer les doublons · 3. Pour paginer automatiquement · 4. Pour réutiliser les éléments DOM existants quand la liste change
   Possible answers: 1. To sort the list · 2. To filter duplicates · 3. To paginate automatically · 4. To reuse existing DOM elements when the list changes
3. La bonne réponse est la 4 : Pour réutiliser les éléments DOM existants quand la liste change.
   The correct answer is 4: To reuse existing DOM elements when the list changes.
4. track (ex. track item.id) permet à Angular de reconnaître chaque élément et de ne recréer que ceux qui ont changé.
   track (e.g. track item.id) lets Angular recognise each item and only recreate the ones that changed.

Gestion d'état

State management



La réponse, étape par étape · *Step-by-step answer*

1. La question : Dans une grosse application, pourquoi centraliser l'état (NgRx, SignalStore) ?
   The question: In a large application, why centralise state (NgRx, SignalStore)?
2. Les réponses possibles : 1. Parce qu'Angular l'impose · 2. Un flux de données unique et prévisible, facile à déboguer et à tester · 3. Pour supprimer les services · 4. Pour éviter les composants standalone
   Possible answers: 1. Because Angular requires it · 2. A single, predictable data flow that is easy to debug and test · 3. To remove services · 4. To avoid standalone components
3. La bonne réponse est la 2 : Un flux de données unique et prévisible, facile à déboguer et à tester.
   The correct answer is 2: A single, predictable data flow that is easy to debug and test.
4. Un store centralise l'état et ses changements (actions, méthodes) : on sait qui modifie quoi. Sur une petite application, des services avec signals suffisent.
   A store centralises state and its changes (actions, methods): you know who changes what. In a small app, services with signals are enough.

Observable ou Promise

Observable or Promise

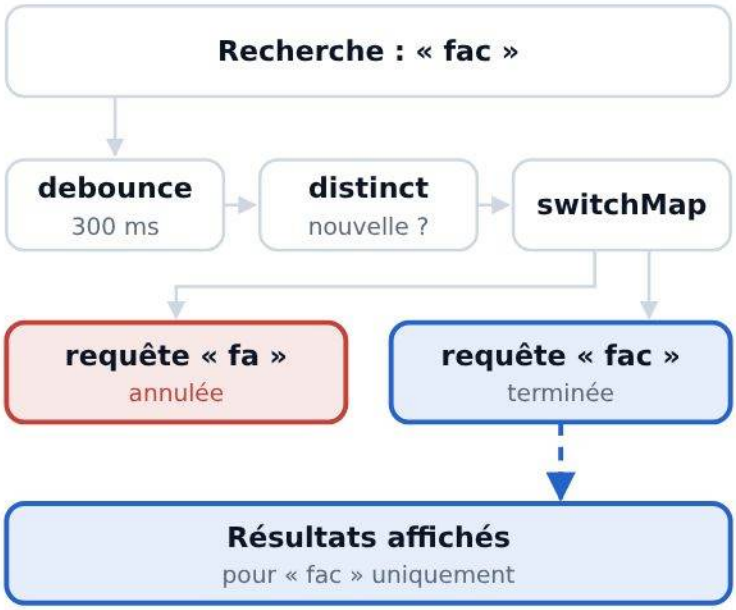


La réponse, étape par étape · *Step-by-step answer*

1. Une Promise démarre dès sa création.
   A Promise starts as soon as it is created.
2. Un Observable ne fait rien tant que personne ne s'abonne.
   An Observable does nothing as long as nobody subscribes.
3. subscribe le démarre.
   subscribe starts it.
4. Un Observable peut envoyer plusieurs valeurs dans le temps. Une Promise, une seule.
   An Observable can send several values over time. A Promise, only one.
5. Et on peut l'annuler avec unsubscribe. Une Promise, non.
   And it can be cancelled with unsubscribe. A Promise cannot.

switchMap

switchMap

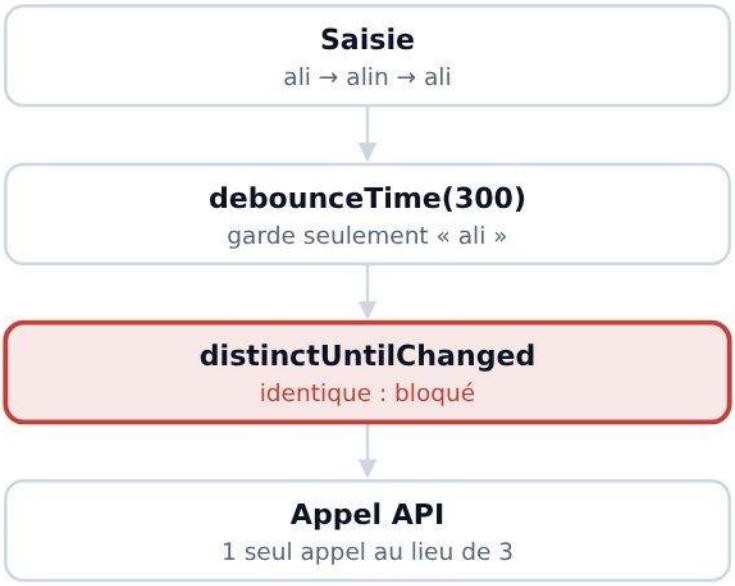


La réponse, étape par étape · *Step-by-step answer*

1. L'utilisateur tape « f » dans la barre de recherche.
   The user types "f" in the search bar.
2. Il tape vite « fa ». debounceTime attend une petite pause (300 ms) avant d'agir. « f » n'est donc jamais envoyé.
   They quickly type "fa". debounceTime waits for a short pause (300 ms) before acting. So "f" is never sent.
3. Pause terminée : la recherche « fa » est lancée.
   Pause over: the search for "fa" starts.
4. Pendant que « fa » cherche encore, l'utilisateur tape « fac ».
   While "fa" is still searching, the user types "fac".
5. switchMap annule l'ancienne recherche « fa » et lance « fac ». On garde seulement la plus récente.
   switchMap cancels the old "fa" search and starts "fac". We keep only the most recent one.
6. Seuls les résultats de « fac » s'affichent : c'est exactement ce que l'utilisateur veut voir.
   Only the "fac" results are shown: exactly what the user wants to see.

Autocomplétion

Autocomplete



La réponse, étape par étape · *Step-by-step answer*

1. L'utilisateur tape « ali » rapidement : 3 valeurs.
   The user quickly types "ali": 3 values.
2. debounceTime attend 300 ms de calme : seule la dernière valeur passe.
   debounceTime waits for 300 ms of calm: only the last value goes through.
3. distinctUntilChanged bloque une valeur identique à la précédente.
   distinctUntilChanged blocks a value identical to the previous one.
4. switchMap lance l'appel. Résultat : 1 requête au lieu de 3.
   switchMap makes the call. Result: 1 request instead of 3.
5. S'il tape une lettre puis l'efface, la valeur redevient « ali » : aucun appel inutile.
   If they type a letter then delete it, the value is "ali" again: no useless call.

Les Subjects

Subjects



La réponse, étape par étape · *Step-by-step answer*

1. Subject n'a pas de valeur au départ. BehaviorSubject en a toujours une.
   Subject has no value at the start. BehaviorSubject always has one.
2. Les deux envoient « Ali ».
   Both send "Ali".
3. Un abonné arrivé après coup ne reçoit rien du Subject.
   A subscriber that arrives later receives nothing from the Subject.
4. Avec BehaviorSubject, il reçoit tout de suite la dernière valeur : idéal pour l'utilisateur connecté.
   With BehaviorSubject, it immediately receives the last value: ideal for the logged-in user.
5. ReplaySubject rejoue les N dernières valeurs à chaque nouvel abonné.
   ReplaySubject replays the last N values to each new subscriber.

Fuites mémoire

Memory leaks

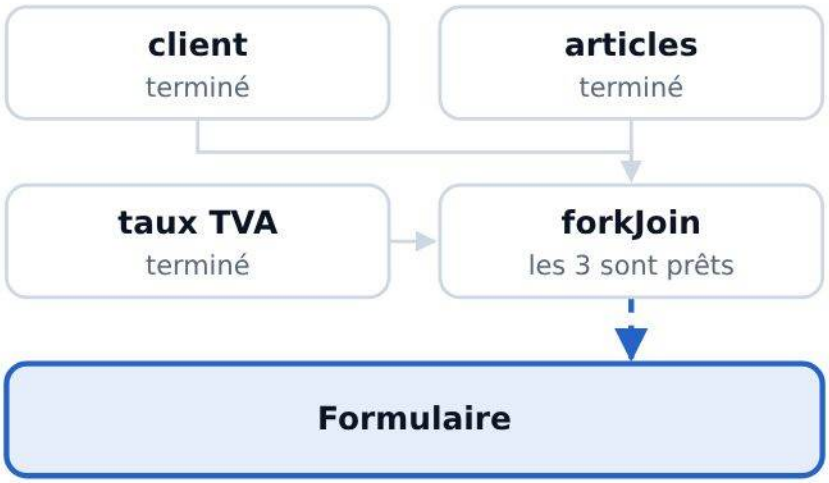


La réponse, étape par étape · *Step-by-step answer*

1. Deux abonnements, toutes les secondes.
   Two subscriptions, every second.
2. L'utilisateur quitte l'écran : le composant est détruit.
   The user leaves the screen: the component is destroyed.
3. Le premier continue pour rien. À chaque visite, un nouveau s'ajoute : fuite mémoire.
   The first one keeps running for nothing. On each visit, a new one is added: memory leak.
4. takeUntilDestroyed (ou le pipe async) arrête l'abonnement tout seul.
   takeUntilDestroyed (or the async pipe) stops the subscription on its own.

forkJoin

forkJoin



La réponse, étape par étape · *Step-by-step answer*

1. Trois appels partent en même temps.
   Three calls go out at the same time.
2. Deux sont terminés. forkJoin attend encore le dernier.
   Two are finished. forkJoin is still waiting for the last one.
3. Le dernier arrive : forkJoin envoie les 3 résultats ensemble.
   The last one arrives: forkJoin sends the 3 results together.
4. On ouvre le formulaire une seule fois, avec toutes les données.
   We open the form only once, with all the data.

catchError et retry

catchError and retry



La réponse, étape par étape · *Step-by-step answer*

1. Premier essai : échec.
   First attempt: failure.
2. retry(2) réessaie automatiquement…
   retry(2) automatically tries again…
3. …une deuxième fois. Toujours en échec.
   …a second time. Still failing.
4. catchError prend le relais avec une valeur de repli : l'écran continue de fonctionner.
   catchError takes over with a fallback value: the screen keeps working.

Froid ou chaud

Cold or hot

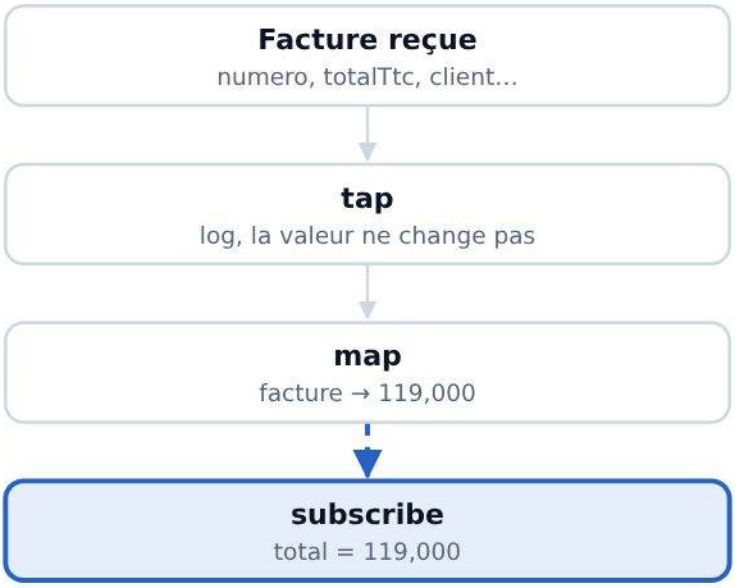


La réponse, étape par étape · *Step-by-step answer*

1. Observable froid : le premier abonné déclenche une requête.
   Cold Observable: the first subscriber triggers a request.
2. Le deuxième abonné déclenche une autre requête, identique. Gaspillage.
   The second subscriber triggers another, identical request. Waste.
3. Avec shareReplay(1), la requête est partagée.
   With shareReplay(1), the request is shared.
4. Le deuxième abonné reçoit le résultat déjà obtenu, sans nouvel appel.
   The second subscriber receives the result already obtained, with no new call.

map ou tap

map or tap



La réponse, étape par étape · *Step-by-step answer*

1. Le serveur renvoie une facture complète.
   The server returns a full invoice.
2. tap fait une action à côté (un log) et laisse passer la facture telle quelle.
   tap does a side action (a log) and lets the invoice through as is.
3. map transforme : on ne garde que le total.
   map transforms: we keep only the total.
4. L'abonné reçoit directement le total.
   The subscriber receives the total directly.

subscribe imbriqués

Nested subscribes

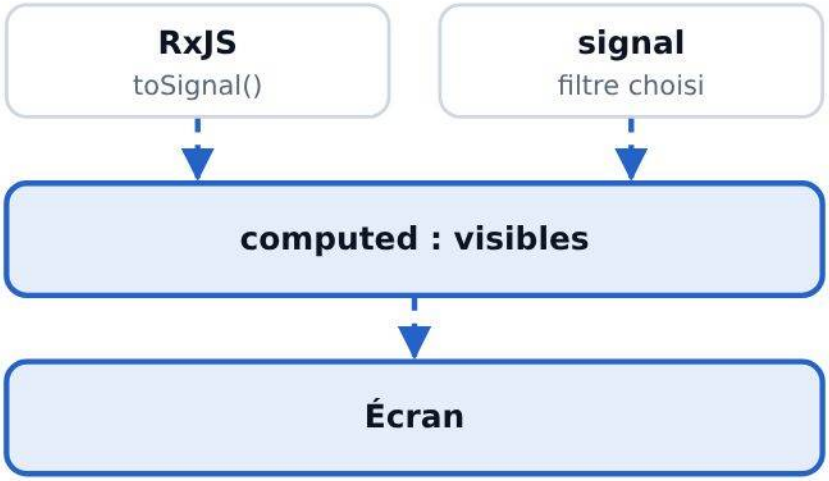


La réponse, étape par étape · *Step-by-step answer*

1. Un subscribe dans un subscribe : difficile à lire, et celui de l'intérieur n'est jamais annulé.
   A subscribe inside a subscribe: hard to read, and the inner one is never cancelled.
2. switchMap enchaîne les deux appels dans un seul flux.
   switchMap chains the two calls into a single stream.
3. Un seul abonnement, géré par le pipe async : rien à fermer à la main.
   A single subscription, managed by the async pipe: nothing to close by hand.

Signals ou RxJS

Signals or RxJS

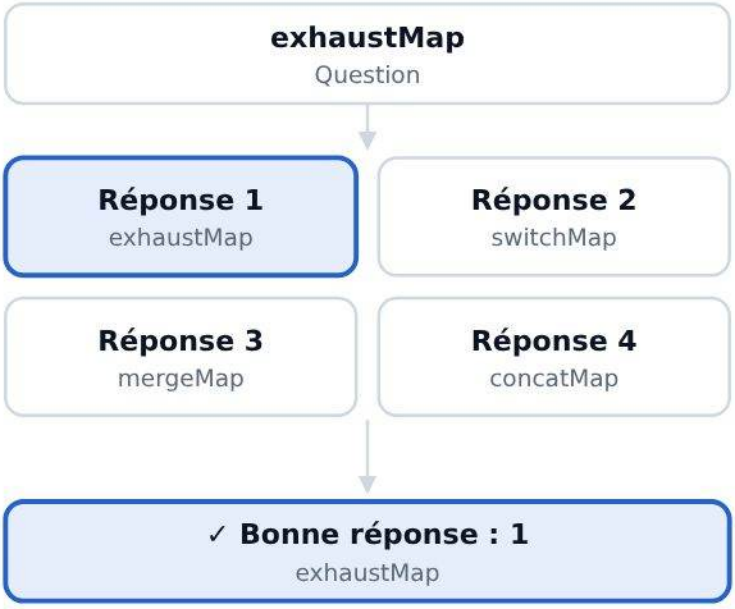


La réponse, étape par étape · *Step-by-step answer*

1. Les données arrivent du serveur : c'est un flux asynchrone, le domaine de RxJS.
   Data arrives from the server: it is an asynchronous stream, the domain of RxJS.
2. toSignal transforme ce flux en signal.
   toSignal turns this stream into a signal.
3. Le filtre choisi par l'utilisateur est un simple état : un signal suffit.
   The filter chosen by the user is simple state: a signal is enough.
4. computed combine les deux et l'écran se met à jour tout seul.
   computed combines the two and the screen updates on its own.

exhaustMap

exhaustMap

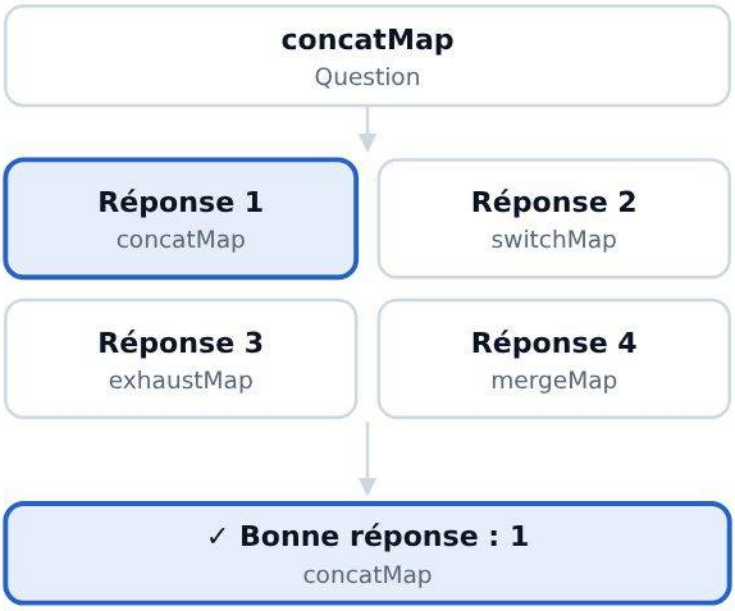


La réponse, étape par étape · *Step-by-step answer*

1. La question : Un utilisateur clique plusieurs fois sur « Payer ». Quel opérateur ignore les clics tant que la requête est en cours ?
   The question: A user clicks "Pay" several times. Which operator ignores clicks while the request is running?
2. Les réponses possibles : 1. exhaustMap · 2. switchMap · 3. mergeMap · 4. concatMap
   Possible answers: 1. exhaustMap · 2. switchMap · 3. mergeMap · 4. concatMap
3. La bonne réponse est la 1 : exhaustMap.
   The correct answer is 1: exhaustMap.
4. exhaustMap ignore les nouvelles valeurs tant que l'observable interne n'est pas terminé. switchMap annulerait la requête en cours.
   exhaustMap ignores new values until the inner observable completes. switchMap would cancel the running request.

concatMap

concatMap

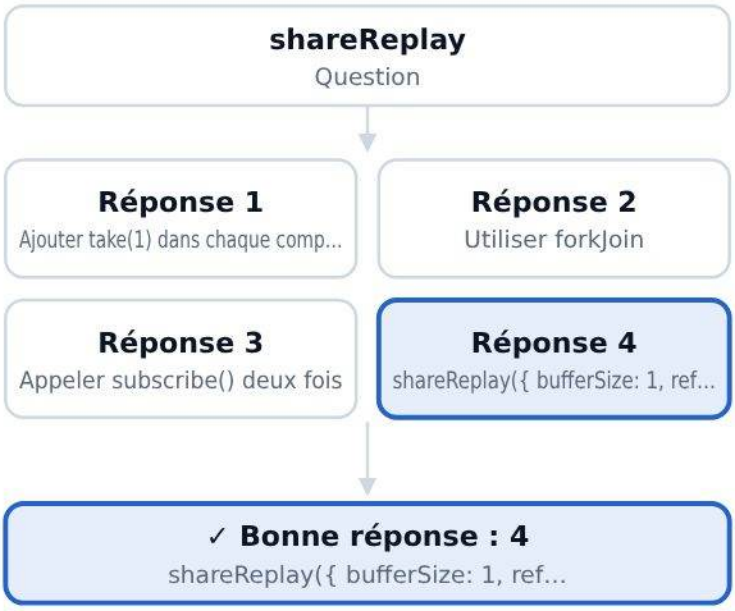


La réponse, étape par étape · *Step-by-step answer*

1. La question : Pour envoyer des sauvegardes dans l'ordre, une à la fois, sans en perdre aucune :
   The question: To send saves in order, one at a time, without losing any:
2. Les réponses possibles : 1. concatMap · 2. switchMap · 3. exhaustMap · 4. mergeMap
   Possible answers: 1. concatMap · 2. switchMap · 3. exhaustMap · 4. mergeMap
3. La bonne réponse est la 1 : concatMap.
   The correct answer is 1: concatMap.
4. concatMap met les valeurs en file et attend la fin de chaque requête avant la suivante.
   concatMap queues values and waits for each request to finish before the next one.

shareReplay

shareReplay



La réponse, étape par étape · *Step-by-step answer*

1. La question : Trois composants s'abonnent au même http.get(). Comment n'envoyer qu'une requête ?
   The question: Three components subscribe to the same http.get(). How do you send only one request?
2. Les réponses possibles : 1. Ajouter take(1) dans chaque composant · 2. Utiliser forkJoin · 3. Appeler subscribe() deux fois · 4. shareReplay({ bufferSize: 1, refCount: true })
   Possible answers: 1. Add take(1) in each component · 2. Use forkJoin · 3. Call subscribe() twice · 4. shareReplay({ bufferSize: 1, refCount: true })
3. La bonne réponse est la 4 : shareReplay({ bufferSize: 1, refCount: true }).
   The correct answer is 4: shareReplay({ bufferSize: 1, refCount: true }).
4. Un observable HTTP est froid : chaque abonnement relance la requête. shareReplay partage l'exécution et rejoue la dernière valeur.
   An HTTP observable is cold: each subscription restarts the request. shareReplay shares the execution and replays the last value.

combineLatest

combineLatest



La réponse, étape par étape · *Step-by-step answer*

1. La question : Quand combineLatest([filtre$, page$]) émet-il ?
   The question: When does combineLatest([filter$, page$]) emit?
2. Les réponses possibles : 1. Une seule fois, quand les deux sources sont terminées · 2. Uniquement quand les deux émettent en même temps · 3. À chaque valeur de filtre$ seulement · 4. Dès que chaque source a émis au moins une fois, puis à chaque nouvelle valeur de l'une d'elles
   Possible answers: 1. Only once, when both sources complete · 2. Only when both emit at the same time · 3. Only on each filter$ value · 4. Once every source has emitted at least once, then on each new value from any of them
3. La bonne réponse est la 4 : Dès que chaque source a émis au moins une fois, puis à chaque nouvelle valeur de l'une d'elles.
   The correct answer is 4: Once every source has emitted at least once, then on each new value from any of them.
4. combineLatest attend une première valeur de chaque source, puis réémet la combinaison à chaque changement. forkJoin, lui, attend la fin de toutes.
   combineLatest waits for a first value from each source, then re-emits the combination on every change. forkJoin waits for all to complete.

Retry avec délai

Retry with backoff



La réponse, étape par étape · *Step-by-step answer*

1. La question : Comment réessayer une requête 3 fois avec un délai croissant ?
   The question: How do you retry a request 3 times with a growing delay?
2. Les réponses possibles : 1. repeat(3) · 2. debounceTime(3000) · 3. retry({ count: 3, delay: (e, i) => timer(i * 1000) }) · 4. catchError(() => of(null))
   *Possible answers: 1. repeat(3) · 2. debounceTime(3000) · 3. retry({ count: 3, delay: (e, i) => timer(i * 1000) }) · 4. catchError(() => of(null))*
3. La bonne réponse est la 3 : retry({ count: 3, delay: (e, i) => timer(i * 1000) }).
   *The correct answer is 3: retry({ count: 3, delay: (e, i) => timer(i * 1000) }).*
4. retry accepte un délai calculé par tentative (backoff). repeat relance même en cas de succès ; catchError seul ne réessaie pas.
   retry accepts a per-attempt delay (backoff). repeat re-runs even on success; catchError alone does not retry.

API REST

REST API



La réponse, étape par étape · *Step-by-step answer*

1. Chaque ressource a une adresse. GET /api/factures renvoie la liste en JSON.
   Each resource has an address. GET /api/factures returns the list as JSON.
2. Le verbe dit l'action, l'adresse dit sur quoi.
   The verb says the action, the address says on what.
3. Sans état : chaque requête contient tout ce qu'il faut (dont le token). Le serveur ne se souvient de rien entre deux appels.
   Stateless: each request contains everything needed (including the token). The server remembers nothing between two calls.

Verbes HTTP

HTTP verbs

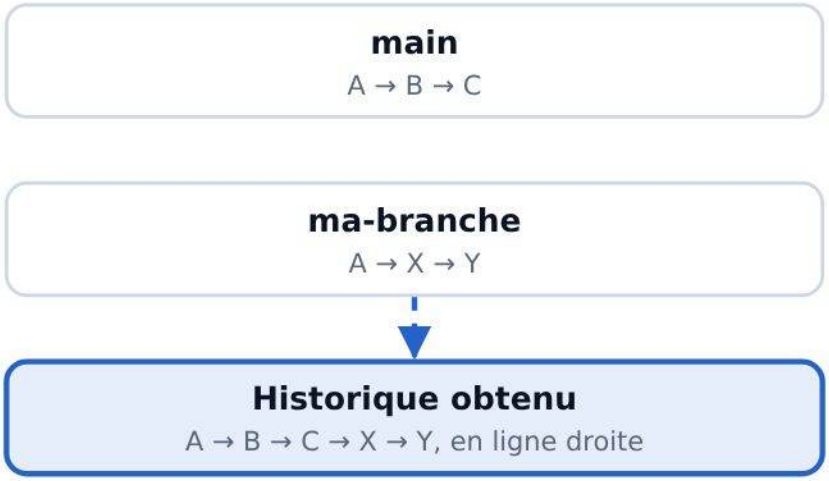


La réponse, étape par étape · *Step-by-step answer*

1. PUT est idempotent : le répéter donne le même état.
   PUT is idempotent: repeating it gives the same state.
2. DELETE aussi : la facture reste supprimée.
   So is DELETE: the invoice stays deleted.
3. POST n'est pas idempotent : un double clic crée deux factures. D'où l'intérêt de désactiver le bouton.
   POST is not idempotent: a double click creates two invoices. Hence the point of disabling the button.
4. PATCH modifie une partie. PUT remplace la ressource entière.
   PATCH modifies a part. PUT replaces the entire resource.

merge ou rebase

merge or rebase



La réponse, étape par étape · *Step-by-step answer*

1. main a avancé (B, C) pendant que tu travaillais (X, Y).
   main moved forward (B, C) while you were working (X, Y).
2. merge réunit les deux avec un commit de fusion : l'historique réel est conservé.
   merge joins the two with a merge commit: the real history is kept.
3. rebase rejoue tes commits après C : historique en ligne droite. À éviter sur une branche déjà partagée.
   rebase replays your commits after C: a straight-line history. Avoid it on a branch that is already shared.

Conflit Git

Git conflict

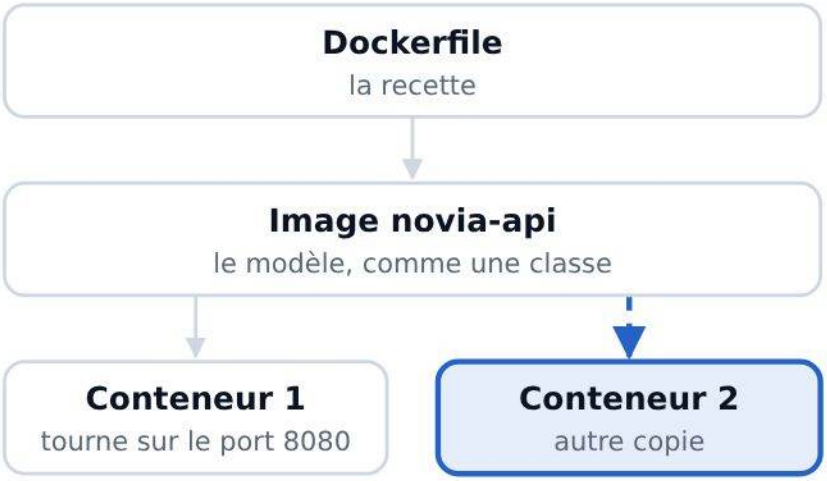


La réponse, étape par étape · *Step-by-step answer*

1. Deux personnes ont modifié la même ligne : Git ne peut pas choisir seul.
   Two people changed the same line: Git cannot choose on its own.
2. On compare les deux versions, et on vérifie avec le collègue ou le besoin laquelle est juste.
   We compare the two versions, and check with the colleague or the requirement which one is right.
3. On garde la bonne version et on supprime les lignes <<<<, ==== et >>>>.
   We keep the right version and delete the <<<<, ==== and >>>> lines.
4. On teste, puis git add et commit pour terminer.
   We test, then git add and commit to finish.

Image et conteneur

Image and container

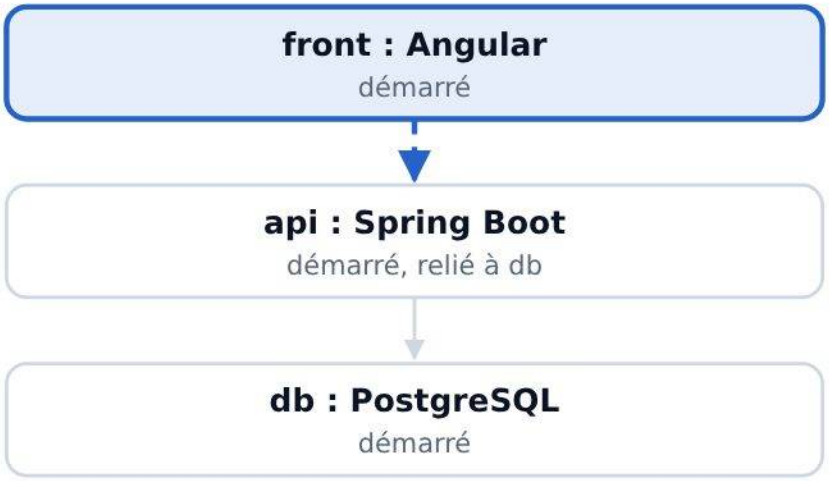


La réponse, étape par étape · *Step-by-step answer*

1. Le Dockerfile est la recette : partir de Java 21, copier le jar, le lancer.
   The Dockerfile is the recipe: start from Java 21, copy the jar, run it.
2. docker build fabrique l'image : un paquet figé qui contient tout.
   docker build makes the image: a frozen package that contains everything.
3. docker run crée un conteneur : l'image qui tourne, comme un objet créé depuis une classe.
   docker run creates a container: the running image, like an object created from a class.
4. On peut lancer plusieurs conteneurs à partir de la même image.
   We can start several containers from the same image.

Docker Compose

Docker Compose



La réponse, étape par étape · *Step-by-step answer*

1. Un seul fichier décrit les trois services.
   A single file describes the three services.
2. docker compose up démarre d'abord la base…
   docker compose up first starts the database…
3. …puis l'API, qui dépend de la base…
   …then the API, which depends on the database…
4. …puis le front. Tout l'environnement tourne avec une seule commande.
   …then the front end. The whole environment runs with a single command.

Monolithe ou microservices

Monolith or microservices

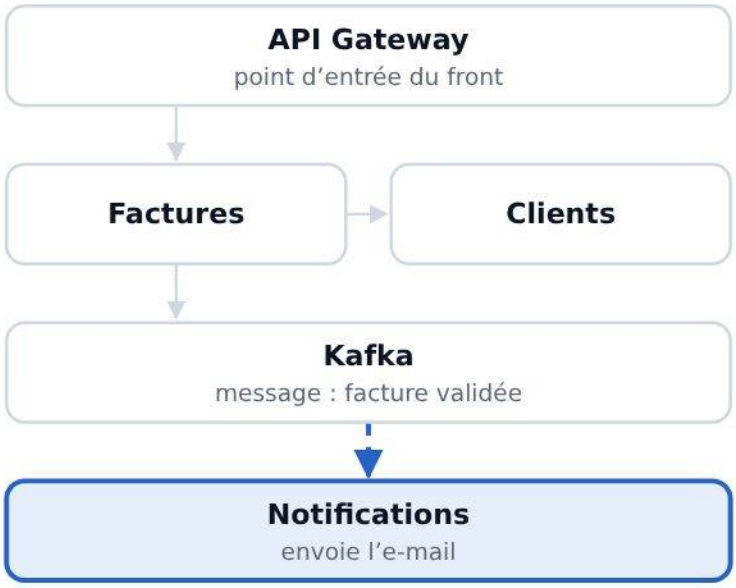


La réponse, étape par étape · *Step-by-step answer*

1. Monolithe : tout est dans une seule application. Simple à développer, tester et déployer.
   Monolith: everything is in a single application. Simple to develop, test and deploy.
2. Microservices : chaque partie est une application séparée.
   Microservices: each part is a separate application.
3. Avantage : on peut modifier et déployer la paie sans toucher aux factures.
   Advantage: we can change and deploy payroll without touching invoices.
4. Inconvénient : réseau, plusieurs bases, surveillance… beaucoup plus complexe. On commence souvent par un monolithe bien organisé.
   Drawback: network, several databases, monitoring… much more complex. We often start with a well-organized monolith.

Communication

Communication

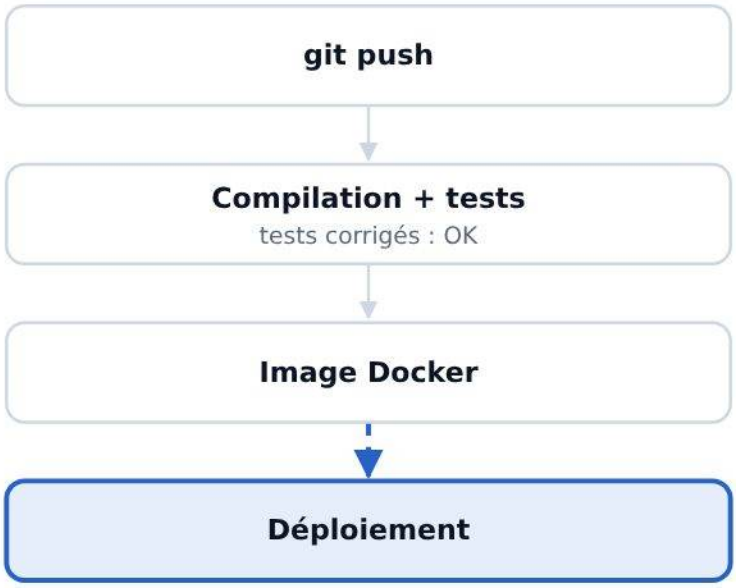


La réponse, étape par étape · *Step-by-step answer*

1. Le front passe par la Gateway, qui envoie vers le bon service.
   The front end goes through the Gateway, which routes to the right service.
2. REST : Factures demande le client et attend la réponse.
   REST: Factures asks for the customer and waits for the answer.
3. Message : Factures publie « facture validée » et continue sans attendre.
   Message: Factures publishes "invoice validated" and continues without waiting.
4. Notifications lit le message quand il peut. Si ce service est en panne, le message attend.
   Notifications reads the message when it can. If this service is down, the message waits.

CI/CD

CI/CD

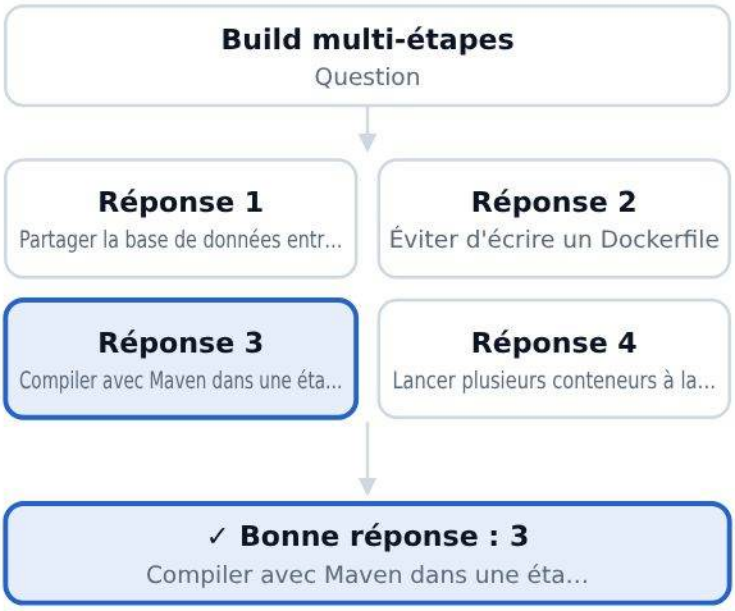


La réponse, étape par étape · *Step-by-step answer*

1. Tu envoies ton code.
   You push your code.
2. CI : le serveur compile et lance tous les tests automatiquement.
   CI: the server compiles and runs all the tests automatically.
3. Si un test échoue, tout s'arrête et tu es prévenu. Rien de cassé n'arrive en production.
   If a test fails, everything stops and you are notified. Nothing broken reaches production.
4. Une fois les tests verts, l'image Docker est construite.
   Once the tests are green, the Docker image is built.
5. CD : la nouvelle version est déployée automatiquement.
   CD: the new version is deployed automatically.

Build multi-étapes

Multi-stage build



La réponse, étape par étape · *Step-by-step answer*

1. La question : Pourquoi utiliser un Dockerfile multi-étapes pour une application Spring Boot ?
   The question: Why use a multi-stage Dockerfile for a Spring Boot application?
2. Les réponses possibles : 1. Partager la base de données entre étapes · 2. Éviter d'écrire un Dockerfile · 3. Compiler avec Maven dans une étape, ne garder que le JAR et un JRE dans l'image finale · 4. Lancer plusieurs conteneurs à la fois
   Possible answers: 1. Share the database between stages · 2. Avoid writing a Dockerfile · 3. Build with Maven in one stage, keep only the JAR and a JRE in the final image · 4. Run several containers at once
3. La bonne réponse est la 3 : Compiler avec Maven dans une étape, ne garder que le JAR et un JRE dans l'image finale.
   The correct answer is 3: Build with Maven in one stage, keep only the JAR and a JRE in the final image.
4. L'image finale ne contient ni Maven ni les sources : elle est plus petite et expose moins de failles.
   The final image has neither Maven nor the sources: it is smaller and exposes fewer vulnerabilities.

Versionner une API

API versioning

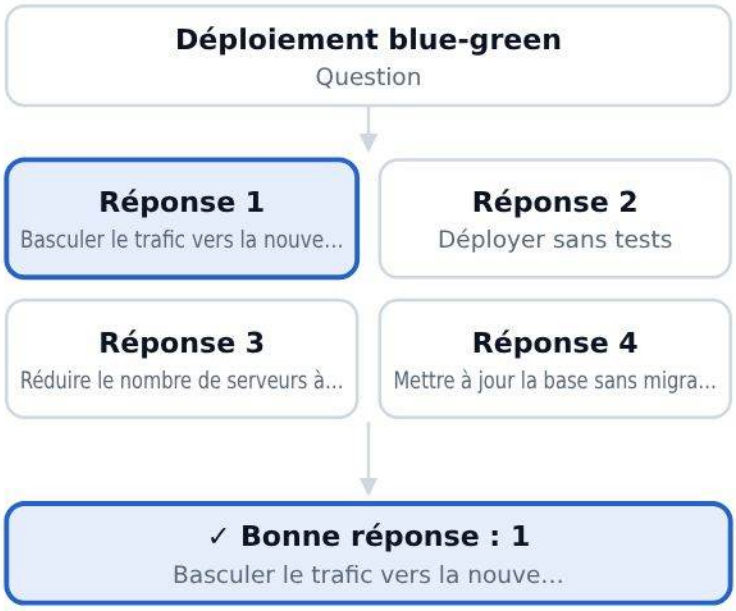


La réponse, étape par étape · *Step-by-step answer*

1. La question : On doit changer le format d'une réponse utilisée par des clients existants. Bonne pratique ?
   The question: You must change a response format used by existing clients. Best practice?
2. Les réponses possibles : 1. Changer le verbe HTTP · 2. Renvoyer un code 500 aux anciens clients · 3. Publier une nouvelle version (/api/v2) et garder la v1 un temps · 4. Modifier la v1 directement
   Possible answers: 1. Change the HTTP verb · 2. Return a 500 to old clients · 3. Publish a new version (/api/v2) and keep v1 for a while · 4. Change v1 directly
3. La bonne réponse est la 3 : Publier une nouvelle version (/api/v2) et garder la v1 un temps.
   The correct answer is 3: Publish a new version (/api/v2) and keep v1 for a while.
4. Un changement non rétrocompatible casse les clients : on versionne et on annonce la fin de la v1.
   A breaking change breaks clients: version the API and announce v1's end of life.

Déploiement blue-green

Blue-green deployment

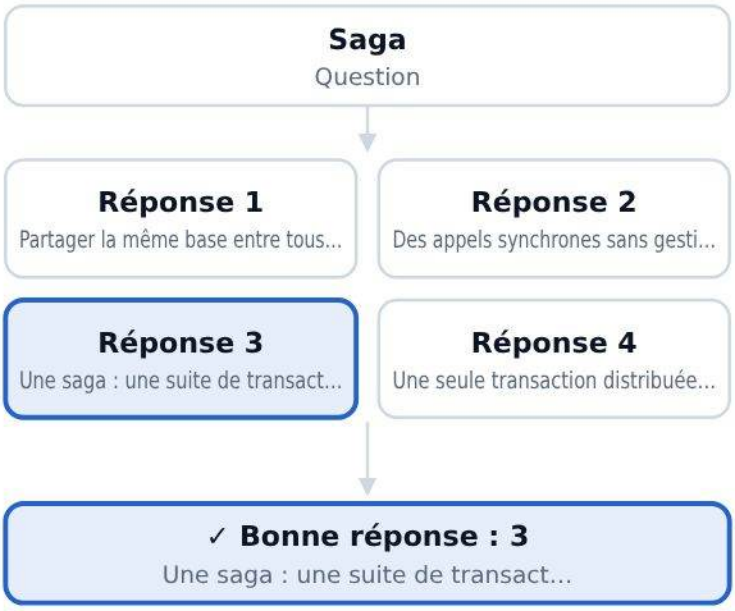


La réponse, étape par étape · *Step-by-step answer*

1. La question : Quel est l'intérêt d'un déploiement blue-green ?
   The question: What is the benefit of a blue-green deployment?
2. Les réponses possibles : 1. Basculer le trafic vers la nouvelle version d'un coup, avec retour arrière immédiat · 2. Déployer sans tests · 3. Réduire le nombre de serveurs à un seul · 4. Mettre à jour la base sans migration
   Possible answers: 1. Switch traffic to the new version at once, with instant rollback · 2. Deploy without tests · 3. Reduce to a single server · 4. Update the database without migrations
3. La bonne réponse est la 1 : Basculer le trafic vers la nouvelle version d'un coup, avec retour arrière immédiat.
   The correct answer is 1: Switch traffic to the new version at once, with instant rollback.
4. Deux environnements identiques : on prépare « green », on bascule le trafic, et on revient à « blue » en cas de problème.
   Two identical environments: prepare green, switch traffic, and switch back to blue if something goes wrong.

Saga

Saga

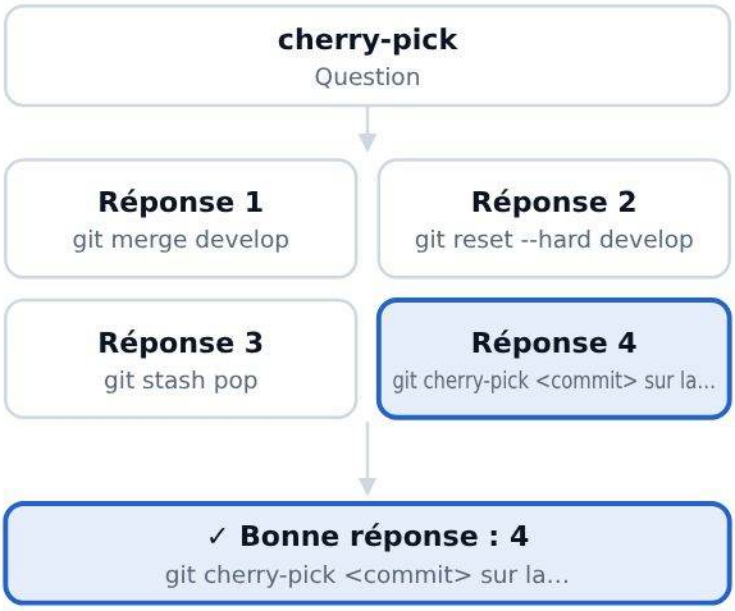


La réponse, étape par étape · *Step-by-step answer*

1. La question : Dans des microservices, comment garder la cohérence d'une commande qui touche stock, paiement et livraison ?
   The question: In microservices, how do you keep an order consistent across stock, payment and delivery?
2. Les réponses possibles : 1. Partager la même base entre tous les services · 2. Des appels synchrones sans gestion d'erreur · 3. Une saga : une suite de transactions locales avec des actions de compensation · 4. Une seule transaction distribuée sur toutes les bases
   Possible answers: 1. Share the same database across all services · 2. Synchronous calls with no error handling · 3. A saga: a sequence of local transactions with compensating actions · 4. One distributed transaction across all databases
3. La bonne réponse est la 3 : Une saga : une suite de transactions locales avec des actions de compensation.
   The correct answer is 3: A saga: a sequence of local transactions with compensating actions.
4. Chaque service valide sa transaction locale ; en cas d'échec, les étapes déjà faites sont annulées par compensation (remboursement, remise en stock).
   Each service commits its local transaction; on failure, completed steps are undone by compensation (refund, restock).

cherry-pick

cherry-pick

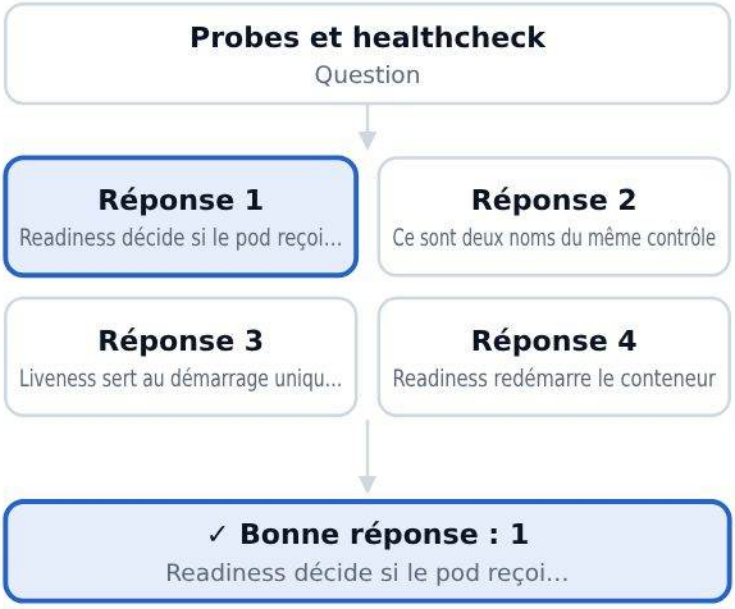


La réponse, étape par étape · *Step-by-step answer*

1. La question : Un correctif a été fait sur develop et doit partir en production sans le reste de develop. Commande Git adaptée ?
   The question: A fix was made on develop and must go to production without the rest of develop. Which Git command?
2. Les réponses possibles : 1. git merge develop · 2. git reset --hard develop · 3. git stash pop · 4. git cherry-pick <commit> sur la branche de production
   Possible answers: 1. git merge develop · 2. git reset --hard develop · 3. git stash pop · 4. git cherry-pick <commit> on the production branch
3. La bonne réponse est la 4 : git cherry-pick <commit> sur la branche de production.
   The correct answer is 4: git cherry-pick <commit> on the production branch.
4. cherry-pick applique un commit précis sur la branche courante, sans importer les autres changements.
   cherry-pick applies one specific commit to the current branch, without bringing other changes.

Probes et healthcheck

Probes and healthcheck



La réponse, étape par étape · *Step-by-step answer*

1. La question : Dans Kubernetes, quelle différence entre liveness et readiness probe ?
   The question: In Kubernetes, what is the difference between liveness and readiness probes?
2. Les réponses possibles : 1. Readiness décide si le pod reçoit du trafic ; liveness décide s'il faut le redémarrer · 2. Ce sont deux noms du même contrôle · 3. Liveness sert au démarrage uniquement · 4. Readiness redémarre le conteneur
   Possible answers: 1. Readiness decides if the pod gets traffic; liveness decides if it must be restarted · 2. They are two names for the same check · 3. Liveness is only for startup · 4. Readiness restarts the container
3. La bonne réponse est la 1 : Readiness décide si le pod reçoit du trafic ; liveness décide s'il faut le redémarrer.
   The correct answer is 1: Readiness decides if the pod gets traffic; liveness decides if it must be restarted.
4. Un pod pas encore prêt (cache en chargement) est retiré du load balancer ; un pod bloqué (liveness en échec) est redémarré. Spring Actuator expose les deux.
   A pod not ready yet (cache loading) is removed from the load balancer; a stuck pod (failing liveness) is restarted. Spring Actuator exposes both.

Limitation de débit

Rate limiting



La réponse, étape par étape · *Step-by-step answer*

1. La question : Quel code HTTP renvoyer quand un client dépasse sa limite d'appels ?
   The question: Which HTTP status should you return when a client exceeds its rate limit?
2. Les réponses possibles : 1. 429 Too Many Requests, avec un en-tête Retry-After · 2. 403 Forbidden · 3. 503 Service Unavailable · 4. 400 Bad Request
   Possible answers: 1. 429 Too Many Requests, with a Retry-After header · 2. 403 Forbidden · 3. 503 Service Unavailable · 4. 400 Bad Request
3. La bonne réponse est la 1 : 429 Too Many Requests, avec un en-tête Retry-After.
   The correct answer is 1: 429 Too Many Requests, with a Retry-After header.
4. 429 signale un dépassement de quota ; Retry-After indique quand réessayer. 503 concerne une indisponibilité du serveur.
   429 signals an exceeded quota; Retry-After says when to retry. 503 is for server unavailability.